KALDRIVON

Kaldrivon MiniSMO

User Guide

Product version	**0.6.0**
Documentation version	**0.6.0**
Platform	**Windows 11 x64**
Published	**September 2026**

© 2026 Kaldrivon. All rights reserved. https://kaldrivon.com
Kaldrivon MiniSMO is an independent product and is not represented as certified or endorsed by the O-RAN Alliance, 3GPP, ETSI, any network operator, or any equipment vendor. Demo Network data is synthetic.

Contents

1. Introduction
2. What Kaldrivon MiniSMO Is
3. Version 0.4 Scope
4. System Requirements
5. Installation
6. Communication Matrix
7. First Launch
8. Application Layout
9. Dashboard
10. Creating the Demo Network
11. Understanding the Demo Topology
12. Network Function Inventory
13. Adding an External NF
    - 13.1 Adding an External NF
    - 13.2 Example: Adding an External O-RU
    - 13.3 Identity Mapping
    - 13.4 NETCONF Call Home
    - 13.5 SSH Authentication Methods
    - 13.6 Confirming the SSH Host Key
14. NETCONF Connection Requirements
    - 14.1 NETCONF Connection Requirements
    - 14.2 What is NETCONF?
    - 14.3 Why Port 830?
    - 14.4 NETCONF over SSH
15. Testing an NF Connection
    - 15.1 NETCONF Connection Test
    - 15.2 The Hello Exchange
16. Understanding NETCONF Capabilities
17. Running GET
18. Running GET-CONFIG
    - 18.1 Running GET-CONFIG
    - 18.2 Running, Candidate and Startup Datastores
19. Viewing Raw NETCONF XML
20. NETCONF Log

21. Monitoring External NFs
21.1 Monitoring External NFs
21.2 Capability Discovery
21.3 Method Choice and Fallback
22. Write Protection
23. Subscription States and Gaps
24. YANG Library Discovery
25. The VES Collector
25.1 VES Collector
25.2 Matching Events to NFs
26. Pointing an NF at the VES Collector
27. Fault Management
27.1 Fault Management
27.2 Alarm History
27.3 About Simulated Alarms
28. External Alarms
29. Configuration Management
29.1 Configuration Management
29.2 How MiniSMO Writes to External NFs
30. External Configuration History
31. Changing External Configuration
32. Performance Management
32.1 Performance Management
32.2 About PM Simulation
33. External Performance Data
33.1 External Performance Data
33.2 Measurement Files (File-based PM)
34. Running Scenarios
35. Lab Simulator Checks
36. Topology View
37. Inventory, Search and Notifications
37.1 Inventory and Network Tree
37.2 NF Identity and Cells
37.3 Search
37.4 Notifications
38. YANG Model Registry
39. Importing a YANG Model
40. Editions and Capacity
40.1 Editions and Capacity
40.2 What a License Changes

41. Configuration Operations
41.1 Configuration Backups
41.2 Comparing Configurations
41.3 Configuration Templates
41.4 Desired State and Drift
41.5 Rollback
41.6 Bulk Operations
42. Automation
42.1 Automation
42.2 Approvals
42.3 Schedules
43. Analytics and Troubleshooting
43.1 Analytics
43.2 Investigate
43.3 Diagnostic Collection
44. Reports and System Health
45. Integrations
45.1 Northbound REST API
45.2 O-RAN R1 Interface
45.3 Webhooks
46. Software Campaigns and Audit
46.1 Software Upgrade Campaigns
46.2 Audit Log
47. Community Edition and Licensing
48. Installing a License
49. Understanding External NF Limits
49.1 External NF Capacity and License Limits
49.2 Expired License, Downgrades and Unlicensed NFs
50. System Capacity Assessment
50.1 Capacity & Storage Overview
50.2 Running a Capacity Test
50.3 Understanding Technical Capacity
51. Storage Management
52. Understanding MiniSMO Data Storage
53. PM Data Retention
53.1 PM Retention
53.2 PM Aggregation
53.3 Why PM Data Can Grow Quickly
54. FM History Retention
55. NETCONF History Retention
56. Configuration History
57. Automatic Cleanup
58. Manual Cleanup

59. Disk Space Warnings
60. Capacity vs Storage Constraints
60.1 License Capacity vs Technical Capacity
60.2 Capacity vs Storage Constraints
61. Estimating Storage Requirements
61.1 Storage Forecast
61.2 Changing Retention Settings
62. Exporting Capacity Reports
63. Exporting Storage Reports
64. Settings
65. Logs
66. Data Storage Locations
67. Backup Considerations
68. Team Mode and Web Access
68.1 Local and Team Mode
68.2 Users and Roles
68.3 Using MiniSMO in a Browser
68.4 Running as a Windows Service
68.5 Running Without the Desktop App
69. Software Updates
70. Resetting and Uninstalling
70.1 Reset MiniSMO
70.2 Uninstalling
71. Troubleshooting
71.1 Troubleshooting
71.2 Backend Startup Problems
71.3 Local API Protection
72. NETCONF Troubleshooting
73. Licensing Troubleshooting
74. Common Error Messages
75. Security Notes
76. Known Limitations
77. Glossary
78. Support Information
78.1 Support Information
78.2 About MiniSMO
Keyword Index

1. Introduction

This guide covers Kaldrivon MiniSMO version 0.6.0 for Windows 11. It explains how to install the application, create the built-in Demo Network, connect real NETCONF-managed network functions, read their configuration and state, and work with the simulated fault, performance and scenario features.

The guide is written for lab engineers, developers, testers, students and solution architects who need a small, local management environment for O-RAN-style network functions. You don't need to be a NETCONF expert; the NETCONF chapters start from the basics.

Conventions used in this guide:

Convention	Meaning
CREATE DEMO NETWORK	A button or menu item in the application
`%LOCALAPPDATA%`	A Windows path or a value you type
SYSTEM > *License*	Navigation: the *License* page in the *SYSTEM* group

> **Synthetic data**
>
> Everything the Demo Network produces (configuration, alarms, counters, software versions) is synthetic. It is realistic enough to learn and test with, but it does not come from real equipment.

2. What Kaldrivon MiniSMO Is

Kaldrivon MiniSMO is a lightweight, O-RAN-oriented management, simulation and test environment that runs on a single Windows PC. It has two halves that you can use together or separately:

- **A real NETCONF manager.** MiniSMO connects to external network functions over NETCONF/SSH (normally TCP port 830), performs the hello exchange, discovers capabilities, and runs `<get>` and `<get-config>`. Every message is recorded in the NETCONF Log with its full XML.
- **A built-in simulator.** The Demo Network is a small simulated RAN (O-CU-CP, O-CU-UP, two O-DUs and ten O-RUs) with editable configuration, alarms, smoothly changing performance counters and scripted scenarios such as cell outage and congestion.

Typical uses:

- Checking that a lab O-RU or O-DU answers NETCONF correctly before integrating it with a larger SMO.
- Looking at the capabilities and YANG modules a device advertises.
- Teaching or learning SMO concepts (FM, PM, CM, topology) without access to live equipment.
- Demonstrating operational workflows with repeatable scenarios.

MiniSMO is not a production network management system and does not replace an operator SMO. See Known Limitations.

3. Version 0.4 Scope

Version 0.6.0 includes:

Area	What is included
External NFs	Add, edit and delete NETCONF-managed NFs; staged connection test; connect/disconnect; capability discovery; `<get>`; `<get-config>` on advertised datastores
NETCONF	NETCONF over SSH, base 1.0 and 1.1 framing, password, keyboard-interactive and SSH key authentication, host-key pinning on first connect, YANG library discovery (RFC 8525), NETCONF Call Home (RFC 8071)
External monitoring	FM, CM and PM from external NFs: NETCONF notifications (RFC 5277/6470), YANG-Push (RFC 8639/8641), O-RAN FM/PM, ietf-alarms, 3GPP AlarmList, PM pull from the NF's model, with reconnect, replay, gap tracking and freshness
VES collector	VES 7.2.1 event listener (fault, perf3gpp, heartbeat, pnfRegistration, 3GPP stndDefined), attribution by sourceName, and the NF's VES destination set when the NF is added (can be turned off) with the standard models only: 3GPP `NtfSubscriptionControl` and, for O-RUs, the O-RAN WG4 configured subscription
Demo Network	14 simulated NFs with topology, configuration, alarms, PM and scenarios
Configuration	GET / VALIDATE & STAGE / APPLY / REVERT for simulated NFs; parameter form and XML for external NFs, editable once enabled per NF
Fault Management	One Alarm Console for simulated, external and internal alarms; external alarms from NETCONF notifications, alarm-list reads and VES; alarm resync; internal supervision and health alarms; generate and clear simulated alarms
Performance Management	Nine simulated counters with 1, 5, 15 or 30 second intervals; external PM from YANG-Push, the NF's own PM model (including O-RAN WG4 O-RU measurements), O-RAN PM notifications, VES perf3gpp and 3GPP measurement files
Scenarios	Six built-in scenarios with RUN, PAUSE, RESUME, STOP and RESET
Logs	NETCONF Log with search, filter, copy, export and clear; Audit Log
YANG	YANG model registry (*SYSTEM* > *Settings* > *YANG Models*)
Licensing	All features included. Free for up to 5 external NFs. A paid license (`.klicense`) is required for additional external NFs. Built-in simulated NFs never count

Area	What is included
Operations	Inventory tree, NF identity and capabilities, global search, alarm CSV export, configuration backups and compare, audit log, in-app notifications, system health, network health report, purge of FM, PM, CM or log data, northbound REST API, O-RAN R1 for rApps (SME and DME), webhooks, automation rules and schedules, approvals, alarm correlation and flapping detection, KPI rankings, advanced compare, desired state and drift, templates, bulk operations, rollback, SW campaign planning, scheduled reports, Investigate, diagnostics, advanced retention and audit export
Capacity & Storage	Capacity test of this PC, storage usage by category, storage health, growth forecast, tiered PM retention (raw, 5-minute, hourly), automatic and manual cleanup, capacity and storage reports
Help	This offline Help Center, the User Guide (HTML and PDF) and a Quick Start guide

Version 0.6.0 changes external devices only in two ways: the VES collector address, set automatically when you add the NF, and configuration changes you enable and confirm per NF. One more request acts on the NF: a file upload of a measurement file (`file-upload`), sent only after you confirm it under **FILES ON NF...**; it changes no configuration. Templates, remediation, rollback and automation actions all go through the same per-NF enablement, preview and confirmation. See How MiniSMO Writes to External NFs.

4. System Requirements

Item	Requirement
Operating system	Windows 11 x64 (22H2 or later recommended)
Processor	Any x64 CPU with 2 or more cores
Memory	4 GB RAM minimum, 8 GB recommended
Disk	About 300 MB for the application, plus room for logs and PM history
Display	1366 × 768 or larger
Components	Microsoft Edge WebView2 Runtime (included with Windows 11)
Network	Only needed to reach external NFs. Help and the Demo Network work offline
Permissions	Administrator rights to install; a standard user account to run

You do not need to install Python, .NET or any other runtime. The installer contains everything MiniSMO needs.

To reach an external NF, the PC must be able to open a TCP connection to the device's NETCONF port (830 by default). Corporate firewalls and VPN split-tunnel rules often block this. See NETCONF Connection Requirements.

5. Installation

1. Download `Kaldrivon-MiniSMO-0.6.0-Setup.exe`.
2. Optional but recommended: verify the download. The release includes `Kaldrivon-MiniSMO-0.6.0-Setup.exe.sha256`. In PowerShell:

   ```
   Get-FileHash .\Kaldrivon-MiniSMO-0.6.0-Setup.exe -Algorithm SHA256
   ```

   The hash must match the value in the `.sha256` file.
3. Run the installer. Windows may ask for administrator permission.
4. Accept the license agreement (Kaldrivon MiniSMO End User License Agreement; a copy is installed as `EULA.txt` in the installation folder), keep the default folder `C:\Program Files\Kaldrivon\MiniSMO` or choose another, and choose whether you want a desktop shortcut.
5. Select **Install**, then **Finish**. You can launch MiniSMO directly from the last page of the installer.

The installer creates a Start Menu entry **Kaldrivon MiniSMO**, an optional desktop shortcut, and an entry in *Settings* > *Apps* > *Installed apps* for uninstalling.

> **SmartScreen**
>
> If the release you received is not code-signed, Windows SmartScreen may show "Windows protected your PC". Select **More info** and check that the file name and hash match what Kaldrivon provided before choosing **Run anyway**.

6. Communication Matrix

Every network connection MiniSMO makes or accepts. The default ports are shown; every port marked *configurable* can be changed in *SYSTEM* > *Settings* (or per NF). MiniSMO never changes Windows Firewall itself: the installer adds the inbound rules you tick, scoped to the MiniSMO backend program and the Domain and Private network profiles. For another port, the Settings section concerned shows the PowerShell command to run as administrator.

Inbound: connections to this PC

Service	Port / protocol	From	Default state	Firewall rule from the installer
VES collector, HTTPS	8443/TCP, configurable	NFs sending VES 7.2.1 events (fault, perf3gpp, heartbeat, pnfRegistration, fileReady, stndDefined)	on	**Allow NFs to reach the VES collector** (8443)
PM file receiver, SFTP	2222/TCP, configurable	NFs uploading PM measurement files (O-RAN WG4 `remote-file-uploads`)	off until switched on, or until an NF's PM path is measurement files (automatic)	**Allow NFs to upload PM measurement files to MiniSMO**
NETCONF Call Home, SSH	4334/TCP (IANA `netconf-ch-ssh`), configurable	NFs that dial in (RFC 8071)	off	**Allow NFs to call MiniSMO**
Northbound API (rApps, other systems), HTTPS	8770/TCP, configurable	rApps and other management systems	off; listens on 127.0.0.1 only unless you set another address	none: add a rule if you open it to the network
MiniSMO backend API	8765/TCP	the MiniSMO app on this PC	always, 127.0.0.1 only	none needed: never reachable from the network, and protected with a per-launch key (see Local API Protection)

The PM file receiver uses port 2222, not the SSH port 22: port 22 is often taken by the Windows OpenSSH server, and MiniSMO's receiver only accepts PM file uploads.

Outbound: connections from this PC

Service	Port / protocol	To	When
NETCONF over SSH	830/TCP (IANA `netconf-ssh`), set per NF	each external NF	connect, monitoring, reads, configuration changes
Measurement file retrieval	SFTP 22/TCP, FTPES 21/TCP plus the server's passive data ports, HTTPS 443/TCP, HTTP 80/TCP (no credentials sent over plain HTTP); the port in the file location wins	the file location an NF signals (only the NF itself and the servers you allow)	File PM retrieval on, when a file is signalled
Webhooks, HTTPS	443/TCP or the port in the URL	the webhook URLs you configure	when an event matches a webhook
R1 data delivery, HTTPS	443/TCP or the port in the URL	the delivery URLs rApps register	while an rApp has a data job

Windows Firewall allows outbound connections by default. Firewalls between this PC and the NFs must allow both directions above. Nothing is sent to Kaldrivon or any other Internet service: there is no telemetry, and licenses are validated on this PC.

7. First Launch

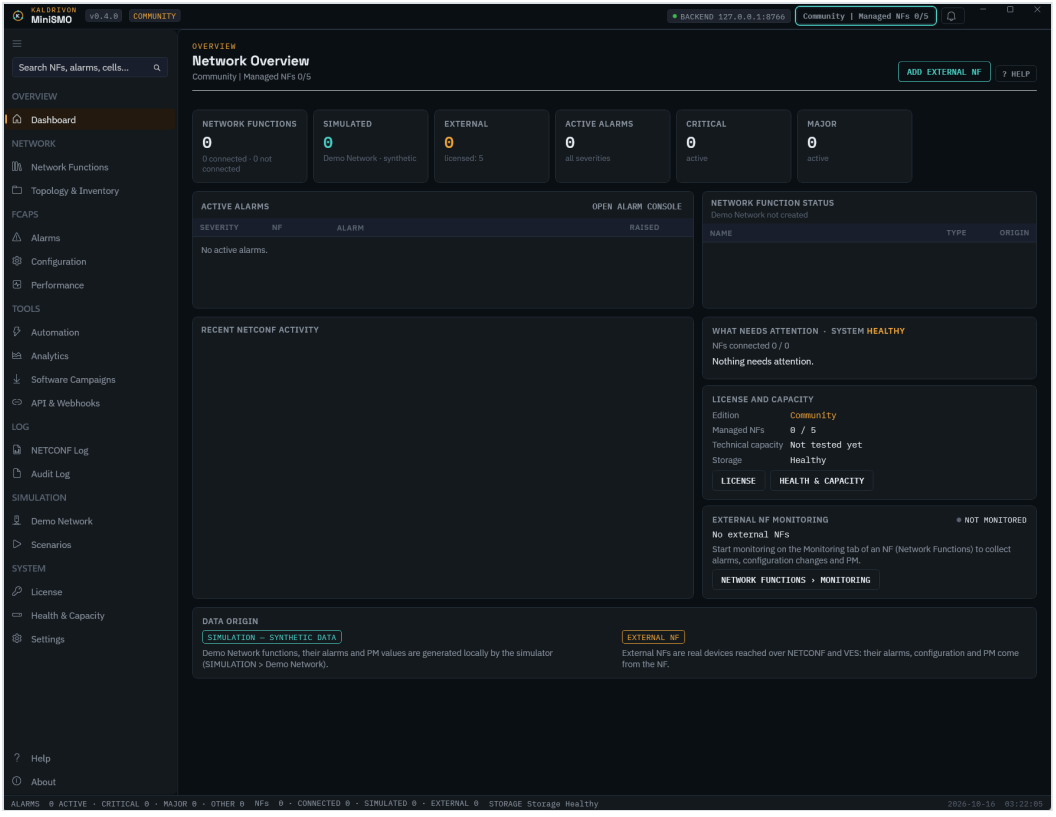
When you start MiniSMO:

1. The application checks whether its local backend service is running on `http://127.0.0.1:8765`, and uses it only if it proves that it is this user's MiniSMO backend (see Local API Protection).
2. If not, it starts the bundled backend (`KaldrivonMiniSMO.Backend.exe`) in the background with a new key. No console window appears.
3. On first run the backend creates its data folders under `%LOCALAPPDATA%\Kaldrivon\MiniSMO`, creates the database and applies migrations.
4. The Dashboard opens.

On a fresh installation the Dashboard shows:

- `Community | Managed NFs 0/5` under the page title and in the title bar.
- The **LICENSE AND CAPACITY** card with Edition *Community* and Managed NFs *0/5*.
- The **ADD EXTERNAL NF** button. The Demo Network is created under *SIMULATION > Demo Network*.

If the backend does not start within about 30 seconds, MiniSMO shows a diagnostic panel with the reason and an **Open startup troubleshooting** link. See Backend Startup Problems.



Dashboard on first launch: Community, Managed NFs 0/5

8. Application Layout

The main window has a navigation pane on the left and the working area on the right.

Group	Pages (tabs)
OVERVIEW	Dashboard
NETWORK	Network Functions (per NF: Overview, Capabilities, Monitoring); Topology & Inventory (Topology, Inventory)
FCAPS	Alarms; Configuration (Configuration, Configuration Operations); Performance
TOOLS	Automation, Analytics, Software Campaigns, API & Webhooks
LOG	NETCONF Log, Audit Log
SIMULATION	Demo Network, Scenarios
SYSTEM	License; Health & Capacity (Capacity & Storage, Reports); Settings (General, Storage & Retention, VES Collector, YANG Models)

Help and **About** are at the bottom of the navigation pane. The search box at the top of the pane finds NFs, alarms and cells. Pages with several sections show them as tabs across the top.

The title bar shows the edition chip (COMMUNITY, SMALL DEPLOYMENT or UNLIMITED), the backend state, the license indicator with the effective edition and external NF usage, for example `Community | Managed NFs 2/5`, and the notifications bell.

Confirmations and exports. Buttons that run something (exports, diagnostics, reports, snapshots, running a schedule, testing a webhook, analyses) ask for confirmation first. After an export a dialog shows where the file was saved, with **Open file** and **Show in folder**.

Most pages have a **?** help button in their header. It opens the help topic for that page. **F1** does the same from anywhere on the page, including the Add External NF dialog.

Keyboard basics:

Key	Action
F1	Help for the current page
Ctrl+F	Focus the search box (Help, NETCONF Log, Alarm Console, Network Functions capabilities)
F5	Refresh the current page
Esc	Close a dialog

9. Dashboard

OVERVIEW > *Dashboard* (page title **Network Overview**) gives a one-screen summary of everything MiniSMO manages. It refreshes every 5 seconds. The line under the title is the license indicator, for example `Community | Managed NFs 3/5` or `Small Deployment | Managed NFs 12/25`. When NFs beyond the capacity are hidden it adds their number, for example `Community | Managed NFs 5/5 · 2 unlicensed`. If a license has expired or is not valid, it says so, for example `Community (license expired) | Managed NFs 3/5`.

Summary tiles

Tile	Meaning
Network functions	All NFs, simulated and external
Simulated	Demo Network NFs (synthetic data)
External	Real NETCONF NFs you added and that are managed. These count against your license; hidden (unlicensed) NFs are not shown
Active alarms	Alarms that have not been cleared, all severities
Critical / Major	Active alarms at that severity

Cards

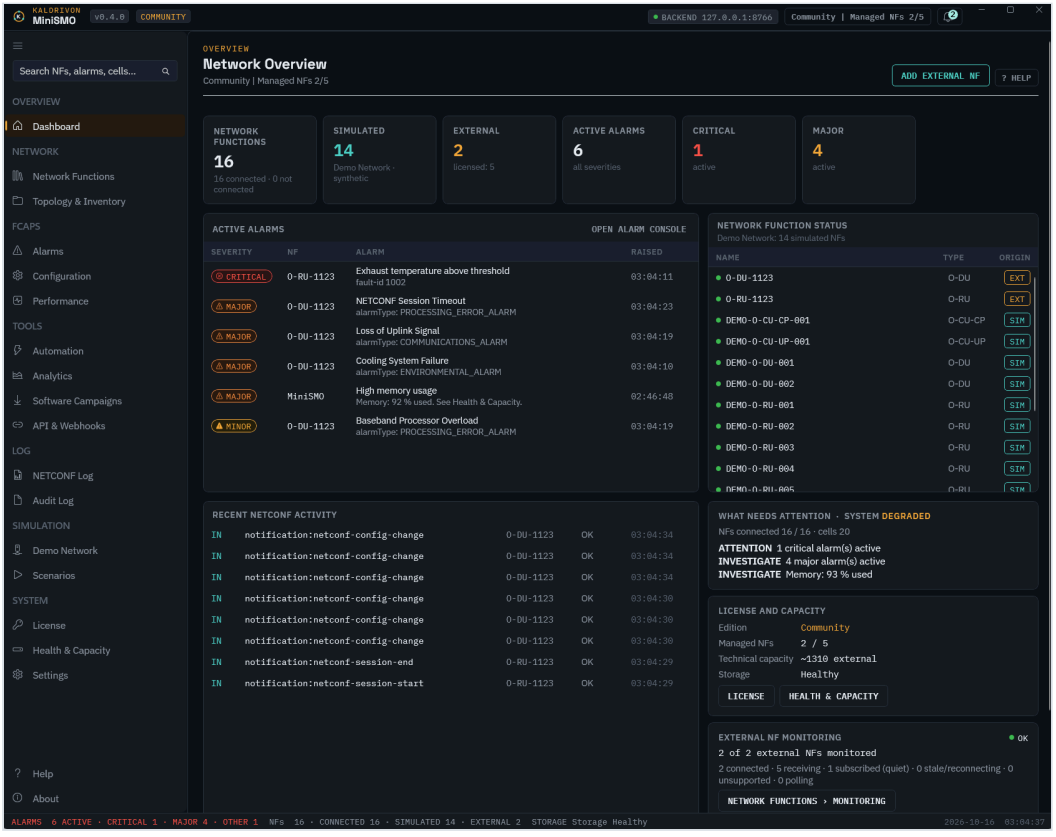
Card	Shows
ACTIVE ALARMS	The active alarms among the eight most recent ones. Select an alarm or **OPEN ALARM CONSOLE** to open *FCAPS* > *Alarms*
NETWORK FUNCTION STATUS	Every NF with its type, origin (simulated or external) and connection state
RECENT NETCONF ACTIVITY	The last eight NETCONF log entries. Select one to open it in the NETCONF Log
WHAT NEEDS ATTENTION · SYSTEM	Items that need a look now (for example disconnected NFs, critical alarms, failed jobs, storage state) and the System health state (HEALTHY, DEGRADED or CRITICAL). See Reports and System Health
LICENSE AND CAPACITY	Edition, Managed NFs (for example *3 / 5*), Technical capacity from the latest capacity test (for example *~180 external*, or *Not tested yet*) and Storage (*Healthy*, *Warning*, *Critical* or *Cleanup running*). **LICENSE** opens *SYSTEM* > *License*; **HEALTH & CAPACITY** opens *SYSTEM* > *Health & Capacity*

Card	Shows
EXTERNAL NF MONITORING	How many external NFs are monitored and how many of them are connected; then the monitored functions, counted per NF (FM, CM and PM each, so two NFs give up to six): receiving, subscribed (quiet), stale or reconnecting, unsupported or polling. DEGRADED means at least one function is stale, reconnecting or failed. **NETWORK FUNCTIONS › MONITORING** opens the Monitoring tab of Network Functions
DATA ORIGIN	Which data is synthetic (Demo Network) and which comes from external NFs

See Capacity & Storage Overview.

Actions

- **ADD EXTERNAL NF** (top right): opens the Add External NF dialog.
- The Demo Network is created and managed under *SIMULATION* > *Demo Network*.



Dashboard with the Demo Network running

10. Creating the Demo Network

The Demo Network is a small simulated O-RAN deployment built into MiniSMO. It lets you explore every page of the application without any real equipment.

To create it, open *SIMULATION* > *Demo Network* and select **CREATE DEMO NETWORK**. MiniSMO creates 14 simulated network functions:

Type	Count	Names
O-CU-CP	1	DEMO-O-CU-CP-001
O-CU-UP	1	DEMO-O-CU-UP-001
O-DU	2	DEMO-O-DU-001, DEMO-O-DU-002
O-RU	10	DEMO-O-RU-001 to DEMO-O-RU-010

Every simulated NF starts **UNLOCKED**, **ENABLED** and **CONNECTED**, carries the **SIMULATED** badge, and immediately starts producing performance data.

The Demo Network does **not** use external NF license capacity. After creating it, the license indicator still shows `Managed NFs 0/5`.

Managing the Demo Network (*SIMULATION* > *Demo Network*):

- **CREATE DEMO NETWORK** creates the 14 NFs, or **RESTORE n MISSING** recreates only the simulated NFs you deleted.
- **Alarm activity** sets how often simulated NFs raise and clear alarms on their own (off, low, normal, high); it is saved immediately.
- **RESET** deletes all 14 simulated NFs with their configuration, alarms and PM history, and creates them again in their initial state. You are asked to confirm.
- **DELETE** removes the simulated NFs without recreating them. You are asked to confirm.

External NFs are never affected by these actions.

> **Note**
>
> The Demo Network names carry the DEMO- prefix (DEMO-O-RU-001 and so on), so they never clash with a real NF's Managed Object ID or with what a real NF calls itself (for example an O-RU that reports O-RU-001). An existing Demo Network from an earlier version is renamed on update; its history is kept. The names are reserved while the Demo Network exists. If you already have an external NF with one of those names, MiniSMO refuses to create the Demo Network and tells you which name clashes.

11. Understanding the Demo Topology

The Demo Network is wired like a typical small split-7.2 deployment:

```
    DEMO-O-CU-CP-001 (F1-C)        DEMO-O-CU-UP-001 (F1-U)
           \        \            /        /
            DEMO-O-DU-001       DEMO-O-DU-002
                  |                  |
DEMO-O-RU-001 ... 005     DEMO-O-RU-006 ... 010
        (fronthaul)           (fronthaul)
```

- **DEMO-O-DU-001** serves DEMO-O-RU-001 through DEMO-O-RU-005.
- **DEMO-O-DU-002** serves DEMO-O-RU-006 through DEMO-O-RU-010.
- Both O-DUs connect to **DEMO-O-CU-CP-001** (control plane, F1-C) and **DEMO-O-CU-UP-001** (user plane, F1-U).

Scenarios use this structure. For example CELL OUTAGE takes down DEMO-O-RU-007 under DEMO-O-DU-002, and HIGH CPU LOAD stresses DEMO-O-DU-001. See Running Scenarios and Topology View.

12. Network Function Inventory

NETWORK > *Network Functions* lists every NF MiniSMO knows about, simulated and external, in one table.

Column	Meaning
NAME / ADDRESS	Unique name you gave the NF (or the Demo Network name), and host and port for external NFs
TYPE	O-RU, O-DU, O-CU-CP, O-CU-UP or GENERIC NETCONF NF
ORIGIN	Simulated (Demo Network) or external (real device over NETCONF)
CONNECTION	CONNECTED or DISCONNECTED
MONITORING	External NFs only: ONGOING, CONNECTING, STARTING, RECONNECTING, FAILED, STOPPED or NOT MONITORED. Empty for simulated NFs

Use the **Show** box to list All, Simulated or External NFs.

Select an NF to open its detail pane, with three tabs:

- **Overview**: identity, state, last error, NETCONF session ID and version, host key fingerprint, and **FORGET HOST KEY**. When a connection was refused because the host key changed, a red **SSH host key changed** bar shows the pinned and the new fingerprint with **FORGET HOST KEY AND RECONNECT...**.
- **Administrative and operational state** of an external NF are shown only as the NF reports them. The standard source is the NF's hardware model (`ietf-hardware` , RFC 8348, component `admin-state` / `oper-state`): an O-RAN O-RU reports its radio unit there (class `O-RAN-RADIO`), and the row says which component the value came from. The 3GPP network resource model (`ManagedElement` , `GNBDUFunction`) has no state for the NF as a whole: 3GPP defines administrative and operational state per cell (`NRCellDU`), shown with the cells in Inventory (select the cell for its administrative, operational and cell state). An NF that reports neither shows **Not reported by the NF**. Being connected does not make an NF *enabled*; the state is read again with the NF's identity (on connect and with RE-VERIFY).
- **Capabilities**: the NETCONF capabilities the device advertised in its last hello. See NETCONF Capability Discovery.
- **Monitoring**: FM, CM and PM monitoring, VES and file-based PM of an external NF. See Monitoring External NFs.

NETCONF reads (**GET** and **GET-CONFIG** with a subtree filter) are on the **NETCONF read** tab of *FCAPS* > *Configuration*.

Toolbar actions for the selected NF (the ones that do not fit move to **...**):

Action	What it does
Connect / Disconnect / Test connection	External NFs only

Action	What it does
Edit	Change an external NF's settings
Delete	Asks for confirmation, with the option to stop the NF sending VES to MiniSMO (see Pointing an NF at the Collector). A progress window then shows each step: VES destination, monitoring stopped, history deleted (row count), NF removed. It closes only when the deletion has finished; a second delete of the same NF is refused while one runs. Deleting an external NF frees a license slot
Resync alarms	Reads the NF's active alarm list again and reconciles the alarm view. See Monitoring External NFs
Discover identity	Reads vendor, model, serial number and versions again. See NF Identity and Cells
Site / region	Sets the NF's site and region for the network tree
Capabilities	Vendor-neutral summary of what the NF can technically do (this is not the Capabilities tab)
Back up config	Stores a configuration backup. See Configuration Backups
Investigate	See Investigate
Diagnostics	Asks for confirmation, then writes a diagnostic ZIP. See Diagnostic Collection
Export CSV	Asks for confirmation, then writes the NF inventory to a CSV file and shows **Open file** / **Show in folder**

13. Adding an External NF

13.1 Adding an External NF

An *external NF* is a real network function that MiniSMO manages over NETCONF/SSH. To add one:

1. Select **ADD EXTERNAL NF** on the Dashboard or on *NETWORK* > *Network Functions*. If NFs are hidden because they exceed the license capacity and a slot is free, MiniSMO first offers them: select one and **BRING BACK** (it keeps its settings and history and is monitored again), or **ADD A NEW NF** to continue. See Unlicensed (hidden) NFs.
2. Fill in the form:

Field	What to enter
Managed Object ID	Your own ID for the NF, unique, for example `SITE-A/O-RU-01`. It is independent of what the NF calls itself; see Identity Mapping
Label	Optional name shown in lists (defaults to the Managed Object ID)
NF Type	O-RU, O-DU, O-CU-CP, O-CU-UP or GENERIC NETCONF NF
Hostname/IP	DNS name or IP address of the device's management interface. Prefer a DNS name for a device whose address can change: MiniSMO looks the name up again on every reconnection, and the VES collector follows the new address. With an IP address, edit the NF when its address changes (monitoring reconnects as soon as you save). A Call Home NF is recognised by its identity, not its address
Authentication	Password, Keyboard-interactive or SSH key. For SSH key, **LOAD KEY FILE...** and enter the key passphrase if the key is encrypted. See SSH Authentication Methods
NETCONF Port	Usually **830** (the default is taken from Settings)
Username	NETCONF/SSH user on the device
Password	That user's password (not used with SSH key). Stored encrypted with Windows DPAPI, never in plain text
Description	Optional free text

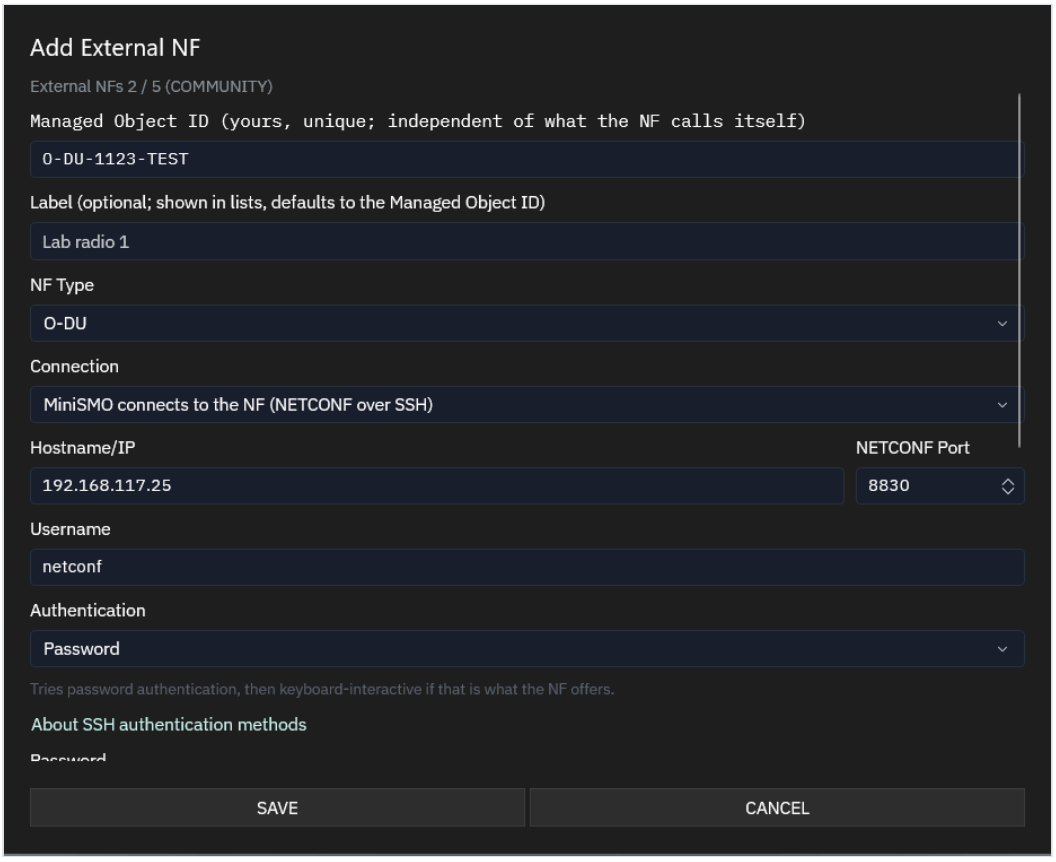
3. Select **TEST CONNECTION**. MiniSMO runs every connection stage and shows which ones passed. See NETCONF Connection Test. It also reads the NF's own ManagedElement (read-only) and shows it under **NATIVE IDENTITY**; **USE AS MANAGED OBJECT ID** copies it into your ID. An NF that exposes none over NETCONF (an O-RAN O-RU) says so; you can enter the **Expected native ID** it will report in its measurement files or VES events, which is checked when they arrive.
4. Select **SAVE**. MiniSMO checks your external NF capacity before saving. If you are at the limit, you see *Kaldrivon Mini SMO is free for up to 5 external NFs (5 in use); all features are included. A paid license is required for additional external NFs ...* (with a license, its limit) and nothing is saved. An NF that is already stored with

the same address and port (a DNS name is resolved, so a name and an IP address of the same device match), or, for Call Home, the same caller identity, is refused as a duplicate (*'O-DU-1' already manages this NF*), so one NF never takes two slots; see External NF Capacity. A saved NF gets the next integration number; when NFs exceed the capacity, the lowest numbers stay managed.

5. MiniSMO integrates the NF straight away: it connects, starts FM / CM / PM monitoring and points the NF's VES destination at the collector. A dialog shows each step. If the device is unreachable, the NF is saved anyway and shows **DISCONNECTED**; choose **CONNECT** later. Turn automatic integration off in *SYSTEM* > *Settings* > *General*, section **Integration**.

The **Help with NETCONF connection** link in the dialog opens NETCONF Connection Requirements. **F1** opens this topic.

Saving does not require a successful test, so you can add devices that are currently powered off. They still use a license slot.



Add External NF: the form filled in for an O-DU, before SAVE

13.2 Example: Adding an External O-RU

This walkthrough adds a lab O-RU and reads its configuration.

1. Enter the details

Field	Value
Name	`LAB-O-RU-01`
NF Type	O-RU

Field	Value
Hostname/IP	`192.168.10.50`
NETCONF Port	`830`
Username	`admin`
Password	`********`

2. TEST CONNECTION. The result list shows each stage. A healthy device looks like this:

Stage	Result	Example detail
Host Resolution	✓	192.168.10.50 -> 192.168.10.50
TCP Connection	✓	connected to 192.168.10.50:830
SSH	✓	SSH-2.0-OpenSSH_8.x; host key ssh-ed25519 SHA256:…
Authentication	✓	authenticated as 'admin'
NETCONF subsystem	✓	netconf subsystem opened
Server Hello	✓	session-id 42; NETCONF 1.1; 37 capabilities

Below the list MiniSMO shows the negotiated **NETCONF version** and the **capability count**.

If a stage fails, the stages after it show *not reached*. What to check at each stage:

Failed stage	Typical message	First things to check
Host Resolution	cannot resolve 'lab-ru-01'	Spelling; use the IP address instead; DNS/VPN
TCP Connection	timed out connecting to 192.168.10.50:830	Device powered and reachable (ping), port number, firewalls between you and the device
SSH	SSH handshake failed	Something other than SSH listens on that port; very old or restricted SSH algorithms on the device
Authentication	authentication failed (wrong username or password)	Credentials; account locked; device allows only public-key login
NETCONF subsystem	the 'netconf' subsystem was refused	SSH works but NETCONF is not enabled for that port or user
Server Hello	timed out / not well-formed XML	Device NETCONF service unhealthy or non-standard; try again after restarting it

3. SAVE. The O-RU appears in *Network Functions* as an external NF and the license indicator goes up by one. With automatic integration on (the default), MiniSMO connects, starts monitoring and points the NF's VES destination at the collector; a dialog shows each step.

4. CONNECT (only if integration is off or the device was unreachable). MiniSMO opens a session and stores the capabilities. The row changes to CONNECTED.

5. GET. In *FCAPS* > *Configuration*, select the NF, open the **NETCONF read** tab and select **GET**. MiniSMO sends `<get/>` and shows the operational and configuration data returned.

6. GET-CONFIG. Choose the **running** datastore and select **GET-CONFIG**. Only datastores the device advertised are selectable. See Running GET-CONFIG.

All requests and replies are in *LOG* > *NETCONF Log*.

13.3 Identity Mapping

Every external NF has three identities, kept apart as in 3GPP TS 32.300 naming:

Identity	What it is
Inventory UUID	MiniSMO's own key, created once and never changed
Managed Object ID	Yours: assigned when the NF is added, unique, changeable at any time in **Edit** without losing history (all FM, PM, CM and VES records stay with the NF)
Native identifiers	What the NF itself reports, exactly as reported, with where it came from

Native identifiers are taken only from what the NF provides, never derived from your ID, the label or the address:

Kind	Source
3GPP ManagedElement	`ManagedElement/id` (and `dnPrefix`, when the NF's revision has it) of `_3gpp-common-managed-element`, read over the NF's NETCONF session. Its DN is the DN prefix plus `ManagedElement=<id>` (TS 32.300)
Measurement file localDn	`managedElement` / `fileSender` `localDn` and `dnPrefix` of the NF's measCollecFile (TS 32.435)
VES sourceName	`commonEventHeader.sourceName` of the NF's VES events
Hardware serial number	`serial-num` from `ietf-hardware` (the physical unit, not a managed element)

An O-RAN O-RU has no 3GPP ManagedElement in its M-Plane; its own ID appears only in its measurement files or VES events.

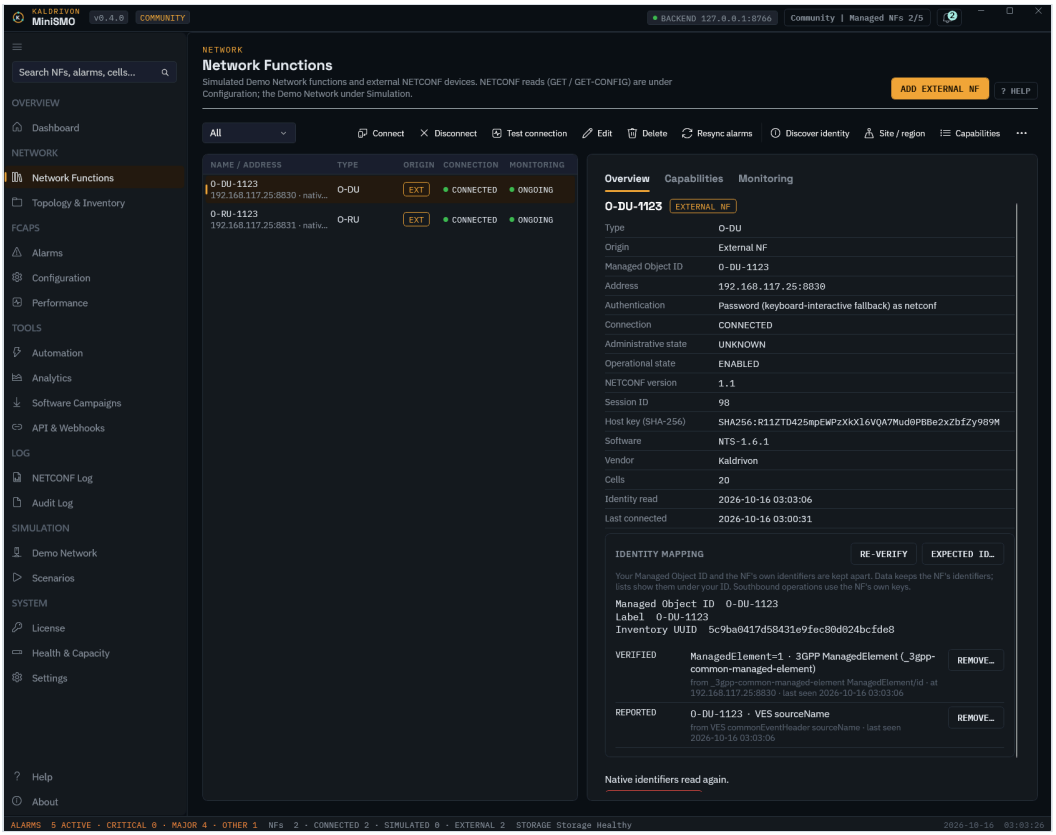
Status of each identifier

Status	Meaning
VERIFIED	Read over the NF's own authenticated channel: its NETCONF session, or its own login at MiniSMO's PM file receiver
REPORTED	From an unauthenticated source (VES, or a file location named in a VES event)

Status	Meaning
CLAIMED	Entered by you (**EXPECTED ID...**), not reported by the NF yet
CONFLICT	The NF now reports another value than the verified one, another value than you expected, or another NF reports the same identifier
STALE	The NF's host or port changed; checked again on the next connection

How the mapping is used. Stored records keep the NF's own identifiers. Lists show them under your Managed Object ID: an alarm's faulty object `ManagedElement=1,GNBDUFunction=1,NRCellDU=1` on the NF you call `OPS-DU-1123` is shown as `OPS-DU-1123 › GNBDUFunction=1,NRCellDU=1`, with the reported value in the details; the same applies to PM objects and VES events. Only the exact DN or LDN of the NF's own ManagedElement is replaced; anything else stays as reported. NETCONF operations always use the NF's own names and keys. The northbound API returns `uuid`, `mo_id` and the native identifiers next to the reported values.

Conflicts are never resolved by merging. NFs are never joined by address, label or a partly matching identifier. When an NF's ManagedElement or serial number is in conflict (for example after its address changed and another device answers there), MiniSMO keeps reading from it but does not change anything on it (configuration changes, VES or PM file destinations) and adds a notification. A VES event naming an identifier that two NFs report is not attributed to either. In the NF's **IDENTITY MAPPING** section (Overview tab), **ACCEPT...** confirms that the NF really is this device now (the previous identifier of that kind is replaced; history stays), **REJECT...** removes the conflicting identifier. **RE-VERIFY** reads the NF's own identifiers again. A file or VES event naming another element than the verified one is flagged too; its data is kept with the identifier it carries.



Network Functions, Overview tab: the Identity Mapping section of an external NF

13.4 NETCONF Call Home

Normally MiniSMO opens the connection to the NF. With *NETCONF Call Home* (RFC 8071) the NF opens the TCP connection to MiniSMO instead, which suits NFs behind NAT or a firewall that blocks incoming connections. MiniSMO stays the SSH client and the NETCONF client: after the connection arrives, everything is the same as for any external NF. It is the same NF record, with the same credentials, license slot, capability discovery, FM / CM / PM monitoring, VES, configuration history and PM discovery.

Only NFs you registered are accepted. Call Home is off until you switch it on in *SYSTEM* > *Settings*, section **NETCONF Call Home**: **Listen for Call Home connections**, the listen address, the port (default 4334, IANA `netconf-ch-ssh`) and how long a connect waits for the NF to call. The installer opens TCP 4334 in Windows Firewall for the backend (Domain and Private networks) when **Allow NFs to call MiniSMO** is ticked during setup; it is removed on uninstall. For another port or network, the section shows the PowerShell command. A caller that is not one of your NFs is closed during the SSH key exchange, before MiniSMO sends any credential. It is listed under **REFUSED CALLERS** (address, SSH host key fingerprint, reason). A device that keeps calling is one row with a count ("12 times since 13:06"), not a new row per attempt. A caller that drops the connection during the SSH key exchange has no fingerprint: MiniSMO never got to see who it was, and the cause is on the device (its SSH server ended the connection), so check the device's own log. After comparing the fingerprint with the one the NF itself shows, **ADD AS NF…** opens *Add External NF* in Call Home mode with the fingerprint and address filled in.

To add one, choose **Connection: The NF calls MiniSMO (NETCONF Call Home, RFC 8071)** in *Add External NF*. The port field disappears, the hostname becomes optional,

and the **CALL HOME IDENTITY (ALLOW-LIST)** fields appear. The NF must say how its caller is recognised:

Field	Meaning
SSH host key fingerprint	`SHA256:...` as the NF shows it. Checked during the key exchange; the strongest check. A different key is never accepted
Serial number, hardware id	Checked after login against what the NF reports about itself (`ietf-hardware` `serial-num` and `hardware-rev`). If the NF does not report them, it is refused. Without a fingerprint these need **pin the host key on the first call**, so the first verified call fixes the key for good
Allowed source	Optional IP address or network (for example `10.20.0.0/24`) the calls must come from

Hostname and port are not needed: the address is learned from each call. One session per NF is used for monitoring and for reads. If the NF disconnects it calls again and monitoring resumes. An NF that calls with the wrong identity is refused and listed as refused; its host key is not pinned.

EDIT on the NF changes the connection mode and these identity fields later; changing them makes the NF prove its identity again on its next call. The NF's details show the connection mode, its identity and the last call (time and address). TEST CONNECTION does not apply to a Call Home NF: it is checked when it calls, and the integration dialog after **SAVE** waits for that call.

The same is available through the API: `POST /api/nfs/external` with `connect_mode: "call-home"`, `host_key_fingerprint`, `ch_serial`, `ch_hardware_id`, `ch_source`, `ch_trust_first_use`; the listener state is `GET /api/callhome`.

13.5 SSH Authentication Methods

When you add an external NF you choose how MiniSMO signs in over SSH:

Method	Use it when
Password	The usual case. If the NF only offers keyboard-interactive, MiniSMO falls back to it with the same password
Keyboard-interactive	The NF asks for the password through a keyboard-interactive prompt, possibly with non-standard prompt text (O-RAN-SC NTS does this). Every prompt is answered with the password
SSH key	The NF has your public key in its authorized keys. Load the private key file (OpenSSH or PEM; Ed25519, ECDSA or RSA) and enter its passphrase if it is encrypted

Passwords, private keys and passphrases are stored protected for your Windows user (DPAPI) and never shown again. They are masked in all logs. Nothing about an NF (address, port, username, secrets) is built into MiniSMO: it is all entered when you add the NF.

13.6 Confirming the SSH Host Key

Before MiniSMO sends a password or uses a private key, it checks that it is talking to your NF and not to another device that took its address. The check is the NF's SSH host key: MiniSMO logs in only to a key you confirmed.

When adding an NF. The first **TEST CONNECTION** stops after the SSH stage and shows the fingerprint the NF presents, for example `SHA256:Xq3…`. Nothing has been sent to the NF at that point. Compare it with the fingerprint the NF itself shows (its console, its documentation, or its administrator). If it matches, select **CONFIRM HOST KEY**: the test runs again with the credentials, and **SAVE** stores the confirmed key with the NF.

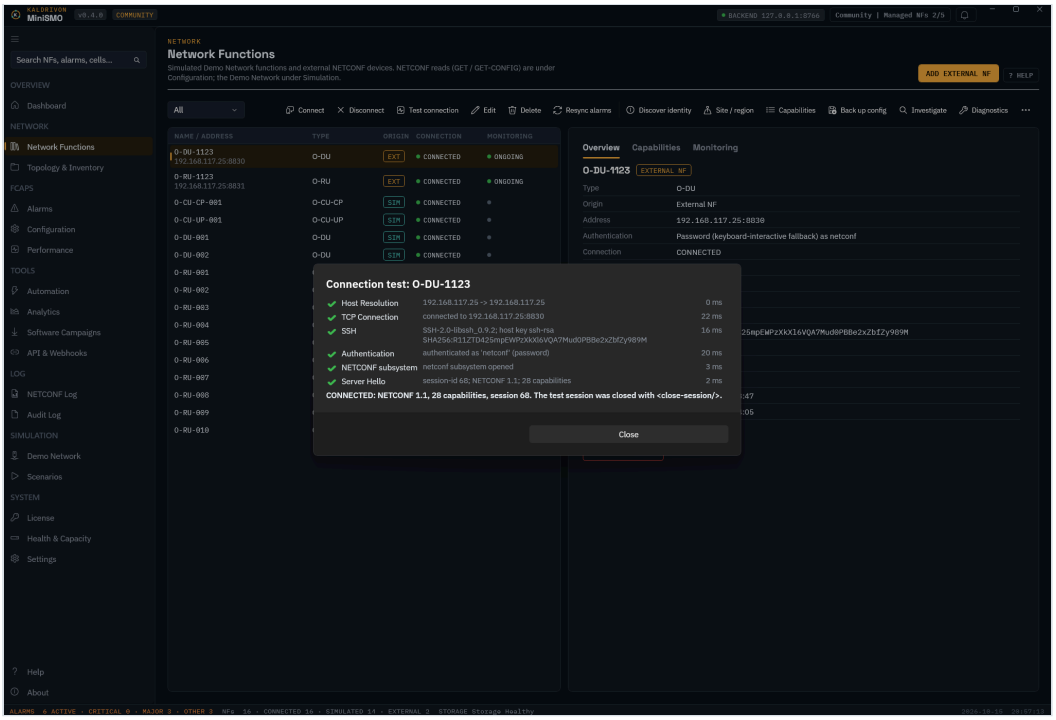
A saved NF without a confirmed key (saved without testing, or its address changed) is not logged in to: connecting, testing and monitoring stop after the key exchange. Its Overview tab shows **Confirm the NF's SSH host key** with the fingerprint it presents and **CONFIRM HOST KEY…**, which pins that key after confirmation and connects.

When the key changes later, MiniSMO refuses the NF with *host key changed* and shows both fingerprints (see NETCONF Connection Test).

Lab option. *Add External NF* has **Lab only: trust the host key the NF presents on the first connection**. With it, the first key is pinned without confirmation, as earlier versions did. Use it only for simulators and lab devices on a network you control. A key that changes later is still refused.

Call Home NFs are identified by their fingerprint, or by serial number / hardware ID with **pin the host key on the first call** (see NETCONF Call Home).

API: `POST /api/nfs/test-connection` returns `host_key_unconfirmed: true` and `host_key` until `host_key_fingerprint` is given; `POST /api/nfs/external` and `PATCH /api/nfs/{id}` accept `host_key_fingerprint` and `host_key_first_use`; `POST /api/nfs/{id}/host-key/confirm?confirm=true` pins the key the NF presented (`host_key_seen` on the NF).



NETCONF connection test: all six stages passed

14. NETCONF Connection Requirements

14.1 NETCONF Connection Requirements

For MiniSMO to manage a device, all of the following must be true:

1. **Network reachability.** Your PC can reach the device's management IP address, and nothing between them (Windows Firewall, corporate firewall, VPN, security groups) blocks the NETCONF port.
2. **NETCONF over SSH is enabled** on the device, normally on TCP port **830**. Some devices use port 22 with the `netconf` SSH subsystem instead.
3. **A NETCONF user account** exists, with a password or an SSH key. MiniSMO supports password, keyboard-interactive and public-key authentication (see SSH Authentication Methods).
4. **The device speaks standard NETCONF**: it sends a `<hello>` with at least the base 1.0 or base 1.1 capability.
5. **Read permission** for `<get>` and `<get-config>`. NACM on the device may limit what the account can see.

Quick checks from a Windows PowerShell prompt:

```
Test-NetConnection 192.168.10.50 -Port 830
ssh -p 830 admin@192.168.10.50 -s netconf
```

The first should report `TcpTestSucceeded : True`. The second should print the device's XML `<hello>` after login (press Ctrl+C to exit).

14.2 What is NETCONF?

NETCONF (Network Configuration Protocol, RFC 6241) is the IETF protocol for reading and changing the configuration and state of network devices. O-RAN uses NETCONF with YANG data models for O-RU management (the M-plane) and on the O1 interface between the SMO and O-RAN network functions.

The basics:

- The client (here, MiniSMO) and the server (the device) exchange XML messages.
- The client sends **RPCs** such as `<get>` or `<get-config>`, each with a `message-id`. The server answers with an `<rpc-reply>` carrying the same `message-id`, containing either data, `<ok/>` or an `<rpc-error>`.
- The structure of the data is defined by **YANG** modules. The device tells the client which modules it supports in its capabilities.
- Configuration is kept in **datastores**: *running* always exists; *candidate* and *startup* are optional.

14.3 Why Port 830?

TCP port **830** is the port IANA assigned to NETCONF over SSH (RFC 6242). Most devices listen for NETCONF there, which is why MiniSMO uses 830 as the default.

Some devices instead offer NETCONF on the normal SSH port 22 through the `netconf` SSH subsystem, or on a vendor-chosen port. If your device uses a different port, enter it in the **NETCONF Port** field when adding the NF, or change the **Default NETCONF Port** in Settings.

If port 830 is blocked by a firewall, the connection test fails at the **TCP Connection** stage.

14.4 NETCONF over SSH

NETCONF itself does not provide encryption or login, so it runs on top of a secure transport. MiniSMO uses SSH (RFC 6242):

1. A normal SSH connection is established and the user logs in.
2. The client requests the SSH **subsystem** named `netconf`, instead of a shell.
3. On that channel both sides send `<hello>` messages, then exchange RPCs.

Messages are delimited ("framed") in one of two ways:

- **NETCONF 1.0**: each message ends with the marker `]]>]]>`.
- **NETCONF 1.1**: *chunked framing*, where each chunk is preceded by its length.

MiniSMO supports both and uses 1.1 when the device advertises it.

On first connect, MiniSMO records the device's SSH host key fingerprint. Later connections must present the same key, which protects you from connecting to an impostor. Changing the NF's host or port clears the pinned key.

15. Testing an NF Connection

15.1 NETCONF Connection Test

TEST CONNECTION runs the whole NETCONF connection pipeline and reports each stage separately, so you can see exactly where a problem is:

1. **Host Resolution**: turns the host name into IP addresses.
2. **TCP Connection**: opens a TCP connection to the NETCONF port.
3. **SSH**: SSH protocol handshake and server host key.
4. **Authentication**: logs in with the username and password.
5. **NETCONF subsystem**: asks SSH for the `netconf` subsystem.
6. **Server Hello**: exchanges `<hello>` messages, reads the session ID and the server capabilities, and agrees the NETCONF version (1.1 if both sides support it, otherwise 1.0).

Run it from the Add External NF dialog before saving, or for a saved NF with **Test connection** on the *NETWORK* > *Network Functions* toolbar. Both show every stage with ✓ or ✗, its result and how long it took.

MiniSMO shows **CONNECTED** only after a real NETCONF session has been established, meaning the server hello was received and parsed. A successful TCP or SSH connection alone is not enough.

The test uses the timeouts from *SYSTEM* > *Settings* (NETCONF Connect Timeout, default 10 seconds). After the test, MiniSMO closes the session with `<close-session/>`. Test sessions are recorded in the NETCONF Log with the NF name prefixed by `TEST`.

For a saved NF, the test also checks the SSH host key against the confirmed one (see Confirming the SSH Host Key). If the key changed, the SSH stage fails with *host key changed*. The error names both fingerprints. If the device was legitimately replaced, reinstalled or re-keyed, compare the new fingerprint with the one the device itself shows, then choose **FORGET HOST KEY AND RECONNECT...** in the red bar on the NF's Overview tab (or **FORGET HOST KEY**, then CONNECT). After confirmation MiniSMO pins the new key and connects; a monitored NF also restarts its monitoring session right away.

Related: The Hello Exchange, NETCONF Connection Troubleshooting.

15.2 The Hello Exchange

Immediately after the `netconf` subsystem opens, both sides send a `<hello>` message:

- The **server hello** lists every capability the device supports and contains a `<session-id>`.
- The **client hello** from MiniSMO advertises `urn:ietf:params:netconf:base:1.0` and `urn:ietf:params:netconf:base:1.1`.

If both sides list base 1.1, the session uses 1.1 chunked framing; otherwise it uses 1.0.

In MiniSMO the hello exchange is the last stage of **TEST CONNECTION** and of **CONNECT**. The Server Hello stage fails if:

- the device does not send a hello within the connect timeout,
- the hello is not well-formed XML,
- the hello lists no capabilities or no NETCONF base capability.

Both hellos are recorded in the NETCONF Log with operation `hello`.

16. Understanding NETCONF Capabilities

The **capabilities** in a device's hello describe what it can do. MiniSMO stores them each time you connect and shows them on the NF's **Capabilities** tab.

Column	Meaning	Example
Capability URI	The full URI as sent by the device	`urn:ietf:params:xml:ns:yang:ietf-interfaces?module=ietf-interfaces&revision=2018-02-20`
Module	YANG module name, if the URI names one	`ietf-interfaces`
Revision	YANG module revision date, if present	`2018-02-20`
Parameters	Remaining URI parameters, such as features or deviations	`features=ntp`

Protocol capabilities such as `urn:ietf:params:netconf:capability:candidate:1.0` are shown with a module of `netconf:candidate` and a parameter `version=1.0`.

Use the search box (Ctrl+F) to filter by any part of the URI, for example `o-ran`, `ietf-hardware` or `candidate`.

Capabilities tell you:

- which YANG modules (and revisions) the device implements, so you know which models to import into the YANG Model Registry,
- which datastores exist (`:candidate`, `:startup`),
- optional protocol features (`:with-defaults`, `:xpath`, notifications, and so on).

17. Running GET

`<get>` asks the device for its **running configuration plus operational state** (counters, status, hardware inventory and so on).

1. Open *FCAPS* > *Configuration* and select the NF in the list.
2. Open the **NETCONF read** tab.
3. Optionally enter a **subtree filter** (an XML fragment) to limit the reply, for example:

   ```
   <interfaces-state xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"/>
   ```

4. Select **GET**.

If the NF is not connected, MiniSMO connects first. The result panel shows:

Field	Meaning
Status	OK, or ERROR with the device's error message
Message ID	The `message-id` of the RPC
Session ID	The NETCONF session ID
Timestamp	When the request was sent
Duration	Round-trip time ("12 ms", or seconds above one second)
Request / Response	Full XML, as an expandable tree or raw text

Unfiltered `<get>` on a large device can return several megabytes and take a while; use a filter if you know what you need. The NETCONF Operation Timeout in Settings (default 30 seconds) applies.

For simulated NFs, GET returns synthetic XML with session ID `SIM`.

18. Running GET-CONFIG

18.1 Running GET-CONFIG

`<get-config>` returns **configuration only** (no operational state) from a named datastore.

1. In *FCAPS* > *Configuration*, select the NF and open the **NETCONF read** tab.
2. Choose the **Datastore** in the toolbar above the tabs. Only datastores the device advertised are enabled: *running* always, *candidate* if the device has the `:candidate` capability, *startup* if it has `:startup`.
3. Optionally enter a subtree filter.
4. Select **GET-CONFIG**.

For external NFs, the **Parameters**, **XML tree** and **Raw XML** tabs of the same page show the running configuration, as an XML tree and as raw XML.

Related: Running, Candidate and Startup Datastores.

18.2 Running, Candidate and Startup Datastores

Datastore	What it holds	Always present?
running	The configuration the device is using right now	Yes
candidate	A scratch copy that can be edited and then committed to running	Only with `:candidate`
startup	The configuration loaded at boot	Only with `:startup`

Why might a device not support a datastore? Candidate and startup are optional in NETCONF. Many O-RUs, for example, apply changes directly to running and keep it persistent, so they advertise neither. MiniSMO greys out datastores that a device has not advertised; if you try one through the API anyway you get *DATASTORE_NOT_SUPPORTED*.

Simulated NFs support running and candidate.

19. Viewing Raw NETCONF XML

Everywhere MiniSMO shows a NETCONF exchange, you can switch between:

- **Tree**: an expandable view of the XML elements with their values. Good for browsing large replies.
- **Raw XML**: the exact text sent or received, pretty-printed for reading.

Select **COPY** to put the raw XML on the clipboard, for example to paste into a ticket or compare with another tool.

The same raw XML is stored in the NETCONF Log. Passwords never appear in NETCONF XML; authentication happens in SSH, before any NETCONF message is exchanged.

20. NETCONF Log

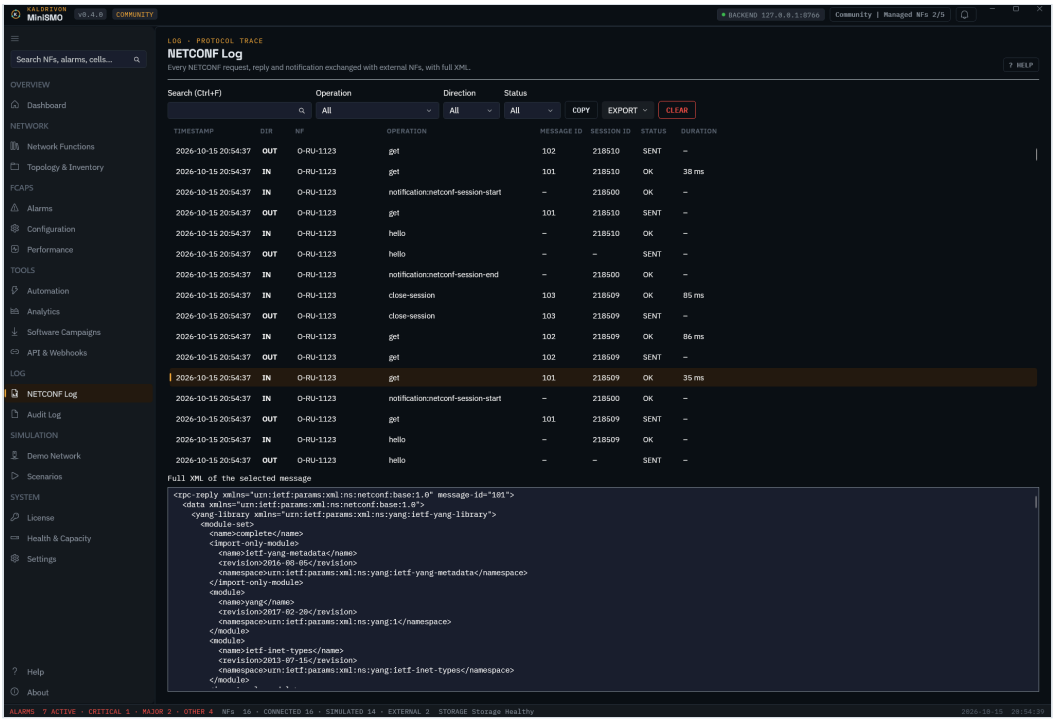
LOG > *NETCONF Log* records every NETCONF message MiniSMO sends or receives, for external and simulated NFs.

Column	Meaning
Timestamp	Local time the message was logged
DIR	OUT (MiniSMO to device) or IN (device to MiniSMO)
NF	NF name (`TEST <name>` for connection tests)
Operation	For example hello, get, get-config, get-schema, create-subscription, establish-subscription, notification, edit-config, commit, retrieve-file-list, file-upload, close-session
Message ID	RPC message-id (empty for hello)
Session ID	NETCONF session ID (`SIM` for simulated NFs)
Status	SENT for requests; OK or ERROR for replies; BLOCKED for a request refused before it was sent (see Write Protection)
Duration	Round trip ("12 ms" or "1.24 s"), on replies

Select a row to see the full XML below the table. The list refreshes every few seconds while the page is open; new messages appear without moving your scroll position or the selected row.

- **Search** (Ctrl+F) matches NF name, operation, message ID and XML content.
- **Filters**: Operation (the operations that occur in the log, for example `get-config` or `notification:alarm-notif`), Direction (OUT, IN) and Status (OK, ERROR, SENT). **All** shows everything. The column header stays visible while you scroll.
- **COPY** copies the selected message's XML.
- **EXPORT** asks for confirmation, saves the current view as JSON or CSV to `%LOCALAPPDATA%\Kaldrivon\MiniSMO\exports` and shows **Open file** / **Show in folder**.
- **CLEAR** deletes all log entries after confirmation.

Messages are kept for 7 days by default, optionally with a size limit, and very large payloads are capped in history and marked *PAYLOAD TRUNCATED FOR HISTORY*. See NETCONF Log Retention. Passwords are never logged.



NETCONF Log: messages with a selected message

21. Monitoring External NFs

21.1 Monitoring External NFs

The **Monitoring** tab of each external NF (*NETWORK* > *Network Functions*, select the NF) collects alarms (FM), configuration changes (CM) and performance data (PM) from external NETCONF devices. Monitoring observes, reads and subscribes. Only two things change an NF's configuration: pointing it at MiniSMO's VES collector (see Pointing an NF at the Collector) and configuration changes you enable and confirm for that NF (see Changing External Configuration). A third request acts on the NF without changing its configuration: a measurement file upload you confirm under **FILES ON NF...** (see Measurement Files).

When you add an NF under *Network Functions* (see Adding an External NF), MiniSMO integrates it straight away: it connects, discovers the capabilities, starts FM, CM and PM monitoring with the best method each function supports, and points the NF's VES destination at the collector. A dialog shows each step as it completes. Turn this off in *SYSTEM* > *Settings* > *General*, section **Integration**, if you prefer to start by hand.

To start by hand, select the NF in *Network Functions*, open its **Monitoring** tab and choose **DISCOVER** (MiniSMO reads the server `<hello>`, the YANG library and the event stream list; see YANG Library Discovery), then **START MONITORING**, pick FM, CM and/or PM, and confirm. While the NF is monitored, the button reads **UPDATE MONITORING...** and opens the same dialog to change the settings.

MiniSMO opens a separate NETCONF session for notifications, reads the current state (active alarms, a configuration snapshot) and subscribes. If the device does not answer one of these initial reads (some simulators never reply to a read of the O-RAN active alarm list), that function is marked *initial sync incomplete* with the reason; the session, the subscription and the other functions carry on.

The tab also has a **VES** card: whether the NF's VES destination points at MiniSMO, how many VES events have arrived, the last event, the heartbeat and the last pnfRegistration. It shows what the collector actually receives, not what the NF's models say it could send. An NF only sends VES when its own VES reporting is on (for O-RAN-SC NTS simulators, the simulator's VES features) and it can reach this PC. The subscription lives only as long as that session. Nothing is stored on the device, and **STOP** just closes the session. While monitoring of an NF is stopped, its VES events are still listed in the VES event log (status NOT_MONITORED) but create no alarms and no PM. **STOP** ends FM, CM, PM and VES processing for the NF. The **MONITORING** column in *Network Functions* shows ONGOING, CONNECTING, STARTING, RECONNECTING, FAILED, STOPPED or NOT MONITORED for each external NF.

Supervision and alarm resync. When the monitoring session of an NF is down for more than 30 seconds, MiniSMO raises the internal alarm **Loss of supervision** on that NF and clears it when the session is back. On reconnection MiniSMO reads the NF's active alarm list again and reconciles the alarm view: alarms no longer active on the NF are cleared. For an NF that reports its faults by VES and keeps a 3GPP AlarmList, a VES

alarm whose alarmId is no longer in the list is cleared too (after a one-minute grace period, because a new fault may not be in the list yet).

RESYNC ALARMS (on the Monitoring tab and in the Network Functions toolbar) does the same on request. If the NF's alarm list cannot be read (some NFs only send notifications), MiniSMO tells you so and offers to clear the NF's active alarms locally; faults still active on the NF come back with its next notification or VES event. Nothing is sent to the NF.

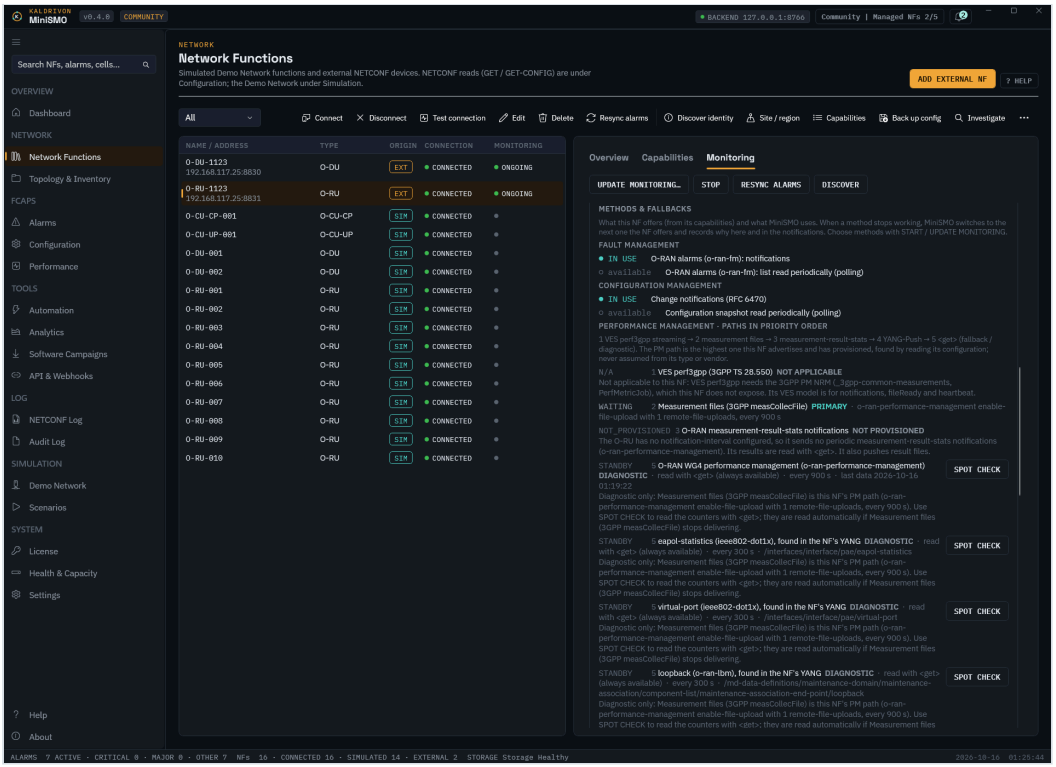
O-RAN alarms (o-ran-fm). An `alarm-notif` is mapped field by field: the title is `fault-text`, the faulty object is `fault-source` (the `affected-objects` are listed in the details), the severity is `fault-severity`, `is-cleared` = true closes the matching alarm, and the time is `event-time`. `fault-id` is shown only as the alarm ID. One alarm instance is NF + `fault-id` + `fault-source`, because the same fault-id can be active on several sources. Many O-RUs only send these notifications and keep `active-alarm-list` empty or unreadable; the alarm view then follows the notification stream.

NFs that allow one session at a time. MiniSMO normally uses two NETCONF sessions per NF: one for monitoring (notifications) and one for requests (get-config, diagnostics, backups, configuration changes). Some devices accept only one session. When the second connection is refused while the monitoring session is up, MiniSMO marks the NF **One session at a time** (NF overview and Monitoring tab, *Connection health*) and adapts:

- the NF advertises `:interleave` (RFC 5277): requests run over the monitoring session, alongside the notifications; the overview shows **Requests: use the monitoring session**;
- it does not: notifications and requests cannot share the session, so MiniSMO keeps one session without a subscription and collects FM and CM by polling (listed under **METHODS & FALLBACKS**). Alarms then arrive at the next poll instead of immediately.

Nothing is changed on the NF. If the device later advertises `:interleave` (for example after a firmware update), MiniSMO subscribes again after the next reconnection of the monitoring session. If it later accepts a second session, MiniSMO uses two sessions again after MiniSMO restarts.

What you asked to monitor is stored locally. When MiniSMO restarts, it reconnects and subscribes again.



Network Functions, Monitoring tab: connection, FM / CM / PM state and PM sources of an external NF

21.2 Capability Discovery

Discovery shows, per function, what the device advertises and what MiniSMO will do with it. Everything is taken from the NF: its `<hello>`, its YANG library (RFC 8525 / 7895) and its event streams. Nothing is assumed per vendor or per NF type, so any O-RU, O-DU or O-CU that implements these standards can be monitored.

Function	Methods, in the order MiniSMO prefers them	Needs
FM	1. Alarm notifications of each alarm model the NF has	`notification:1.0` (RFC 5277) or `ietf-subscribed-notifications` (RFC 8639), plus `o-ran-fm`, `ietf-alarms` (RFC 8632) or the 3GPP AlarmList with RFC 6470 `netconf-config-change`
FM	2. Reading each model's active alarm list periodically	the alarm model alone
CM	1. Change notifications	`ietf-netconf-notifications` (RFC 6470)
CM	2. YANG-Push on-change	`ietf-yang-push` with the on-change feature
CM	3. Reading the configuration periodically	nothing beyond `<get-config>`
PM	1. VES `perf3gpp` streaming	`_3gpp-common-measurements` or the VES models; provisioned by a stream-based `PerfMetricJob` (3GPP TS 28.550 / 28.622)

Function	Methods, in the order MiniSMO prefers them	Needs
PM	2. Measurement files (3GPP TS 32.432 `measCollecFile`)	file markers (`o-ran-file-management`, `_3gpp-common-files`); provisioned by a file-based `PerfMetricJob`, or by `enable-file-upload` with `remote-file-uploads` in `o-ran-performance-management`
PM	3. O-RAN `measurement-result-stats` notifications	`o-ran-performance-management` plus notifications; provisioned by `notification-interval`
PM	4. YANG-Push periodic (RFC 8641)	`ietf-yang-push` with `ietf-subscribed-notifications`
PM	5. `<get>` of the NF's counters: fallback and diagnostic only	the NF's own PM model (`o-ran-performance-management`, or a model whose YANG declares counter / gauge types)

The 3GPP AlarmList is also found by a read-only `get-config` probe, because some NFs (O-RAN-SC NTS) keep it in their configuration without announcing the module.

Namespaces come from the NF. Every read uses the namespace the NF announced for the module (for example a later version of `urn:o-ran:fm`). The standard namespace is used only for a module the NF did not announce. Notifications are recognized by module, so a newer version of the same module is still understood.

When an NF offers nothing for a function, the function is shown as **Unsupported** with the reason. Devices that report through VES are served by the VES Collector. File-based PM is described under Measurement Files.

21.3 Method Choice and Fallback

Automatic (default). For FM and CM, MiniSMO uses the first method in the table above that the NF offers.

For PM, the order in the table is fixed, and the path used for an NF is always the result of discovery on that NF, never an assumption from its type or vendor. MiniSMO checks which paths the NF advertises (its modules, from the YANG library) and which of them are provisioned. It reads the NF's own configuration for this: `PerfMetricJob` entries (`_3gpp-common-measurements`), the reporting leaves of `o-ran-performance-management` (`enable-file-upload`, `file-upload-interval`, `remote-file-uploads`, `notification-interval`), and whether data is actually arriving. The highest provisioned path becomes **PRIMARY**. Two NFs of the same type from different vendors can legitimately end up on different paths.

- Other provisioned push or file paths keep running as **ALTERNATE**, because they carry their own data.
- A path the NF supports but has not provisioned is shown as **NOT PROVISIONED** with the reason, never as stale or failed.
- `<get>` is a fallback and diagnostic read. Live counters may be static, cumulative or last-period snapshots, and are not guaranteed to refresh per period. While a

provisioned higher path is delivering, a `<get>` source is on standby (**DIAGNOSTIC**) and is not polled; **SPOT CHECK** reads it once on request. It becomes **FALLBACK** and is polled when no higher path is provisioned, or when the primary path goes stale or fails.

- An O-RU's reporting leaves are learned from its first `<get>` of `performance-measurement-objects`, so that source can poll briefly at the start before it goes on standby.

A model gates its method. VES `perf3gpp` (3GPP TS 28.550) needs the 3GPP PM NRM (`_3gpp-common-measurements`). An NF with only a VES model (for example an O-RU with `o-ran-ves-subscribed-notifications`) uses VES for notifications, fileReady and heartbeat, so its perf3gpp row is **N/A** with the reason, not waiting.

On a 3GPP NF, `perf3gpp` that arrives is taken as it is, even when no `PerfMetricJob` is visible. An NF can stream perf3gpp itself, without a job behind it, so that is not an inconsistency. A missing stream-based job therefore keeps the VES path open (waiting) rather than ruling it out. A missing file-based job means file-based PM is not available on that NF as configured. perf3gpp is expected once per `measDataCollection.granularityPeriod` (for example every 15 minutes, aligned to :00, :15, :30 and :45 with some jitter); a single late or missed period is not a fault, and the path becomes stale only after three missed periods.

Values that arrive twice for the same measurement, object and period are stored once.

Fallback. When the method in use stops working, MiniSMO switches to the next one the NF offers:

- the event stream is refused: alarms are read from the NF's alarm list every 2 minutes, and the configuration is read every 15 minutes (or at the intervals you set);
- an alarm or configuration read fails 3 times in a row: the next method is tried;
- a YANG-Push subscription is refused: the same data is read with `<get>` at the same period (with the XPath when the NF supports XPath filters, or the equivalent subtree filter for the default selection);
- a PM model read fails 3 times: that source is marked failed and retried every few minutes; the other PM sources continue;
- the primary PM path goes stale or fails (for example O-RAN PM notifications stop arriving): the NF's counters are read with `<get>` as the fallback until the primary path delivers again.

Every change is listed on the NF's **Monitoring** tab under **METHODS & FALLBACKS** (what failed, why, what is used now) and appears in the notifications (bell). A source that works again is listed too. After a reconnection MiniSMO starts again from the preferred method.

Choosing yourself. START MONITORING (or **UPDATE MONITORING…** on an NF that is already monitored) lets you pick the FM and CM method from the ones this NF offers, turn single PM sources off, and turn automatic fallback off. A method the NF does not offer is not used, and that is listed. With fallback off, a method that stops working is shown as **Failed** with the reason instead of being replaced.

Discovery also reads the RFC 5277 stream list: stream names and whether the device keeps a **replay log**. Replay lets MiniSMO recover events missed during a disconnection.

22. Write Protection

MiniSMO changes external NFs in two ways only: the VES destination (set when the NF is added, unless turned off in *SYSTEM* > *Settings* > *General*, section **Integration**) and configuration changes you have enabled and confirmed for that NF (both described below). One more request acts on the NF: a measurement file upload (`file-upload`) that you confirm. It changes no configuration. The guarantee sits in the NETCONF layer: every request is checked against a short list of allowed operations before it leaves the application.

Allowed	Why
`get` , `get-config` , `get-schema` , `get-data`	Reading state and configuration
`create-subscription` (RFC 5277)	Transient notification subscription, ends with the session
`establish-subscription` , `delete-subscription` (RFC 8639/8641)	Dynamic subscriptions MiniSMO created itself
`retrieve-file-list` (o-ran-file-management)	Lists files on the NF, changes nothing
`file-upload` (o-ran-file-management)	Only after you confirm **REQUEST UPLOAD...**. Asks the NF to push one file to your upload target. Recorded in the audit log
`close-session`	Ending its own session

Everything else is refused and not sent, unless it is part of one of the two changes below. That includes `edit-config` , `copy-config` , `delete-config` , `commit` , `discard-changes` , `lock` / `unlock` , `kill-session` , reset, reboot, software installation, and alarm acknowledgement or clearing. `copy-config` , `delete-config` , `kill-session` , reset, reboot and software installation are never sent at all. A refused request appears in the NETCONF Log with status BLOCKED.

The backend API refuses these operations too, so a script calling the API directly cannot write either.

Alarms of external NFs cannot be cleared or acknowledged from MiniSMO. You can add a **local note** or a **local acknowledgement**. Both are labelled *local only*, are stored in MiniSMO and are never sent to the NF.

MiniSMO also does not create configured (persistent) subscriptions, activate measurement jobs or set up file upload on the device. When a function needs one of these, MiniSMO says so and leaves it to you.

Exception 1: the VES destination. To receive VES events, an NF has to know the collector's address. When you add the NF, MiniSMO writes it into the NF's own VES settings, unless **Point new NFs at the VES collector** is off in *SYSTEM* > *Settings* > *General*, section **Integration**:

- The integration dialog after **SAVE** shows the result of this step.
- The write goes through a separate check that allows only MiniSMO's own 3GPP `NtfSubscriptionControl` entry under the NF's `ManagedElement` (recipient address, notification types, heartbeat period), and only deleting that same entry. Any other content, another manager's entry, a replace operation or a second container is refused before it is sent.
- MiniSMO reads the settings back afterwards to verify them. Both the write and the read appear in the NETCONF Log, with passwords masked.
- MiniSMO writes again only when the NF's values differ: after a `pnfRegistration`, after a lost monitoring session comes back, or when you change the collector settings. See Pointing an NF at the Collector.

Exception 2: configuration changes you enable. See Changing External Configuration. Changes are off for every NF until you enable them for that NF, and each change is previewed and confirmed.

The generic write API stays closed.

23. Subscription States and Gaps

Connection health and data freshness are shown separately. A quiet alarm stream is not a fault.

State	Meaning
Connected	The notification session is up
Subscribed	The subscription is active; no events recently (normal for alarms)
Receiving data	Events arrived in the last 15 minutes (FM/CM) or within the expected interval (PM)
Stale	PM data expected but not arriving (more than three intervals). The reason says what to check
Reconnecting	The session dropped. MiniSMO retries with increasing waits (1 s up to 5 minutes, with random jitter) and subscribes again
Unsupported	The device offers no method MiniSMO can use for this function
Polling fallback	You enabled explicit polling because push is unavailable. Polling is never shown as a subscription
Failed	The device refused the subscription; the reason shows its error message

Gaps. When the session drops and the device has a replay log, MiniSMO asks for the missed events from the time of the last event it received. The gap is shown as *recovered*.

Without replay, the gap is recorded as *not recoverable*. Current alarms and configuration are resynchronized from the device, but events inside the gap are not available. Gaps are also recorded if events arrive faster than they can be processed.

Duplicates (for example events sent again during replay) are recognized and counted, not stored twice.

24. YANG Library Discovery

NETCONF 1.1 devices often list only a few modules in their `<hello>`. The full module set is in the YANG library, `ietf-yang-library` (RFC 8525), announced with the `yang-library:1.1` capability and its `content-id`. MiniSMO reads the library with a read-only `<get>`, falling back to the older `modules-state` (RFC 7895). Modules learned this way appear in the NF's capability list, marked as coming from the YANG library.

The module set drives what MiniSMO does per NF:

Module	Effect
`_3gpp-common-fm`	FM from the 3GPP AlarmList (pull, plus change notifications if the NF sends them)
`_3gpp-common-measurements`	PM jobs live on the NF; values arrive as VES perf3gpp
`_3gpp-common-subscription-control`	VES destination set with an `NtfSubscriptionControl` entry (with your confirmation)
`o-ran-ves-subscribed-notifications`	O-RU: VES destination set with a configured subscription (RFC 8639), heartbeat via `o-ran-supervision`
Any module with config-false counter containers	PM pull, found by PM discovery in the NF's own YANG (for example a cell PM module)
`ietf-subscribed-notifications`, `ietf-yang-push`	Dynamic subscriptions and YANG-Push

A missing module never breaks monitoring. The function is shown as unsupported, with the reason. When the NF reports a new library `content-id`, or sends a capability or library change notification, MiniSMO reconnects and discovers again.

25. The VES Collector

25.1 VES Collector

Many O-RAN and 3GPP NFs report through VES (the ONAP / O-RAN event listener format, version 7.2.1) instead of, or next to, NETCONF notifications. MiniSMO includes a VES collector: an HTTPS listener that accepts `POST /eventListener/v7` and `/eventListener/v7/eventBatch`.

It runs on its own ports. The MiniSMO API itself stays bound to this PC only. Its event list, NFs and unattributed sources are on *SYSTEM* > *Settings* > *VES Collector*. The listener settings are in *SYSTEM* > *Settings* > *General*, section **VES collector**; **COLLECTOR SETTINGS** on the VES Collector tab opens them. **STOP COLLECTOR** stops the listeners (after confirmation; NFs then get *connection refused* and an internal alarm shows that the collector is not listening) and **START COLLECTOR** starts them again; MiniSMO always starts the collector when it starts. The **LISTENER** card shows the status (running since, stopped, not listening, turned off in Settings), a self-test (whether each listener accepts a connection on this PC), the time of the last request and this PC's addresses, so you can check that NFs point at an address this PC still has. The VES state of one NF is also on that NF's **Monitoring** tab.

Setting	Default
Listen address	`0.0.0.0` (all network interfaces)
HTTPS port	8443, with a self-signed certificate created on first start
Accept events from	**Known NF addresses only**
Authentication	none, or HTTP Basic with a username and password you set

The collector does not offer plain HTTP: it would carry events, and Basic credentials, unencrypted. The installer adds a Windows Firewall rule for TCP 8443. If you choose other ports, the VES Collector tab shows the `New-NetFirewallRule` command to run in an administrator PowerShell. The certificate's SHA-256 fingerprint is shown too, so you can compare it with what the NF reports.

What the collector does with each domain:

Domain	Result
`fault`	Raise, update or clear an alarm. Title `specificProblem` (else `alarmCondition`, else `eventName`), category `alarmCondition`, object `alarmInterfaceA`. The identity is the NF (`sourceName`), `alarmInterfaceA` and `alarmAdditionalInformation.alarmId` (the condition when there is no alarm ID). `eventName` is a category, never an identity. `NORMAL` clears the matching alarm

Domain	Result
`perf3gpp`	PM samples. Every `measInfo` has its own ordered `measTypes.sMeasTypesList`; a `measResults` entry's `p` is a 1-based index into it, so each value is stored under its measurement name (for example `RRU.PrbUsedDl`), per `measObjInstId`, stamped with the period end (`lastEpochMicrosec`) and the `granularityPeriod`. How often perf3gpp events are expected is read from the NF (see *VES perf3gpp: what is expected* under External Performance Data). A value stamped more than 5 minutes (or one period, if longer) after this PC's clock cannot have been measured yet: it is refused, counted in the VES log entry, and the NF's PM source explains how far ahead the NF's timestamps are. Check the NF's clock and the measurement period it reports. `suspectFlag` "true" marks values as suspect in charts; a `p` without a name is dropped
`heartbeat`	Heartbeat state per NF: OK while heartbeats arrive within three intervals, otherwise MISSED. The interval is only what the NF states: `heartbeatFields.heartbeatInterval` of its last heartbeat, else the heartbeat period in the VES subscription MiniSMO set and verified on the NF. Without either, heartbeats are shown as received and none is expected
`pnfRegistration`	Registration details (vendor, model, serial, OAM address). Counts as "the NF is up": MiniSMO resubscribes and, if you opted in, re-points the NF at the collector
`stndDefined`	3GPP FaultSupervision (`notifyNewAlarm`, `notifyChangedAlarm`, `notifyClearedAlarm`) becomes alarms. Other namespaces are stored and shown as not interpreted
other domains	Stored and shown as not interpreted

Which Senders Are Accepted

Anyone who can reach the collector's port could otherwise send events in an NF's name (false alarms, clears, measurements or file-ready signals). So by default the collector accepts events only from:

- the address of each NF in MiniSMO (its host address, the addresses its host name resolves to, and the address its NETCONF Call Home connection comes from). For an NF added by host name, the name is looked up again on every NETCONF (re)connection attempt, so a new address is followed without changes here,
- this PC itself (loopback),
- the addresses and networks in **Other addresses allowed to send** (*Settings* > *VES collector*), for example `198.51.100.7` or `203.0.113.0/28`.

Events from any other address get `403` (VES `policyException` POL0001) before they are read, and are counted. The **REFUSED SENDERS** card on the VES Collector tab lists each refused address, how often and when, and the `sourceName` the sender claims (shown to help you recognise it, never checked). If it is your NF sending from another address, for example behind NAT or from a separate management interface, **ALLOW...**

adds the address after confirmation. **Accept events from: Any address** turns the check off; MiniSMO then shows a recommendation on the VES Collector tab.

The address only decides whether events are accepted at all. Which NF an event belongs to is still decided by its names (see Matching Events to NFs).

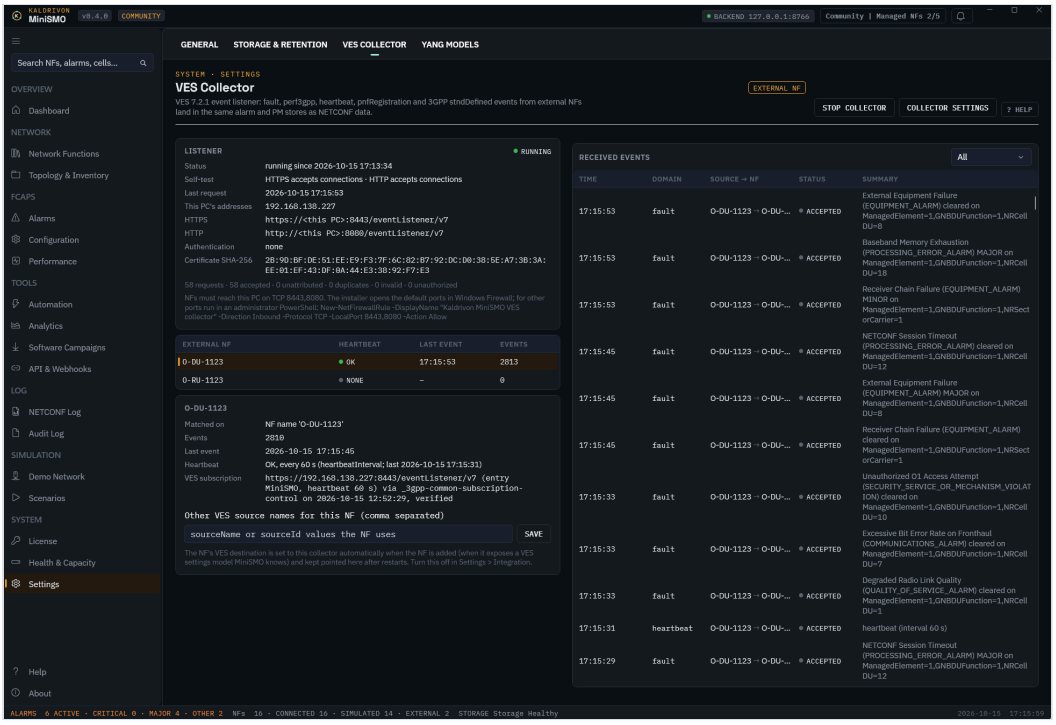
Limits. The collector protects itself and the PC: at most 64 connections at a time (more are closed at once), 30 seconds of silence per connection (including the TLS handshake, which never holds up other senders), and up to 100 requests per second per sender address with bursts of 500. A sender above that gets `429` with `Retry-After` (a VES "retry later"). The counters line on the VES Collector tab shows how many requests or connections were turned away. Real NFs stay far below these limits, also during an alarm storm (batched events count as one request).

When an NF's events stop. For an NF whose heartbeat is *MISSED* (or that never sent anything after MiniSMO pointed it here), the NF's details on the VES Collector tab show **Why nothing arrives**: what the collector itself observed since it started. That is one of:

- TLS handshakes from the NF's address failed, with the reason TLS gave (for example *no shared cipher*, or no handshake within 30 s);
- requests from the NF arrived but none was an event of this NF;
- events naming the NF come from another address and are refused (ALLOW… if it is the NF);
- nothing at all came from the NF's addresses: no TCP connection since the collector started, with the address the NF's subscription points at, and whether that is still one of this PC's addresses. MiniSMO cannot see past that point, whether the NF stopped sending or the connection is blocked on the way, so it says so.

The same text is in the internal alarm *VES heartbeat missed*. Failed TLS handshakes from other senders are listed in the LISTENER card (address, count, last reason) and logged once per sender. The listener's own self-test connection from this PC is not counted.

Invalid requests get `400` with a VES `requestError`. Events from an address that is not accepted get `403`. Wrong or missing Basic credentials get `401`. Accepted events get `202`. The same event received twice is stored once as data and marked *duplicate* in the event list.



VES Collector: listener, NFs with heartbeat state, received events

25.2 Matching Events to NFs

A VES event names its sender in `commonEventHeader`: `sourceName`, `sourceId` and `reportingEntityName`. MiniSMO matches these, case-insensitively, against:

1. the **VES source names** you entered for an NF (VES Collector tab, **Other VES source names for this NF**, then **SAVE**),
2. the NF's name in the inventory,

The sender's IP address is never used to decide which NF an event belongs to: several NFs can share an address (containers on one host), and an address does not identify the reporting entity. An event whose names match no NF is listed as unattributed, where you can assign it once.

An event that matches no NF is kept as **unattributed**. The page lists these sources with the address they came from. **ASSIGN...** adds the source name to an NF, so its later events are attributed. Events received before that stay unattributed.

Tip: an O-RAN-SC NTS device sends its own hostname or NF name as `sourceName`. Name the NF the same in MiniSMO, or add that value as a VES source name.

26. Pointing an NF at the VES Collector

An NF only sends VES events once it knows where to send them. MiniSMO writes the collector address into the NF's own VES settings automatically when you add the NF, so there is nothing to do on the *VES Collector* tab.

MiniSMO uses only the standard way to tell an NF where to send notifications: a **3GPP** `NtfSubscriptionControl` entry under the NF's `ManagedElement` (3GPP TS 28.623, module `_3gpp-common-subscription-control`). It checks that the NF announces this module in its YANG library, reads the NF's `ManagedElement` id and its existing subscription entries, and writes one entry of its own (id `MiniSMO`) with:

Leaf	Value
`notificationRecipientAddress`	the collector URL, e.g. `https://192.168.1.20:8443/eventListener/v7`
`HeartbeatControl/heartbeatNtfPeriod`	the heartbeat period (default 60 s; setting `ves_heartbeat_period_s`, 0 = no heartbeats)

O-RUs use the O-RAN WG4 M-Plane way (clause 18): a *configured subscription* (RFC 8639, `ietf-subscribed-notifications`) whose receiver uses the VES transport of `o-ran-ves-subscribed-notifications`. MiniSMO writes one subscription with the O-RU's `NETCONF` stream (or the first stream it lists), transport `o-ran-ves-connectionless` (no `encoding` leaf: RFC 8639 allows it only for configurable-encoding transports, and VES is JSON by definition) and one receiver named `MiniSMO` with `notification-recipient` = the collector's server root (e.g. `https://192.168.1.20:8443`; the O-RU may add path elements, and the collector accepts events at the root and at any path ending in `/eventListener/v7`). If the O-RU supports non-persistent M-Plane (`o-ran-supervision` `event-collector-monitoring`), MiniSMO also adds itself as heartbeat recipient with the heartbeat interval (at most 255 s, the model's limit). An NF that offers both models uses the 3GPP one.

Nothing device-specific is written, and only that one entry may be sent (MiniSMO checks the XML before it leaves). Entries of other managers are never changed or removed; a device that holds a single entry and has it in use refuses MiniSMO's, and the message says so. The standard model has no credential fields: if the collector requires basic authentication, set its user name and password on the NF itself (the preview warns about this). An NF without the model is reported in the integration step and nothing is written; set its destination with the vendor's tools, and the collector accepts its events anyway.

MiniSMO sends the `edit-config` to the running datastore (or the candidate plus `commit` if the NF has no writable running), then reads the entry back and reports whether it matches. The result is shown in the integration dialog after **SAVE**.

The address used is the local address this PC uses to reach that NF, plus the collector port for the protocol chosen in Settings. If NFs reach this PC through NAT or a DNS name, set **Collector address NFs should use** in *SYSTEM* > *Settings* > *General*, section **VES collector**.

MiniSMO keeps the NF pointed here: it re-applies the same settings after the NF re-registers (`pnfRegistration`), after a lost monitoring session comes back, and when you change the collector address, port, protocol or credentials. It writes only when the NF's entry differs, and only MiniSMO's own entry.

When the NF loses MiniSMO's entry (its configuration was reset or replaced, or another tool removed the entry), MiniSMO puts it back for NFs whose destination it set with **keep this NF pointed at the collector** on (the default when the NF is added). It reads its own entry back with `<get-config>` in the same standard model it was written with (3GPP `NtfSubscriptionControl` , or `o-ran-ves-subscribed-notifications` / RFC 8639 for O-RUs) and writes it again only when it is missing or different:

- right after a `netconf-config-change` notification (RFC 6470) from the NF that touches the subscription or replaces the whole configuration,
- every 10 minutes while the NF's heartbeat is missed,
- every 30 minutes otherwise, and when the NF reconnects or sends a pnfRegistration.

A restored entry is reported as a notification (*VES destination restored on the NF*). Only MiniSMO's own entry is read or written; other managers' entries are never touched.

When this PC's address changes (a new DHCP lease, another Wi-Fi network, a VPN), MiniSMO notices it within a minute: for each NF it has pointed here it compares the recorded destination with the address this PC now uses to reach that NF (or the address set in Settings), and when they differ it updates its own entry on the NF and adds a notification (*NF: VES destination updated to …*). An NF that cannot be reached is retried every 10 minutes and a notification says so; until then the NF's details on the *VES Collector* tab show **Address changed** with the old and the new address. Turn this off with **Follow address changes of this PC** (setting `ves_follow_address`). The same setting moves MiniSMO's PM file upload entry on O-RUs it pointed at its receiver (see External Monitoring). NFs whose destination you set by hand are not changed; point them at an address from **This PC's addresses**.

The collector writes events of at most 4 requests to the database at the same time. During an event storm further requests wait; after 20 s they are answered `503` with `Retry-After` , which VES senders retry, instead of overloading the database.

To stop MiniSMO from writing the VES destination, turn off **Point new NFs at the VES collector (and keep them pointed here)** in *SYSTEM* > *Settings* > *General*, section **Integration**.

Every write and every read-back is in the NETCONF Log. Passwords are masked there and in the application log.

When you delete an NF, the delete dialog has the option **Stop this NF sending VES to MiniSMO** (on by default). With it, before the NF is removed MiniSMO deletes its own `NtfSubscriptionControl` entry (or, on an O-RU, its configured subscription and heartbeat recipient), so the NF stops sending here. Other managers' entries stay as they are. The result is shown after the deletion. If the NF cannot be reached, the NF is still deleted and the message says so: MiniSMO waits at most 30 seconds for the NF to answer, and does not wait at all for a Call Home NF that is not connected at that moment (it calls on its own schedule).

Events that still arrive from a deleted NF (one that could not be changed, or one whose destination you set by hand) are **dropped, not stored**. The *VES Collector* tab lists them under **DELETED NFS STILL SENDING**, and the counters show how many were dropped. Adding an NF with the same name again accepts its events again; **ACCEPT THEIR EVENTS AGAIN…** does that for all of them (they are then stored as unattributed).

27. Fault Management

27.1 Fault Management

FCAPS > *Alarms* opens the **Alarm Console**. It lists simulated, external and internal alarms together. The counts above the list show active alarms by severity.

Column	Meaning
SEVERITY	CRITICAL, MAJOR, MINOR, WARNING, or CLEARED once cleared
NF	The NF the alarm is raised on
ALARM / DESCRIPTION	Alarm title and text
ORIGIN	Simulated, External or Internal
RAISED	When the alarm was raised

Filters: alarm state **Active** (default), **Cleared** or **All**; severity; origin (Any origin, Simulated, External, Internal); NF. The search box (Ctrl+F) matches alarm, text, NF and source.

Select an alarm to see **ALARM DETAILS**: source, managed object, times, the **EVENT HISTORY** and, for external alarms, a bounded copy of the last notification. You can add a **LOCAL NOTE** (**SAVE LOCAL NOTE**) and tick **Acknowledged locally**; both stay in MiniSMO and are never sent to the NF. **INVESTIGATE** is described under Investigate.

EXPORT CSV asks for confirmation, writes all alarms (active and history) to a CSV file in the exports folder and shows **Open file** / **Show in folder**. In every CSV export MiniSMO writes, a text that starts with `=`, `+`, `-`, `@`, a tab or a carriage return gets a leading apostrophe, so a spreadsheet shows it as text and never runs it as a formula (an NF or a sender could put one in an alarm text). Plain numbers such as `-3` are written unchanged.

RAISE TEST ALARM... (at the top of the Alarm Console): choose a simulated NF, an alarm type and a severity, optionally add text, and select **RAISE ALARM**. If the same alarm type is already active on that NF, it is updated instead of duplicated.

CLEAR (SIMULATED ALARM): select an active simulated alarm and choose **CLEAR (SIMULATED ALARM)**. It stays in the list with severity CLEARED and a cleared timestamp.

The worst active severity per NF is also shown on the Topology view, and the Dashboard counts active, critical and major alarms.

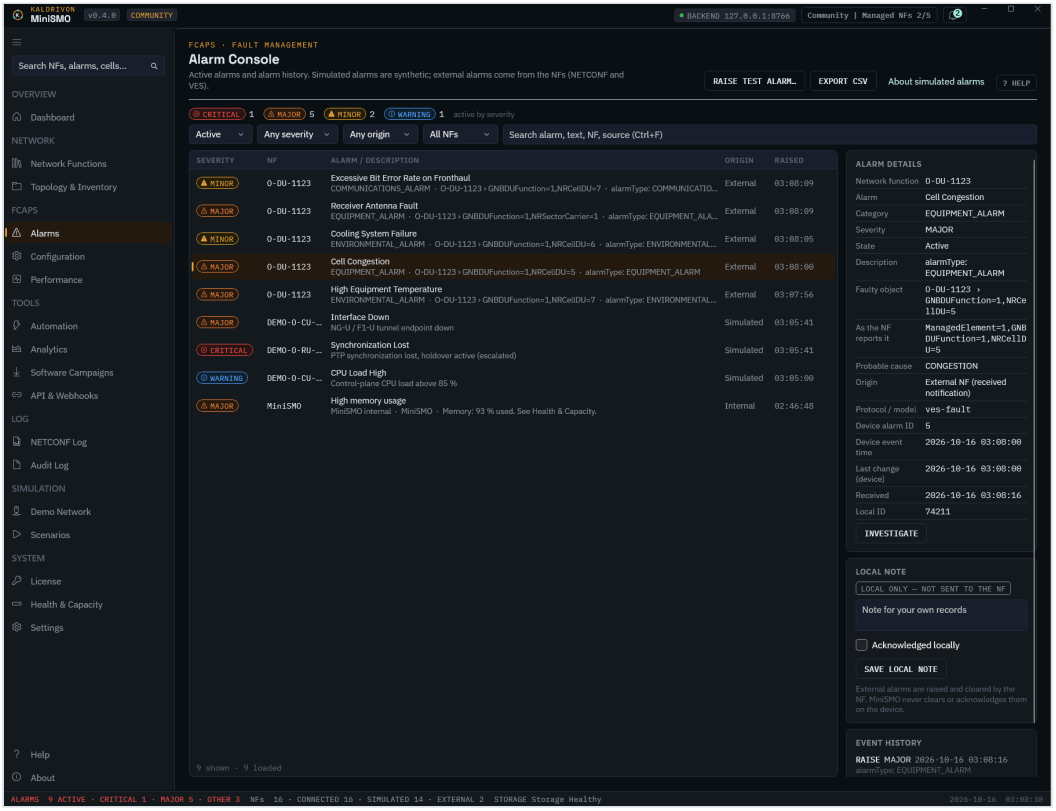
Alarms can only be generated on simulated NFs. Alarms of external NFs arrive by themselves, through NETCONF notifications, alarm-list reads or VES fault events. See External Alarms.

Internal alarms (origin *Internal*, source MINISMO) are raised and cleared by MiniSMO itself:

Alarm	Raised when	Cleared when
Loss of supervision (MAJOR, on the NF)	the monitoring session of the NF is down for more than 30 s	the session is connected again, monitoring is stopped, or the NF is deleted
VES heartbeat missed (MINOR, on the NF)	an NF that sends VES heartbeats misses three intervals	heartbeats arrive again, monitoring is stopped, or the NF is deleted
Storage usage high, High memory usage, High CPU usage, Notification queue backlog, Events dropped by the event bus, Operations jobs failed, Webhook deliveries failed (on *MiniSMO*)	the matching System health check is degraded or critical in two consecutive one-minute readings	the check is healthy in two consecutive readings
VES collector not listening (MAJOR)	a VES listener could not start	it is listening

Use the *Internal* origin filter to see only these. The supervision alarms (Loss of supervision, VES heartbeat missed) belong to the NF: they count in alarm correlation and in *Most alarms* on *TOOLS* > *Analytics*. The health alarms are raised on MiniSMO itself and are left out of both.

For external NFs, **RESYNC ALARMS** reads the NF's alarm list again. See Monitoring External NFs.



Alarms: active alarms of external and simulated NFs

27.2 Alarm History

The Alarm Console shows the newest 500 alarms and refreshes them every few seconds. Older alarms stay stored, also across restarts of MiniSMO, for as long as FM retention keeps them (*SYSTEM* > *Settings* > *Storage & Retention*; the presets keep 30, 90 or 365 days).

HISTORY… on the Alarm Console opens a search over all of them. Nothing is read until you select **SEARCH**:

- **From** and **To**: date and time (24-hour), in this PC's time zone; the *To* minute is included. The default is the last seven days. An alarm is listed when its life overlaps the range: raised before the end of the range and cleared after its start, or still active.
- **Search**: part of the title, text, category, object, probable cause, the NF's alarm ID or the NF name (case-insensitive). **Enter** searches too.
- **State** (all, active, cleared) and **Severity**.

The list shows raised and cleared time, severity, NF, title with text, and object, newest first, 200 at a time; **LOAD MORE** reads the next 200. Select an alarm to see its details below the list, the same as in the Alarm Console (description, faulty object, probable cause, origin, model, device alarm ID and times), and for an alarm from an external NF also its event history and the last notification as received. The line above it says how many alarms match, how long alarms are kept and when the oldest stored alarm was raised. **EXPORT CSV…** writes the alarms of the last search (the same range and filters, all of them, not only the ones shown; at most 100,000) to a CSV file in the exports folder, after you confirm in the window. Columns: ID, NF, title, severity, state, raised and cleared time (UTC, ISO 8601), category, probable cause, object, text, origin, source, device alarm ID, acknowledgement and note. A cell that would start a spreadsheet formula is written as text. The API is `GET /api/alarms/history?since=&until=&q=&state=&severity=&offset=&limit=`, and `GET /api/alarms/history/export` with the same filters for the CSV.

27.3 About Simulated Alarms

The simulator knows ten alarm types, loosely based on what real RAN equipment reports:

Alarm type	Typical meaning on real equipment
VSWR High	Antenna or feeder mismatch, reflected power too high
Cell Unavailable	Cell out of service
Loss of Signal	No RF output or no input on a link
Synchronization Lost	Loss of PTP/SyncE or GNSS timing
High Temperature	Unit above its temperature threshold
Interface Down	Fronthaul or transport interface down
Power Supply Failure	PSU or input power fault

Alarm type	Typical meaning on real equipment
CPU Load High	Processing load above threshold
Memory Usage High	Memory above threshold
High PRB Utilization	Radio resources near saturation (congestion)

Severities follow the usual X.733-style scale: CRITICAL, MAJOR, MINOR and WARNING. A cleared alarm has severity CLEARED.

Scenarios raise and clear alarms automatically; their Source is `SCENARIO:<scenario>`. Stopping or resetting a scenario clears the alarms it raised, but not alarms you generated manually.

All alarm data is synthetic.

28. External Alarms

External alarms appear in the *Alarm Console* together with simulated ones. The **Origin** column and filter tell them apart.

Alarms reach MiniSMO in three ways, and all three go into the same alarm store:

- **Pull.** MiniSMO reads the NF's alarm list (`o-ran-fm` , `ietf-alarms` , a 3GPP `AlarmList` or `openconfig-alarms`) and reconciles it.

OpenConfig alarms. Routers and switches that use OpenConfig models (for example Cisco IOS XE) advertise `openconfig-alarms` . It is a vendor-neutral model published by the OpenConfig operator group, not an SDO standard such as 3GPP, O-RAN or IETF; MiniSMO labels it that way. The active alarms are read-only state under `/system/alarms` , and the model defines no notification, so FM for such an NF is **polling**: the list is read every polling interval (60 s at least) and compared with the previous read. A new entry raises an alarm (title from `text` , faulty object from `resource` , severity from the `severity` identity, raised time from `time-created`); an entry that disappears clears it. The alarm is identified by its `id` . Vendor-specific event modules a device may also advertise are not interpreted. - **NETCONF notifications.** RFC 5277 alarm notifications arrive in real time. - **VES fault events.** Events received by the VES collector are keyed by NF, object DN (`alarmInterfaceA`) and alarm condition. `NORMAL` severity clears the alarm.

When monitoring starts, MiniSMO reads the device's active alarm list (`o-ran-fm` `active-alarm-list` or `ietf-alarms` `alarm-list`). This is the *initial synchronization*. If the list cannot be read, FM shows *sync incomplete*: alarms raised before monitoring started may be missing until they change.

After that, alarm notifications update the current alarms:

- A new alarm identity raises an alarm.
- The same identity with new severity or text updates it.
- A clear clears it.

The identity is the device's own key: `fault-id` + `fault-source` + affected objects for O-RAN, resource + alarm type + qualifier for `ietf-alarms` , or the object DN + `alarmId` for 3GPP alarms. `alarmId` alone is not unique, so it is always combined with the object. A 3GPP alarm seen both as a VES fault and in the NETCONF AlarmList (with its `objectInstance`) is therefore one alarm.

Names and labels come from the alarm itself, never from a built-in table:

Shown as	**3GPP AlarmList / stndDefined**	**VES** `faultFields`	`ietf-alarms` **(RFC 8632)**	`o-ran-fm`
Alarm (title)	`specificProblem` , else `alarmType`	`specificProblem` , else `alarmCondition` , else `eventName`	`alarm-text` , else the alarm type	`fault-text`

Shown as	3GPP AlarmList / stndDefined	VES `faultFields`	`ietf-alarms` (RFC 8632)	`o-ran-fm`
Category	`alarmType`	`alarmCondition`	`alarm-type-id` (+ qualifier)	
Faulty object	`objectInstance` / `href`	`alarmInterfaceA`	`resource`	`fault-source`
Probable cause, raised time	`probableCause`, `alarmRaisedTime`	`alarmAdditionalInformation.probableCause` / `.alarmRaisedTime`	`time`	`event-time`
Device alarm ID	`alarmId`	`alarmAdditionalInformation.alarmId`	`alarm-type-id` (+ qualifier)	`fault-id`

The title is always the text the device gives operators to describe the fault, and the category its alarm type, whichever protocol the alarm arrives by. This follows how the O-RAN Software Community maps an O-RAN alarm to a VES fault (`fault-id` to `alarmCondition`, the fault description to `specificProblem`).

The alarm ID is shown as the device's ID, never as the title. When a device sends no descriptive text at all, the title says so ("Unspecified alarm").

Some devices (the O-RAN-SC NTS simulators, for example) keep a 3GPP AlarmList whose records carry only `alarmId` and severity, and describe the same alarms in VES faults. For an NF that reports by VES (it was pointed at MiniSMO's VES collector, or VES events have arrived from it), such a record updates or clears the matching VES alarm and is never shown as an alarm of its own. When an NF is added, MiniSMO points it at the VES collector *before* it reads the AlarmList for the first time, so these records never appear; any that were shown earlier are removed at the next AlarmList read. For an NF that has only the AlarmList, those records are its only FM data and are shown as "Unspecified alarm".

3GPP AlarmList devices. Some 3GPP NRM devices (O-RAN-SC NTS simulators, for example) keep alarms as `alarmRecords` inside the running configuration. They signal a change with a `netconf-config-change` that points at a record's `perceivedSeverity`. MiniSMO treats these as alarm events, not configuration changes: shortly after such a notification it re-reads only the AlarmList (subtree filter) and records the differences as raise, update or clear. The list is read at most every 5 s, or every 30 s when the NF also reports its faults by VES (the list then only confirms them); notifications in between are coalesced into the next read. A record whose severity is `CLEARED` is a cleared alarm. If a record changes several times between two reads, only the latest state is seen, but every change notification is still counted. Some of these devices publish only `alarmId` and severity; the other columns stay empty then. Such a record is merged with the NF's VES

alarm of the same `alarmId` instead of appearing twice, and an AlarmList read never clears alarms that arrived by VES. When the AlarmList's `administrativeState` is LOCKED, the Monitoring tab says the list is in maintenance: its content may not reflect current faults.

One alarm source per NF. Some NFs report every fault twice: as a NETCONF alarm notification and as a VES fault event. Neither O-RAN nor 3GPP defines how a manager should merge the two (their identities differ), so MiniSMO takes each NF's alarms from one route:

- **NETCONF**, when FM is monitored with `ietf-alarms` or `o-ran-fm`, the session is up and alarms have arrived that way from this NF. These models can be resynchronized from the NF's own alarm list after a disconnection; VES cannot.
- **VES** otherwise: no NETCONF FM, the NF reports its faults only by VES, or the NETCONF session is down (so faults during an outage are not missed).

While NETCONF is the source, the NF's VES faults are kept in the VES event log ("no alarm raised: … reports its alarms over NETCONF") and raise no second alarm. Updates and clears of an alarm that VES raised earlier are still applied. The NF's Monitoring tab shows the choice under **Alarm source**. A 3GPP AlarmList is not a separate route: it mirrors the VES faults and is merged with them (see below).

Every event is kept in the alarm's **event history**: raise, update, clear, snapshot and resync. The history also keeps events that arrive late. An update older than the alarm's latest state is stored as *out of order* and does not overwrite newer state. A clear for an alarm MiniSMO never saw is stored as *clear unmatched*.

The details panel shows the device's event time next to the time MiniSMO received the event, and a copy of the notification (limited in size).

29. Configuration Management

29.1 Configuration Management

FCAPS > *Configuration* shows and edits NF configuration. What you can do depends on the NF's mode.

Simulated NFs (editable)

Select a simulated NF in the NF list. The fields available depend on its type:

Field	Range / format	O-RU / O-DU	O-CU-CP / O-CU-UP
Administrative State	UNLOCKED or LOCKED	✓	✓
Cell Local ID	0 – 16383	✓	
PCI	0 – 1007	✓	
NR ARFCN	0 – 3279165	✓	
Bandwidth	5 – 100 MHz (standard NR channel widths)	✓	
Tx Power	0 – 49 dBm	✓	
TAC	0 – 16777215	✓	✓
MCC	3 digits	✓	✓
MNC	2 or 3 digits	✓	✓
Software Version	Free text	✓	✓

The buttons work like a small candidate/commit workflow:

Button	What it does
GET	Reloads the running configuration from MiniSMO's database
VALIDATE & STAGE	Validates your changes and stores them as a pending candidate. Running is unchanged
APPLY	Makes the candidate the new running configuration and increases the revision number
REVERT	Discards a pending candidate; if there is none, rolls running back to the previously applied revision

Invalid values (for example PCI 1500) are rejected with a message naming the field and its allowed range.

Setting **Administrative State** to LOCKED and applying it makes the NF's operational state DISABLED; its traffic counters drop to zero. Unlock it to bring it back.

Configuration is stored in the local SQLite database and survives restarts. The Demo Network uses the test PLMN 001/01.

HISTORY shows every applied change for the selected NF: timestamp, revision, field, old and new value, and source. See Configuration History.

External NFs

For an external NF nothing is read from the NF when you open the page or select the NF: a `<get-config>` and the YANG behind the parameter details load the NF, the link and MiniSMO, so they run only when you select **READ (GET-CONFIG)**. Until then the page shows the stored snapshot history and the datastores the NF announced. After READ the configuration is shown in the same parameter form. Tabs: **Parameters**, **XML tree**, **Raw XML**, **History** and **NETCONF read**. What you read stays while MiniSMO runs, for each NF: after visiting another page or selecting another NF and coming back, the configuration you read (with its read time in the status), the NETCONF read result and any changes you had not applied yet are still there, and the page reopens on the same NF and tab. Nothing is read from the NF again until you select READ.

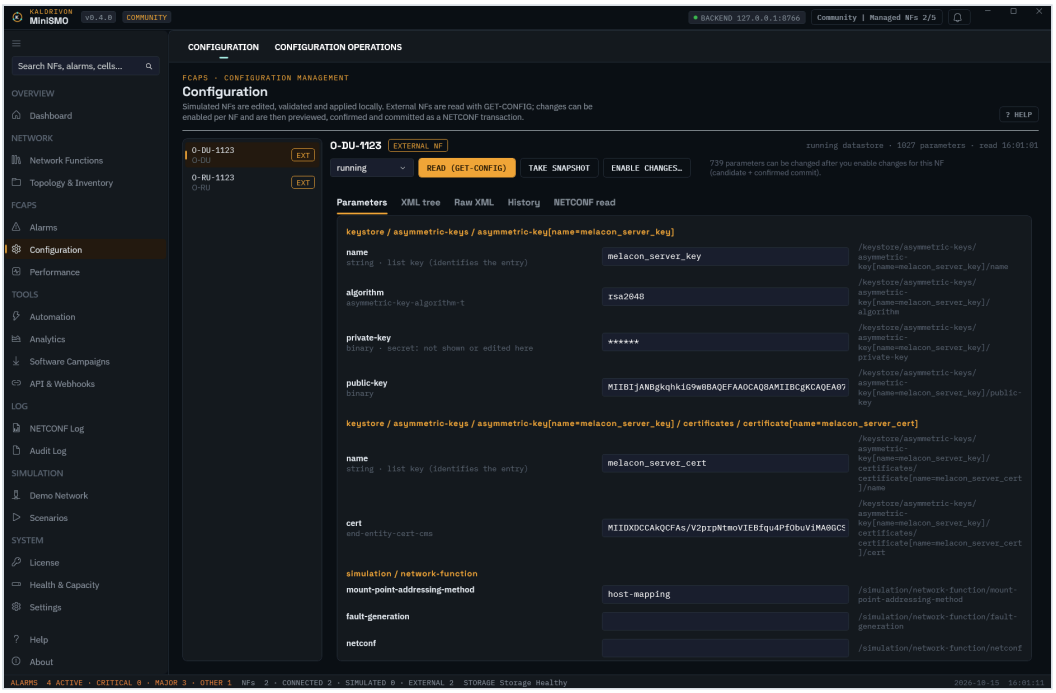
The tree on the left. Each NF is a node. After READ, the NF's configuration objects appear below it as the NF returned them: every list entry is a node (for example `ManagedElement=1` › `GNBDUFunction=1` › `NRCellDU=3`, and below a cell any list entries it has; `interface eth0` for an ietf-interfaces device). Select the NF to see the parameters that are in no list entry; select an object to see its own parameters, with the path at the top of **Parameters**. Everything starts collapsed. Use the arrow to open one level, or **right-click** a node (an NF or an object) to open it with everything below it; right-click again to close it all. Your choice is kept per NF. Each node shows its number of parameters and of changed values (including its sub-objects). **Default values.** A leaf at its YANG default is not stored in the NF's running configuration, so a plain `<get-config>` does not return it. A cell's `administrativeState` is the usual example: TS 28.541 defines its default as LOCKED. When the NF announces it (RFC 6243 `:with-defaults`), MiniSMO reads the parameters with `report-all-tagged` (or `report-all`), so these leaves are listed too, marked *default value (not set on the NF)*, and can be changed like any other: setting a cell to UNLOCKED writes `administrativeState` to the NF. Without that capability only the values the NF stores are shown. The **Raw XML** view shows the plain `<get-config>` reply.

The tree only changes what is shown: READ always reads the whole configuration, and changes in every object are part of the preview.

Button	What it does
Datastore box and **READ (GET-CONFIG)**	Reads the configuration from the selected datastore
TAKE SNAPSHOT	Asks for confirmation, reads the running configuration and stores it as a new version if it changed
ENABLE CHANGES... / **DISABLE CHANGES**	Allows or stops configuration changes on this NF (off by default)
REVIEW CHANGES...	Shows the changed values and the exact `edit-config`, then **APPLY TO NF**

Button	What it does
DISCARD EDITS	Drops your unsent edits

On the **History** tab, **EXPORT** asks for confirmation, writes the selected snapshot (secrets masked) to the exports folder and shows **Open file** / **Show in folder**. See Changing External Configuration and How MiniSMO Writes to External NFs.



Configuration: an external NF after READ (GET-CONFIG)

29.2 How MiniSMO Writes to External NFs

Monitoring only reads: connect, discover capabilities, `<get>`, `<get-config>`, `<get-schema>` and subscriptions. Free-form writes are never sent, because configuration changes on real equipment without schema validation, locking and rollback can take a live cell or fronthaul link down.

Two configuration writes exist, each limited to exactly what it needs:

- **VES destination.** When you add an NF, MiniSMO writes the collector address into the NF's own VES settings (only the leaves of that one container) and reads them back. Turn this off in *SYSTEM* > *Settings* > *General*, section **Integration**. See Pointing an NF at the Collector.
- **Configuration changes you enable for an NF.** Each change is validated against the NF's YANG, reviewed and confirmed, then applied in a transaction with read-back. See Changing External Configuration.

A third request acts on the NF without changing its configuration: **REQUEST UPLOAD…** under **FILES ON NF…** sends an o-ran-file-management `file-upload` after you confirm it, so the NF pushes one measurement file to your upload target. See Measurement Files.

Simulated NFs are fully editable on the *Configuration* page. See Configuration Management.

30. External Configuration History

For external NFs, *Configuration* shows the configuration read with `get-config` in the same parameter form as simulated NFs, which is editable once you enable changes for that NF. The **XML tree** and **Raw XML** views are also there.

The **History** tab lists stored **snapshots** (versions). A new version is stored only when the configuration actually differs. Snapshots are taken:

- when monitoring starts,
- after a change notification,
- when you choose **TAKE SNAPSHOT** (after confirmation),
- by explicit polling, if you enabled it.

Change notifications (RFC 6470) tell MiniSMO *which paths* changed and who changed them. They do not carry old or new values. MiniSMO reads a new snapshot a couple of seconds after a change notification, and at most once every 30 seconds while changes keep coming, so one snapshot can cover several changes. Some NFs report changes continuously without changing their running configuration (for example operational values); while snapshots keep finding the same configuration, the interval doubles up to 15 minutes, and the next real change brings it back to 30 seconds. Every change notification is still recorded. On 3GPP AlarmList devices, changes inside the AlarmList are alarm events (see *External Alarms*): they create no configuration history entries, and the AlarmList is left out of snapshots. **COMPARE** shows the net difference between two versions (changed, added and removed leaves, plus a text diff). It does not claim an exact event-by-event history.

Secret values are replaced with `******` in snapshots, diffs, exports and the NETCONF Log: passwords, secrets and passphrases, pre-shared keys, SNMP communities and keys, key-chain keys (`key-string` , `keystring` , `hexadecimal-string`), cleartext keys of `ietf-crypto-types` , routing-protocol authentication keys, tokens and API keys. MiniSMO recognises them by the leaf name; a vendor leaf that holds a secret under another name is not masked, so check a snapshot before you share it. A snapshot cannot be pushed back to the NF from **History**; changes go through Changing External Configuration or Rollback. **EXPORT** asks for confirmation and saves a local copy of a snapshot.

31. Changing External Configuration

Configuration changes on an external NF are off until you allow them for that NF. In *Configuration*, select the NF and choose **ENABLE CHANGES...**, then confirm. **DISABLE CHANGES** makes it read-only again.

What can be edited. The parameters are the NF's own running configuration (`get-config`), not a list built into MiniSMO. Names, list keys, types and constraints come from the NF's YANG modules. MiniSMO reads them with `get-schema` , or uses models you imported under *YANG Models*, and caches them locally. Next to each value you see its range, length, units or choices. These are never edited:

- list keys (they identify the entry),
- `config false` state,
- secrets (shown as `******`),
- leaf-lists,
- the 3GPP AlarmList (alarm data),
- entries of a list whose YANG is not available to MiniSMO and whose key the defining standard does not fix (3GPP NRM lists are keyed by `id` , ietf-interfaces by `name`): such entries are only numbered, never identified by guessing a key.

When a module cannot be read, its parameters show no constraints and the NF alone validates them.

Making a change.

1. Edit values; the running value stays visible next to each field.
2. Choose **REVIEW CHANGES...**. MiniSMO re-reads the running configuration and checks each value against the YANG constraints. It refuses a value that changed on the NF since you read it (someone else edited it). You then see each old → new value and the exact `edit-config` .
3. **APPLY TO NF** runs a NETCONF transaction: - With `:candidate` : lock the candidate, `edit-config` (merge), `validate` (if `:validate` is advertised), a **confirmed commit** (`:confirmed-commit`), read the running values back, then the confirming `commit` , and unlock. If a value does not read back as requested, MiniSMO cancels the commit (`cancel-commit`) or lets the NF roll back when the confirm timeout expires. If a step fails, the candidate is discarded. - With `:writable-running` only: lock running, `edit-config` with `test-then-set` (if `:validate:1.1`), read back, unlock.
4. The result lists every step. For a refused change it shows the NF's `rpc-error` as returned (`error-tag` , `error-path` , `error-message`).

The `edit-config` contains exactly the leaves you confirmed, plus the list keys that address them, merged into the existing configuration. MiniSMO checks the XML actually sent against the confirmed plan. Any other leaf, value or list entry, or any delete or replace operation, is refused before it leaves the application.

After a change, MiniSMO stores a new configuration version and a change entry by *MiniSMO (this PC)* in **History**. If you monitor the NF, its own `netconf-config-change` notification appears there too.

32. Performance Management

32.1 Performance Management

FCAPS > *Performance* shows performance data for simulated and external NFs. This section covers simulated NFs; for external NFs see External Performance Data.

Select a simulated NF on the left. The page shows six boxes with the latest values, and a chart of the selected counter. Click a box to chart its counter. Right-click a box to show another counter in it; choosing one that another box already shows swaps the two, and **Default boxes for this NF** goes back to the first six. The choice is remembered per NF on this PC. External NFs use the same boxes, at most six and never more than the measurements the NF reports, so no empty box is shown (see External Performance Data). Choose the chart range: 15 minutes, 1 hour, 24 hours, 7 days, 30 days or 90 days. Short ranges use raw samples; longer ranges use 5-minute or hourly aggregates and show the min/max band around the average.

Counter	Unit	Notes
PRB DL %	%	Downlink physical resource block utilization
PRB UL %	%	Uplink PRB utilization
Active UEs	count	Connected users
DL Mbps	Mbit/s	Downlink throughput
UL Mbps	Mbit/s	Uplink throughput
CPU %	%	Processing load
Memory %	%	Memory utilization
Packet Loss %	%	Transport packet loss
Temperature	°C	Unit temperature

O-CU-CP reports no PRB or throughput values (it is control plane only).

Collection interval: choose 1, 5, 15 or 30 seconds. The live tiles update every second either way; the interval controls how often a sample is stored in history (and therefore the chart resolution). The interval is also available in Settings as *PM Update Interval*.

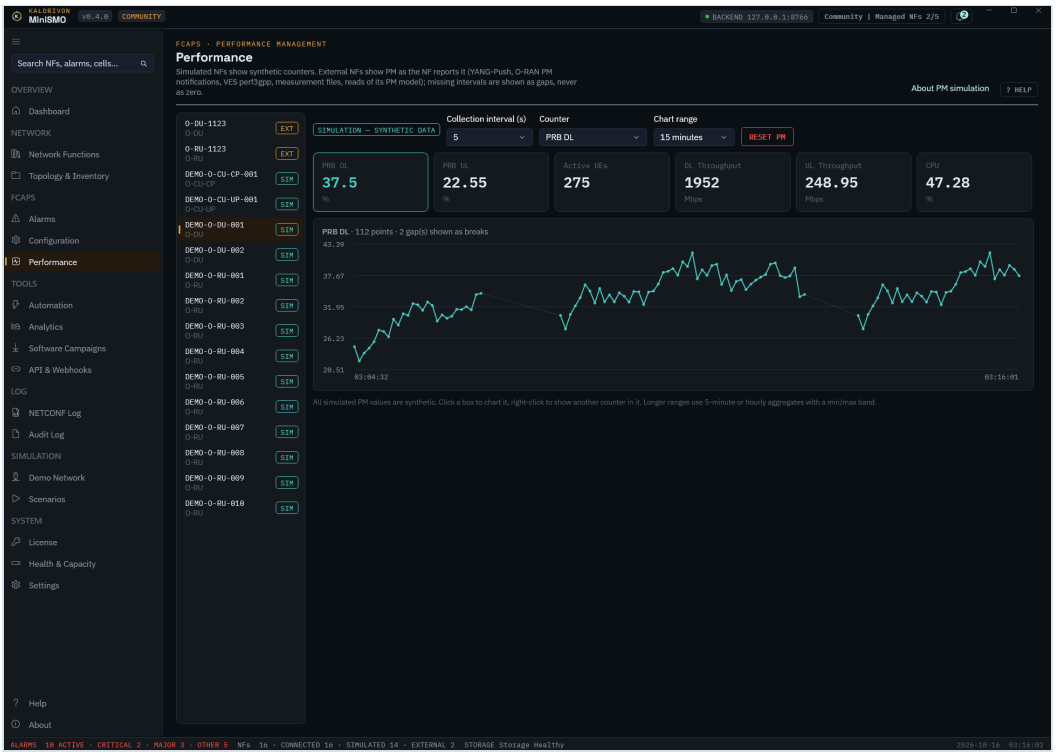
While MiniSMO is not running, the simulator produces no samples. The chart shows that time as a break in the line, not as a straight line across it, and the caption counts the gaps. A change of the collection interval is not a gap.

Reading the chart. Point at the chart to see the time and value of the nearest sample (with min and max for 5-minute and hourly aggregates, and every NF when comparing). A missing interval breaks the solid line; a faint dashed line bridges the break so you can follow the series, but it carries no values. When a series has few samples (external PM arrives once per measurement period, typically every 15 minutes), every sample is drawn as a dot.

Retention: raw samples are kept for 6 hours, 5-minute aggregates for 7 days and hourly aggregates for 30 days by default (the MINIMAL preset). See PM Retention.

RESET PM deletes all stored PM history for all NFs after confirmation. Live generation continues.

EXPORT CSV asks for confirmation, writes the samples to the exports folder and shows **Open file** / **Show in folder**.



Performance: a simulated NF with counter tiles and chart

32.2 About PM Simulation

The simulated counters are designed to look like a real network, not random noise:

- Each NF follows a slow load curve (a roughly 10-minute cycle with a shorter ripple), with a different phase per NF so cells don't move in lockstep.
- Small random variation is smoothed, so values drift instead of jumping.
- CPU, memory and temperature follow load more gently than traffic does.
- When a scenario changes a counter, the value moves towards the new level over a few seconds instead of snapping.
- A LOCKED or DISABLED NF carries no traffic: PRB, UEs and throughput go to zero.

The absolute values are plausible for a mid-band 100 MHz carrier but are synthetic and not tied to any vendor's equipment.

33. External Performance Data

33.1 External Performance Data

Performance lists external NFs next to simulated ones. For an external NF, pick the **Metric** (the NF's own measurement names, for example `RRU.PrbUsedDl`) and then the **Managed object**. Pick the object from the list or from the **Managed objects** tree, where objects are grouped under their parent in the DN (for example the `NRCellDU` cells under their `GNBDUFunction`, the O-DU). Groups expand and collapse with a click. Then choose a time range and, for counters, rate or raw value. Values the NF flagged as suspect (`suspectFlag`) are marked.

Source	Metrics
YANG-Push periodic	Every numeric leaf under the subscribed XPath (default: interface statistics). List keys identify the resource, for example `interface eth0`. Keys, counter types (RFC 6991 `counter32` / `counter64`) and units come from the NF's YANG for that data; while it is not available, only what the defining standard fixes is used (RFC 8343 interface counters, `name` for interfaces, `id` for 3GPP NRM lists) and other values are plain gauges
O-RAN PM notifications	`measurement-result-stats`: transceiver statistics (min, max, first, latest per unit; no unit is shown, because the module gives none as a YANG `units` statement) and window counts (RX_ON_TIME, TX_TOTAL ...). The O-RU sends them only when its `notification-interval` is configured (o-ran-performance-management); that interval is what MiniSMO expects (*expected every*), and a silent source becomes STALE after three intervals. Without `notification-interval` (absent or 0) the O-RU sends no periodic notifications: the source shows **NOT_PROVISIONED**, is never stale and raises no notification, and the results are read with `<get>`. MiniSMO never sets `notification-interval` itself
Pull from a PM model the NF describes	MiniSMO asks the NF which PM models it has: it reads the YANG of the NF's non-IETF modules (YANG library and `<get-schema>`) and offers every `config false` container whose YANG declares measurement data: at least one leaf typed with an RFC 6991 counter or gauge (`yang:counter32`, `counter64`, `zero-based-counter32/64`, `gauge32`, `gauge64`). Names are never interpreted: a container is not PM because it is called *pm* or *statistics*, and plain operational state is left out. Only the numeric leaves the YANG declares in that container are read. Units come from the YANG `units` statement only, counter types mark cumulative values, and a `yang:date-and-time` leaf gives the measurement time. A model that does not declare its data this way can still be read with your own polling filter. These sources are marked *found in the NF's YANG* in the PM sources list and can be switched off one by one. Nothing about them is built into MiniSMO

Source	Metrics
Where a discovered PM model lives	Read with a `<get>` at the interval you set, else the **PM read period** setting (300 s by default, *SYSTEM* > *Settings*). Labelled PULL. A PM module often only augments the 3GPP NRM (for example a container under each `NRCellDU`), so MiniSMO never reads it from the module's own root. It resolves the data path through the NF's YANG augment targets, or the instance tree when the modules cannot be read, and filters from the managed-object root with the right namespace at each level. Values are kept per managed object, identified by its list keys from the YANG. The Monitoring tab shows the path it uses
O-RU measurement results (O-RAN WG4)	For O-RUs with `o-ran-performance-management`: `performance-measurement-objects` read with `<get>` (the results are `config false`, so never with `<get-config>`), once per measurement interval of the O-RU: the shortest `*-measurement-interval` in its own configuration (for example `rx-window-measurement-interval`), else the **PM read period** setting; a polling interval you set when starting monitoring always wins. The measurement intervals say how often the O-RU measures, not whether or how often it reports, so a read source has no *expected every*. How the O-RU reports is decided by separate leaves: `notification-interval` (notifications), and `enable-file-upload` (or the older `enable-SFTP-upload`) with `file-upload-interval` and `remote-file-uploads` (result files). The namespace is the one the O-RU's YANG library gives. Each `rx-window-measurement-objects` entry is stored per object unit: **RU** (the ietf-hardware component, for example `chassis`), **TRANSPORT** (`transport <flow name>`) or **EAXC_ID** (`eAxC <id>`). Measurement names are shown readably with the enum kept, for example `Rx on time (RX_ON_TIME)`. These counts are cumulative, so the change between polls is shown (rate or raw). `transceiver-measurement-objects` give min, max, first and latest per `transceiver <unit id>`; an O-RU without transceiver entries simply has none. Only entries the O-RU returns are used. There is no 3GPP DN: the values belong to the NF and its object unit. `measurement-result-stats` notifications are stored as well when the O-RU sends them
Measurement files	3GPP `measCollecFile` per granularity period, signalled by VES fileReady, 3GPP notifyFileReady or NETCONF file-upload notification, see Measurement Files. Labelled FILE
VES perf3gpp	3GPP TS 28.550 streaming PM received by the VES collector: each `measResults` value is matched to its `sMeasTypesList` name and stored per `measObjInstId` and granularity period. What is expected is read from the NF, see below
Polling fallback	Numeric leaves returned by the subtree filter you give, labelled POLL

VES perf3gpp: what is expected

In 3GPP, an NF reports measurements because a **measurement job** is configured on it: a `PerfMetricJob` of `_3gpp-common-measurements` (TS 28.622 / TS 28.623), with its `granularityPeriod` (seconds), the metrics, and either stream-based reporting

(`streamTarget` : VES perf3gpp) or file-based reporting (`fileReportingPeriod` , minutes). When the NF announces `_3gpp-common-measurements` , MiniSMO reads its jobs with `<get>` when monitoring starts (an XPath filter by local name if the NF supports `:xpath` , otherwise a subtree filter for its `ManagedElement` / `SubNetwork`) and shows them on the Monitoring tab under **3GPP PERFMETRICJOBS ON THE NF**. `PerfMetricJob` comes from a YANG grouping, so it carries the namespace of the list that contains it (the `ManagedElement` or the function), not that of `_3gpp-common-measurements` .

- **VES perf3gpp** source: expected every `granularityPeriod` of the unlocked stream-based jobs. A silent source becomes STALE after three periods.
- **Measurement files** source: expected every `fileReportingPeriod` of the unlocked file-based jobs (else their `granularityPeriod`).
- No job of that kind: the source waits and says the NF is not configured to report that way. MiniSMO creates a job only when you ask for it (below).

Creating a measurement job

MEASUREMENT JOBS... on the NF's Monitoring tab follows the 3GPP process (TS 28.550, NRM in TS 28.622, YANG in TS 28.623):

1. **Query.** MiniSMO reads the NF's `SupportedPerfMetricGroups` (read-only, under its `ManagedElement` and its functions such as `GNBDUFunction`). Each group lists the `performanceMetrics` the NF can produce, its `granularityPeriods` , `reportingMethods` and `reportingPeriods` . `SupportedPerfMetricGroups` is an optional advertisement, not a prerequisite: see *NF without SupportedPerfMetricGroups* below.
2. **Placement.** A job can be created under the `ManagedElement` when the NF announces the YANG feature `MeasurementsUnderManagedElement` , under a function when it announces `MeasurementsUnderManagedFunction` . A group of a function with only the first feature gets a job under the `ManagedElement` with `rootObjectInstances` set to that function.
3. **Choose** a group, the measurements, a granularity period and a reporting method from what that group lists. **Stream** sends the results to MiniSMO's VES collector (`streamTarget`); **files on the NF** (`FILE_BASED_LOC_SET_BY_PRODUCER` , `fileReportingPeriod` in minutes, a multiple of the granularity period) are announced by `notifyFileReady` and retrieved by MiniSMO, which turns on File PM retrieval for the NF. `FILE_BASED_LOC_SET_BY_CONSUMER` is not offered: the model's `fileLocation` has no field for the credential MiniSMO's receiver needs.
4. **Preview and confirm.** MiniSMO shows the exact `edit-config` (job id `MiniSMO-<n>` , `administrativeState` UNLOCKED) and sends it only after you confirm; the NF validates it as well. The PM paths are updated right away.

The same dialog lists the NF's jobs. For a job MiniSMO created:

- **MODIFY...** locks or unlocks it (`administrativeState` ; a LOCKED job stays on the NF but collects nothing) and, when the NF reports the supported group the job belongs to, changes its measurements, granularity period and, for a file job, its file reporting period, again only to values that group lists. `jobId` cannot change (it is invariant in TS 28.622). Only the changed values are sent as an

`edit-config` merge; measurements you remove are deleted from the `performanceMetrics` list one by one (`operation="delete"`), so nothing else of the job is touched. MiniSMO shows the exact `edit-config` , sends it only after you confirm, then reads the job back. If the NF reports no matching group, only lock / unlock is offered.

- **DELETE...** removes it.

Other managers' jobs are never changed.

NF without SupportedPerfMetricGroups (discovery first, catalog fallback). Some NFs do not report the groups (for example an nts-ng O-DU whose compiled schema lacks the read-only node). The job model itself does not depend on them: TS 28.622 defines `performanceMetrics` as measurements defined in TS 28.552 and identified by name. So for such an NF, MiniSMO offers **New job (measurements named from TS 28.552)**:

- type the measurement names (one per line or comma separated), or click the names this NF already shows: those in its existing jobs and those in measurements it already sent to MiniSMO. MiniSMO checks the TS 28.552 name form (`Family.Measurement` , e.g. `DRB.UEThpDl` , `RRU.PrbUsedDl`) and nothing else;
- enter the granularity period in seconds (the periods of the NF's existing jobs are shown), the reporting method and, for files, the file reporting period in minutes (a multiple of the granularity period);
- the job goes under the `ManagedElement` when the NF announces `MeasurementsUnderManagedElement` , otherwise where the NF already has a PerfMetricJob. Without either, no place is known and no job is offered.

The NF validates the job (TS 28.550). A simulator that does not validate accepts any well-formed name and may produce no values for it. **MODIFY...** on such a job works the same way (names and periods entered, lock / unlock); **DELETE...** as usual. The same rule applies to every NF; nothing is vendor specific. An NF with an unresolved identity conflict receives no job (see Identity Mapping). - The NF does not have the module, or the jobs cannot be read: the period the data itself states is used (`measDataCollection.granularityPeriod` of the events, the `granPeriod` of a file). With no stated period at all, nothing is expected and nothing becomes stale.

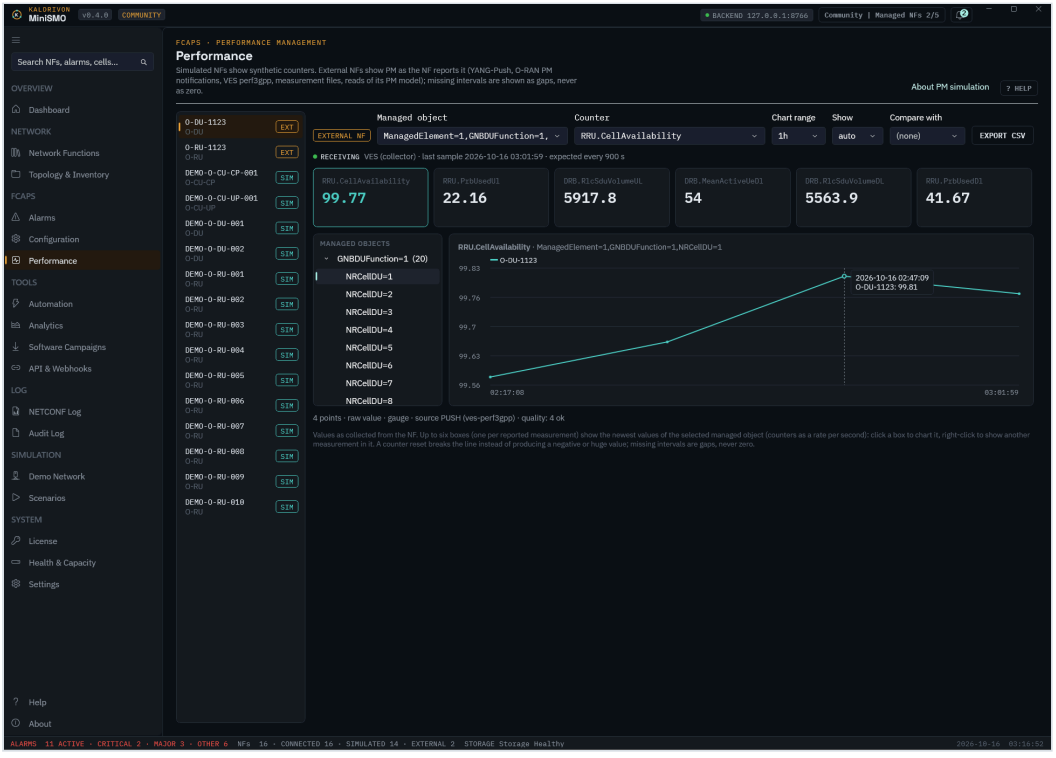
The page for an external NF is laid out like the one for a simulated NF: **Managed object** and **Counter** selectors, **Chart range** (15m to 30d), **Show** (auto, value or rate), **Compare with** another NF and **EXPORT CSV**; up to six boxes with the newest values of the selected managed object (one per measurement the object reports, so fewer when it reports fewer) (a counter as its rate per second, with its unit from the YANG or the standard); and the chart of the selected counter. Click a box to chart that counter. Right-click a box to choose which measurement it shows, from the names the NF actually reported for that object; when another object is selected, a measurement it does not report is replaced by one it does. The choice is remembered per NF on this PC. Next to the chart, the **MANAGED OBJECTS** tree groups the objects under their parent in the DN. Tiles and chart refresh every 10 s, the counter list every 30 s; external PM arrives once per measurement period, not every second.

Counters (leaves the NF's YANG types as counters, and counter results of the standards above) are shown as a rate per second by default. A counter that goes down is marked as a **reset**, and no rate is drawn across it. Missing intervals are drawn as **gaps** (breaks in the line), never as zero. Late samples are stored but flagged *out of order*, and duplicates are ignored.

The freshness indicator shows when the last sample arrived and whether it is Receiving or Stale. **EXPORT CSV** asks for confirmation, writes the samples with their source, method and quality flags, and shows **Open file** / **Show in folder**.

External PM samples, including values from measurement files, are kept as long as the 5-minute PM aggregates (7 days by default) and are not aggregated. They are stored compactly: the counter name and managed object once per series, each value as a small row keyed by series and time (about 35–40 bytes per value). See PM Retention.

O-RAN radio units only count measurements that are activated in their configuration (`performance-measurement-objects` : `active` , measurement intervals, notification interval). The Monitoring tab shows the settings MiniSMO read; it does not change them. If no results are returned or PM stays *Stale*, activate the measurements with the device's own tools. File-based PM is a separate mechanism, see Measurement Files.



Performance: an external O-DU, counter tiles with the names the NF reported, and chart

33.2 Measurement Files (File-based PM)

Bulk PM: the NF writes one 3GPP measurement file (`measCollecFile` , TS 32.432 / 32.435) per granularity period and signals that it is ready. This is separate from the live counters read with `<get>` . MiniSMO takes file names, locations, counters and object DNs from the signals and the file content; nothing about them is built in.

How a file reaches MiniSMO

1. **Collection is configured on the NF** (`o-ran-performance-management` : active measurements, intervals, and the upload target in `remote-file-uploads` , or the deprecated `remote-SFTP-uploads` , which is ignored when `remote-file-uploads` is present). MiniSMO reads and shows this configuration under **SETTINGS...**; it never changes it.
2. **The NF signals the file**, by any of: - a VES `notification` event with `changeType` FileReady: each `arrayOfNamedHashMap` entry gives the file `name` and its `location` . The NF is identified from `sourceName` / `sourceId` , like any VES event. - a VES `stndDefined` 3GPP `notifyFileReady` (`fileInfoList[].fileLocation`). - a NETCONF `file-upload-notification` (`o-ran-file-management`) on the monitoring session's event stream.
3. **MiniSMO retrieves the file** from the location the NF gave: `sftp://` , `ftpes://` (FTP over TLS, encrypted data channel), `https://` or `http://` (credentials only over https). Plain FTP and redirects are not used.

File server identity. Before MiniSMO sends a password to a file server, it checks that the server is the one you expect:

- **SFTP**: the server's SSH host key. On the NF's own address the NF's confirmed NETCONF host key is accepted (the same SSH server). Any other key must be confirmed once.
- **ftpes:// and https://**: a certificate that Windows trusts for that name is accepted. A self-signed or private-CA certificate is accepted only once you confirm its SHA-256 fingerprint.

An identity MiniSMO has not seen before is listed on the **FILE-BASED PM** card as *identity not confirmed, nothing was sent to it*, with its fingerprint. Compare the fingerprint with the one the server's administrator reads on the server (its SSH host key fingerprint, or the certificate's SHA-256 fingerprint), then **TRUST...**. Only a fingerprint the server actually presented can be confirmed. The file is retrieved at the next retry (**RETRY**). If a confirmed identity ever changes, retrieval stops with *identity CHANGED* and nothing is sent. **FORGET...** removes a confirmed identity so it has to be confirmed again (for example after the server's key was replaced on purpose). 4. **The file is parsed and stored**: for each `measInfo` , the `measType` positions are mapped to names (both the `<measType p="1">` and the space-separated `<measTypes>` forms; a file that mixes a `<measTypes>` list with `<r p="1">` values is read by position, p counted from 1), and each `measValue` gives the measured object (`measObjLdn`). Values are stored per NF, object, measurement and period end, with the granularity period. `suspect` values are marked. gzip files are unpacked.

Turning it on. On the NF's **Monitoring** tab, **FILE-BASED PM > SETTINGS...**: turn retrieval on and choose the credentials: the NF's NETCONF credentials (common on O-RU simulators) or a separate user name and password for its file server. Passwords are stored protected and never logged. Files are retrieved only from the NF's own address and the servers you list. Until retrieval is on, file-ready signals are recorded with the reason, and nothing is fetched.

Turned on automatically. When discovery picks measurement files as the NF's PM path (they are the highest path it advertises and has provisioned, see Method Choice and Fallback), MiniSMO turns retrieval on by itself, with the NF's NETCONF credentials. Files

signalled during the last hour while retrieval was off are then retrieved. The Monitoring tab says *turned on automatically* and the change is listed under **METHODS & FALLBACKS**. If you turn retrieval off yourself, MiniSMO remembers that and never turns it on again by itself; turn it on in **SETTINGS…** when you want it back.

Copies kept on this PC. Every retrieved or imported file is kept for a short time in the `pm-files` folder of the MiniSMO data folder. An XML file is stored compressed; a file that arrives compressed is kept as received. Next to each file with a copy:

- **SAVE COPY** writes the file, as the NF delivered it, to the exports folder.
- **REPROCESS** parses the copy again without retrieving it from the NF. Values already stored are recognised and not doubled.

The copies are temporary: the values from them are in PM Data and follow the PM retention. The copies are kept for **Measurement file copies kept** (*Health & Capacity* > *Retention*, default 24 hours). The longest this PC allows comes from a quarter of the free space above the low-disk threshold, divided by how many bytes of copies arrived per hour over the last day, capped at 30 days. The Retention page shows that limit and the reason, and a longer value is refused. Cleanup removes older copies every hour; the file list keeps the record of each file.

SHOW IN FOLDER opens the folder on this PC with the NF's kept copies.

Uploads to MiniSMO (O-RAN WG4 O-RU). An O-RU can upload its result files to a server the SMO provides (`remote-file-uploads` in `o-ran-performance-management`). MiniSMO can be that server: **PM file receiver** in *SYSTEM* > *Settings* is an SFTP server on its own port (default 2222). Each NF gets its own user name (`pm-nf<number>`) and a generated password, stored protected, and can only write its own files. Nothing can be read, renamed or deleted, files are limited to 50 MB, and the number of sessions is limited. MiniSMO does not change Windows Firewall; the section shows the PowerShell command to allow the port. The NF logs in with password or keyboard-interactive authentication (both carry the same password). Below the status, **Recent activity** lists the latest connections, logins, refused logins with the reason (for example *closed without sending a password*, or *wrong password (24 characters given, 32 expected)*, never the password itself) and received files; the same lines are in the backend log. When an NF's last login was refused, its **Measurement files** path under METHODS & FALLBACKS says so. MiniSMO then checks (at most every 30 minutes per NF) whether the O-RU's upload entry holds the receiver password, comparing in memory without showing it: if it differs, MiniSMO writes the right one again; if it matches, the note says the O-RU's SFTP client did not log in with it. **POINT UPLOADS AT MINISMO…** makes the same check and lists *password* as a change when the O-RU holds another one.

Automatic. When measurement files are the highest PM path an NF supports (nothing above them is provisioned or applicable) but the NF does not upload them yet, MiniSMO does it by itself: it switches the receiver on if needed and writes the upload destination below. It tries at most every 30 minutes per NF, and the outcome is listed under **METHODS & FALLBACKS** and in the notifications. Turn this off in *SYSTEM* > *Settings*, **PM file receiver**. **POINT UPLOADS AT MINISMO…** on the NF's File-based PM card does the same on request, showing exactly what will be written first:

- `enable-file-upload` = true;

- `file-upload-interval` = the O-RU's own `*-measurement-interval`, because uploading more often only repeats the same period;
- one `remote-file-uploads` entry, `sftp://pm-nf<number>@<this PC>:<port>/pm`, with the receiver password (the password case of the `o-ran-file-management` credential grouping).

The password is masked in the preview and in the NETCONF log. Measurement objects, intervals and other managers' upload entries stay unchanged; only an older MiniSMO entry for the same NF (for example after this PC's address changed) is replaced. An entry for this PC's receiver that this MiniSMO did not write itself, for example one left by an earlier installation or a test build on the same PC, holds a password this receiver does not know, so the O-RU's uploads would be refused or never reach it. MiniSMO treats such an entry as not provisioned and writes it again with its own password (unless the O-RU shows that it already holds it). Entries that point at another PC are never touched. MiniSMO's SSH host key is not written, because it is optional in the model; its fingerprint is shown in case the O-RU needs it. A completed upload is the new-file signal, because MiniSMO owns the target, so no file-ready notification from any other NF is involved. When the NF is deleted, or MiniSMO is reset with de-integration, MiniSMO removes its own upload entry.

When this PC's address changes (DHCP, another Wi-Fi network, a VPN) or the receiver port is changed, MiniSMO notices it within a minute and replaces its own upload entry on each NF it pointed at the receiver, with a notification (*NF: PM file upload destination updated to …*). An NF that cannot be reached is retried every 10 minutes. This uses the same setting as the VES collector: **Follow address changes of this PC** in *SYSTEM* > *Settings* (`ves_follow_address`). After MiniSMO sets or confirms the upload entry, the NF's PM paths show it as provisioned at once, without waiting for the next read of the O-RU's PM settings.

SMO pull. FILES ON NF… sends `retrieve-file-list` for a logical path (O-RAN WG4 defines `O-RAN/PM` for measurement files; use what your NF documents). It lists files and changes nothing. **REQUEST UPLOAD…** asks the NF to push the selected file to the **upload target** you set (your own `sftp://` or `ftpes://` server) with `file-upload`; it is sent only after you confirm, changes no configuration, and is recorded in the audit log. The NF's `file-upload-notification` then names the file, which is retrieved like any other.

IMPORT FILE… stores a measurement file you have on disk for the NF (for example copied from its file server).

Correctness

- A file is ingested once. A file signalled again (same name for the same NF) is counted as a re-notification, not stored twice.
- Values are per period. They are never treated as cumulative across files.
- When the same measurement of the same object and period already arrived another way (VES `perf3gpp`), it is not stored twice; the file reports how many values were already stored.
- The NF is the one that signalled the file. The file's own `managedElement` (`localDn` , `userLabel` , `swVersion`) is shown with it.

- When a file cannot be retrieved (retrieval off, host not allowed, scheme not supported, server refused the credentials, file larger than 50 MB, not a measurement file), the reason is shown with the file and nothing else is affected. **RETRY FAILED** tries again.

Live counters (`<get>`) are for "right now"; measurement files are the history. Both are kept, with their method (PULL or FILE) on every value.

34. Running Scenarios

SIMULATION > *Scenarios* plays scripted situations on the Demo Network so you can watch how they show up in FM, PM and Topology. Scenarios only ever affect **simulated** NFs; external NFs are never touched. The Demo Network must exist before you can run one.

Scenario	What happens	Duration
NORMAL TRAFFIC	Baseline load with natural variation	Until stopped
MORNING TRAFFIC RAMP	Network-wide load climbs from 30% to 140% of normal	3 minutes, then completes
CELL CONGESTION	DEMO-O-RU-003: DL PRB above 95%, more UEs, higher packet loss; *High PRB Utilization* (MAJOR)	Until stopped
CELL OUTAGE	DEMO-O-RU-007 goes operationally DISABLED with no traffic; *Cell Unavailable* (CRITICAL) and *Loss of Signal* (MAJOR)	Until stopped
HIGH CPU LOAD	DEMO-O-DU-001 CPU above 90%, memory and temperature up; *CPU Load High* (MAJOR)	Until stopped
INTERMITTENT LINK FAILURE	DEMO-O-RU-004 fronthaul flaps every 8 seconds: DISCONNECTED/DISABLED with *Interface Down*, then back	Until stopped

Controls:

Control	Effect
RUN	Starts the selected scenario. If another one is running, it is stopped first
PAUSE	Freezes the scenario timeline. Its effects stay in place
RESUME	Continues a paused scenario
STOP	Ends the scenario, restores normal state and clears the alarms it raised
RESET	Stops any scenario, sets all simulated NFs back to ENABLED/CONNECTED and clears all scenario alarms

One scenario runs at a time. The status line shows the state (IDLE, RUNNING, PAUSED, STOPPED or COMPLETED) and elapsed time. Every run is recorded in the database with its start and end time.

Tip: start a scenario, then switch to *NETWORK* > *Topology & Inventory*, *FCAPS* > *Alarms* or *FCAPS* > *Performance*. Scenarios keep running while you move between pages.

35. Lab Simulator Checks

Some checks against a lab simulator can only be done on your network, for example an O-RAN-SC simulator in a Docker container. A practical order:

1. Confirm reachability of the **host port** (not necessarily 830), for example `Test-NetConnection 192.0.2.25 -Port 8830` in PowerShell. The SSH banner should appear with **TEST CONNECTION**.
2. Add the NF with that host port and the simulator's credentials. MiniSMO stores the password protected with Windows DPAPI.
3. **DISCOVER** and compare the capability matrix with what the simulator is expected to support.
4. **START MONITORING** and watch the *NETCONF Log* for `create-subscription` / `establish-subscription` replies and incoming `notification:*` entries.
5. Trigger events with the simulator's own tools (fault generation, configuration change from another client), and check that they appear in the Alarm Console, Configuration history and Performance.
6. For VES: check the VES step in the integration dialog after **SAVE**, then the **VES** card on the NF's **Monitoring** tab and *SYSTEM* > *Settings* > *VES Collector*. Watch heartbeat, fault and perf3gpp events arrive. If they don't, check that the simulator host can reach this PC on the collector port.

O-RAN-SC NTS devices typically sign in with keyboard-interactive, list their modules only in the YANG library, keep alarms in a 3GPP `AlarmList` in the running configuration, expose cell PM counters in their own YANG module, and (in current images) accept the 3GPP `NtfSubscriptionControl` for their VES destination. MiniSMO discovers all of this from the device's YANG library and modules, with no NTS-specific knowledge; you only enter the address, port and credentials.

A container port bound to `127.0.0.1` on the simulator host cannot be reached from another PC. MiniSMO does not change firewall rules on the simulator host or expose containers. The only change it makes to a simulator's configuration is its VES subscription entry, written when the NF is added (unless turned off).

36. Topology View

NETWORK > *Topology & Inventory* > *Topology* draws the network in three layers, top to bottom:

1. **O-CU**: O-CU-CP and O-CU-UP
2. **O-DU**
3. **O-RU**, grouped under the O-DU that serves them

External NFs appear in their layer by type, marked EXTERNAL. MiniSMO cannot read how external NFs are connected (there is no standard model it can rely on), so it lets you place them:

- An external **O-RU** starts in the group **not under an O-DU**. Select **PLACE EXTERNAL NF...**, or right-click the node and choose **Place in topology...**, and tick the O-DU(s) it is connected to over fronthaul.
- An external **O-DU** can be placed under the O-CU-CP (F1-C) and O-CU-UP (F1-U) that serve it.
- Clear every box to take the NF out of the hierarchy again.

Placement is stored in MiniSMO only and is never written to the NF; it is recorded in the audit log. The Demo Network topology is fixed. NFs of other types (GENERIC NETCONF NF) are listed under **OTHER NFs**.

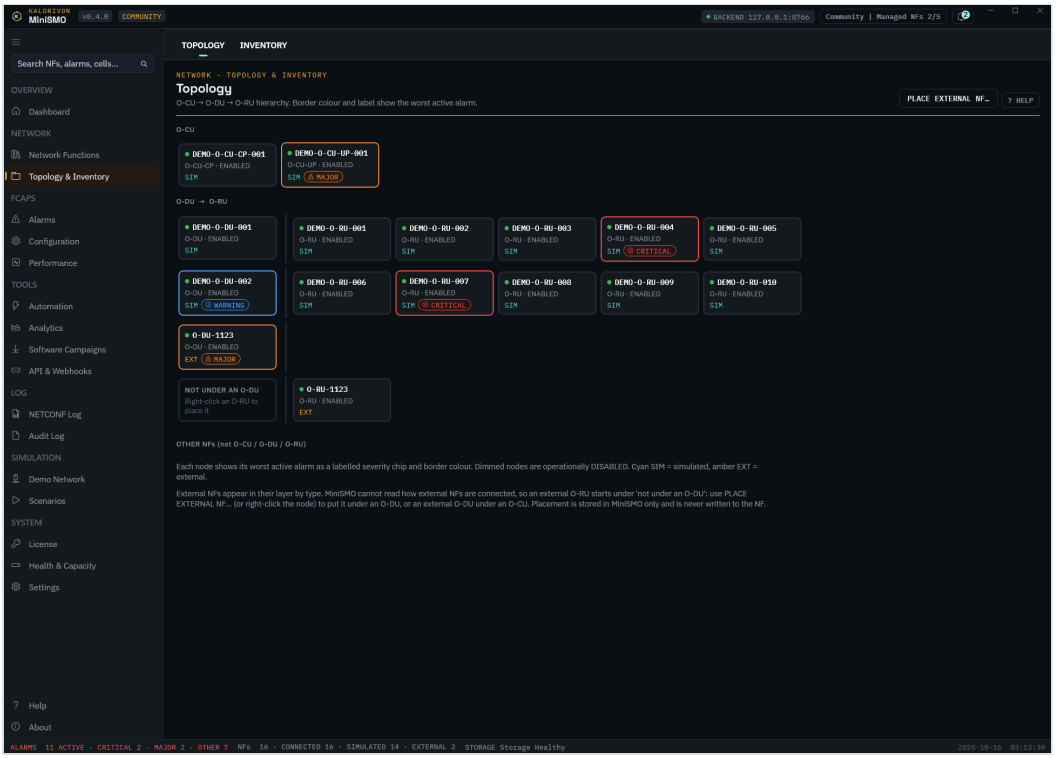
The **Inventory** tab next to *Topology* shows the same NFs as a tree by region and site, with their cells. See Inventory and Network Tree. Software versions are on *TOOLS* > *Software Campaigns*.

Each node shows:

Item	How it is shown
Name and type	Text on the node
Mode	**SIM** (cyan) or **EXT** (amber) badge
Connection	Green dot for CONNECTED, grey for DISCONNECTED
Operational state	Node dimmed when DISABLED
Alarm state	Labelled severity chip and border colour for the worst active alarm: red CRITICAL, orange MAJOR, yellow MINOR, blue WARNING

Select a node to open that NF in Network Functions. Right-click a node for **Open in Network Functions** and **Place in topology...**. The view refreshes every 5 seconds, so you can watch scenarios play out.

See Understanding the Demo Topology for how the Demo Network is wired.



Topology: the Demo Network in three layers

37. Inventory, Search and Notifications

37.1 Inventory and Network Tree

NETWORK > *Topology & Inventory* > *Inventory* shows the network as a tree: **Region** → **Site** → **NF hierarchy** → **Cells**. Parent and child NFs come from the topology (for example O-DU → O-RU), and cells from each NF's running configuration.

Region and site are inventory fields you set per NF: select an NF in the tree and choose **SET SITE / REGION...**, or use **Site / region** in the *Network Functions* toolbar. They are stored in MiniSMO only and never written to the NF. NFs without a region or site are grouped under *(no region)* and *(no site)*. An NF without its own site and region is shown under its topology parent (an O-RU under its O-DU). An NF whose site or region differs from its parent's is shown at its own site, and its node names the topology parent. An NF with the same site and region as its parent stays under the parent.

Each NF node shows its type, connection state and active alarms with the worst severity. Select a node to see its details. NF nodes have **OPEN NF**, **SET SITE / REGION...** and **INVESTIGATE**; cells have **INVESTIGATE CELL**. See Investigate. A cell is named by its own RDN (`NRCellDU=1`); its full DN (`ManagedElement=1,GNBDUFunction=1,NRCellDU=1`) is shown in the cell's details.

EXPORT CSV... asks for confirmation, then writes the inventory (name, type, address, vendor, model, serial number, software and hardware version, site, region, states, number of cells) to the exports folder and shows **Open file** / **Show in folder**. Software versions across the NFs are on Software Campaigns.

37.2 NF Identity and Cells

MiniSMO reads each external NF's vendor, software version, NF type, model, serial number and hardware version with read-only `<get>` requests. It does this automatically when the NF connects, when it sends a VES `pnfRegistration`, and for any NF not read yet; **Discover identity** in the *Network Functions* toolbar reads it again on request.

Sources, in order. A later source only fills fields the earlier ones left empty:

Source	Fields
3GPP `ManagedElement/attributes` (TS 28.622), read with `<get>` and an explicit subtree filter; the standard O1 location	`vendorName`, `swVersion`, `managedElementTypeList` (NF type), `userLabel`, `locationName`, `managedBy`
`o-ran-software-management`, if the NF advertises it	build version and vendor code of the active, running software slot
`ietf-hardware` (RFC 8348), if the NF advertises it	manufacturer, model, serial number, hardware and software revision

Source	Fields
`openconfig-platform`, if the NF advertises it (routers and switches that use OpenConfig, for example Cisco IOS XE)	from the top-level CHASSIS component: manufacturer (`mfg-name`), model (`part-no` , the vendor's part number), serial number, hardware and software version; the software version of an OPERATING_SYSTEM component when the chassis reports none. Leaves a device fills with a placeholder such as `NULL` count as not reported, and a multi-line version banner is shown by its first line
VES `pnfRegistration` (fallback)	vendor, model, serial number, software version, firmware version
VES event header (fallback)	`nfVendorName` (vendor), `nfNamingCode` (NF type code)

The `ManagedElement` leaves are operational (`config false`), so they are only in `<get>` replies, never in `<get-config>` , and MiniSMO never tries to write them. Some devices, including the O-RAN-SC NTS simulators, do not fill them in; the VES values are then used. `locationName` becomes the NF's site if you have not set one.

The dialog shows which source each value came from. A field is shown as unknown only when neither NETCONF nor VES reports it; nothing is guessed. Cells (NRCellDU, NRCellCU, EUtranCell) are listed from the latest configuration snapshot with their DN, PCI and ARFCN. Their state (3GPP TS 28.541: `administrativeState` , which is configuration, and the read-only `operationalState` and `cellState`) is then read with `<get>` , selecting exactly those cells by their keys; with `report-all` when the NF announces it (RFC 6243), so a default administrative state the running configuration leaves out is returned too. Cell state is read with the NF's identity: on connect, with RE-VERIFY and after a cell configuration change. In **Inventory**, a cell shows *locked / disabled* (administrative / operational) in the tree and all three states in its details; a state the NF does not report is shown as *not reported by the NF*.

Capabilities in the *Network Functions* toolbar summarises, vendor-neutral, what the NF can technically do (the **Capabilities** tab of the NF lists the raw NETCONF capabilities instead): configuration (writable datastore), candidate datastore, confirmed commit, rollback, validate, notifications, YANG-Push, FM, PM, software management, cell discovery and hardware inventory. It is derived from the NF's `<hello>` , YANG library and stream list.

37.3 Search

The search box at the top of the navigation pane finds NFs by name, address, vendor, model, serial number, software version, site or region; alarms by title, alarm ID, managed object or text; and cells by ID, DN or PCI. Select a result to open the NF or the alarm console.

37.4 Notifications

The **bell** at the top right of the window lists local notifications: automation alerts, approvals waiting for a decision, configuration drift, failed jobs, paused webhooks, monitoring method changes and fallbacks, storage state and license changes. A badge on the bell shows unread notifications; opening the list marks them read. Notifications stay on this PC. To send them elsewhere, use webhooks.

38. YANG Model Registry

SYSTEM > *Settings* > *YANG Models* is a local library of YANG modules. Use it to keep the models your devices implement next to the devices themselves, so you can read the schema while you look at `<get>` and `<get-config>` output.

The list shows, for each imported file:

Column	Meaning
Module	Module (or submodule) name
Revision	Latest `revision` date in the file
Namespace	XML namespace of the module
Prefix	Module prefix
Kind	module or submodule
Size / SHA-256	File size and hash, to check you have the exact file you expect

Select a model to read its source. **DELETE** removes it from the registry and from disk after confirmation.

Files are stored in `%LOCALAPPDATA%\Kaldrivon\MiniSMO\yang` as `<module>@<revision>.yang`.

MiniSMO does not ship third-party YANG models. Standards bodies and vendors publish YANG modules under their own terms; download them from the source you are entitled to use. The NF's Capabilities tab tells you exactly which modules and revisions a device implements.

MiniSMO does not generate screens from these models. When you edit an external NF's configuration, it uses the NF's own modules (read with `get-schema`) or the models imported here for names, types and constraints. See Changing External Configuration and Known Limitations.

39. Importing a YANG Model

1. Open *SYSTEM* > *Settings* > *YANG Models*.
2. Select **IMPORT** and choose one or more `.yang` files.
3. MiniSMO checks each file and adds it to the list.

Checks performed on import:

- The file name ends in `.yang`, is UTF-8 text and is at most 5 MB.
- It contains a `module` or `submodule` statement with a valid name.
- Braces are balanced (comments and quoted strings are ignored), which catches truncated files.
- A `module` has a `namespace` statement.
- The same module and revision is not already imported.

These are sanity checks, not full YANG compilation. A file that passes can still have errors that a YANG compiler such as `pyang` would report.

Common import errors:

Message	Fix
only .yang files can be imported	Rename or pick the right file
unbalanced braces	The file is truncated or corrupted; download it again
module has no namespace statement	Not a valid YANG module
... is already imported	Delete the existing entry first if you want to replace it

40. Editions and Capacity

40.1 Editions and Capacity

All features included. Free for up to 5 external NFs. A paid license is required for additional external NFs.

Kaldrivon Mini SMO is one product with one installation, one database and one set of NFs. Licensing limits only how many external NFs MiniSMO manages:

	Community	Small Deployment	Unlimited
License	none, free	commercial license	commercial license
External NFs	up to 5	up to the number in the license (25)	no numerical limit
Features	all	all	all
Built-in simulated NFs (Demo Network)	never counted	never counted	never counted

Supported is a support entitlement, not a license: it does not change the capacity (5 without a license) and is not installed in MiniSMO.

Every feature works the same way in every edition: onboarding, NETCONF and VES, NETCONF Call Home, capability and YANG discovery, inventory and topology, fault, performance and configuration management, backups and comparison (also across NFs and with templates), templates, desired state and drift, rollback, bulk operations, schedules, automation and approvals, analytics (KPI rankings, biggest changes, alarm correlation, flapping detection and suppression), Investigate, diagnostic collection, operations and scheduled reports, software campaign planning, the northbound REST API, O-RAN R1 for rApps, webhooks, the audit log with search and export, and every retention setting. There are no daily quotas, item limits or feature notes tied to the edition.

MiniSMO is an early release for lab, test and demo work. A license changes the external NF capacity only; it does not add a support or service-level commitment.

40.2 What a License Changes

Only the external NF capacity (see External NF Capacity):

- **Installing a license** raises the capacity at once. Nothing else changes and nothing needs to be re-onboarded.
- **Removing a license, or a license that expires or stops being valid,** returns to 5 external NFs. Nothing is deleted and every feature keeps working. The 5 oldest integrations stay managed; the newer ones are hidden (not connected, monitored or reported) until a license is installed again, brought back into a free slot, or de-integrated in Settings. See Unlicensed (hidden) NFs.

Earlier versions (up to 0.4.0) had a daily *Community allowance* of some features. It was removed: rules, schedules, webhooks, tokens, templates and desired states that were paused beyond that allowance run again after the upgrade, and the daily usage counters are deleted from the database.

Licensing and NF capability are separate. The license never decides whether a feature can be used. The NF decides whether it can technically do something: rollback, for example, needs an NF with a writable datastore; otherwise MiniSMO says *not supported by this NF* and why.

Operational safeguards are not licensing. They protect MiniSMO and the NFs in every edition: the northbound API accepts at most 20 requests a second per token (bursts up to 40) and blocks an address for 60 s after 10 failed authentications in a minute (`429 RATE_LIMITED` with `Retry-After`); webhooks keep at most 500 pending deliveries and pause after repeated failures; automation rules have a cooldown and an hourly limit; the VES collector stores events of at most 4 requests at a time; a single NF keeps at most a fixed number of PM series; list and export requests are bounded.

41. Configuration Operations

41.1 Configuration Backups

FCAPS > *Configuration* > *Configuration Operations* > *Backups*. **BACK UP NOW...** reads the NF's running configuration with a read-only `get-config` and stores it with a timestamp, the NF, its software version, your comment and who took it. Secret values are masked before storage, the same way as in configuration snapshots.

Backups are never overwritten: each one is a new entry. MiniSMO keeps the 50 newest backups per NF; you can protect individual backups (**KEEP**) and change how many are kept in *SYSTEM* > *Settings* > *General*, section **Operations**.

Backups taken automatically are marked by trigger: *PRE_CHANGE* before a template, approval or automation change, *PRE_ROLLBACK* before a rollback, *SCHEDULED* by a schedule, *BULK* by a bulk backup.

41.2 Comparing Configurations

Select a backup and choose **COMPARE WITH CURRENT** (reads the NF now) or **COMPARE WITH...** another backup. The result lists changed parameters with both values, and parameters found only on one side.

Comparison
Backup vs the NF's current configuration
Backup vs another backup of the same NF
Backup vs another NF, or ignoring list-key values (structure across NFs)
Backup or NF vs a template

Masked secret values (`******`) are compared as masked.

41.3 Configuration Templates

Configuration Operations > *Templates* holds named sets of parameter values, for example a cell template:

```
NRCellDU[id=*]/attributes/ssbPeriodicity = 20
NRCellDU[id=*]/attributes/arfcnDL = 632628
```

Paths use the parameter IDs shown on the Configuration page. `*` matches any list-key value or one path segment; `**` matches several segments. A template can be limited to an NF type, a vendor and required YANG modules; MiniSMO refuses to apply it to an NF that does not match (*INCOMPATIBLE*).

PREVIEW ON NF... shows the changes for one NF without writing anything. **APPLY TO NFs...** is a bulk operation.

41.4 Desired State and Drift

A desired state is a set of expected values (from a template or entered directly) for selected NFs. MiniSMO compares it with each NF's latest configuration snapshot every 15 minutes and on **EVALUATE NOW**:

State	Meaning
COMPLIANT	Every expected value matches
NON-COMPLIANT	At least one value differs
UNKNOWN	No snapshot yet, the parameter is not present, or the value is masked
NOT EVALUATED	Not checked yet

When an NF becomes non-compliant, MiniSMO raises a notification and a `DRIFT_DETECTED` event (for webhooks and automation). Nothing is corrected automatically. **PROPOSE REMEDIATION...** creates an approval with the changes that would restore the desired values.

41.5 Rollback

Select a backup and choose **ROLLBACK...**. MiniSMO reads the NF's editable parameters and lists every value that differs from the backup. Confirming runs:

1. **Validate**: the change is previewed against the NF's current values and YANG constraints.
2. **Backup**: the current configuration is saved (*PRE_ROLLBACK*).
3. **Apply**: candidate + validate + confirmed commit, or writable-running with test-then-set, as for any configuration change.
4. **Verify**: the values are read back before the commit is confirmed.

Rollback is available only when the NF has a writable datastore and configuration changes are enabled for it. With confirmed commit the NF reverts on its own if the change is not confirmed; without it, MiniSMO says so before you confirm. Rollback restores leaf values of entries that exist now; list entries added or removed since the backup are listed but not recreated or deleted. A rollback is never guaranteed to restore every value.

41.6 Bulk Operations

Bulk operations run one action on several NFs: configuration backup, read-only health check, or applying a template. Each shows the target list first; template pushes also show every change per NF and need confirmation. The job runs each NF separately, so one NF failing does not stop the others, and shows a per-NF result. Jobs are listed on the *Jobs* tab.

42. Automation

42.1 Automation

TOOLS > *Automation* runs rules of the form **trigger** → **condition** → **action**.

Triggers	Conditions	Actions
Alarm raised or cleared	NFs, NF types	Local notification (routable by webhooks)
NF connected or disconnected	Minimum severity	Send the event to a webhook
Configuration changed	Alarm text contains	Health check (read-only)
Drift detected		Configuration backup
KPI threshold held for N minutes		Collect diagnostics
Schedule (interval or daily)		Health report
		Propose a configuration change (approval)

Example: *PRB utilisation DL > 90 for 30 minutes → notify, send webhook, run a health check*. A KPI rule fires once when the condition has held for the whole period and again only after it has cleared.

TEST (DRY RUN) describes what each action would do without doing it. Every run is recorded in *Run history* with its trigger, per-action results and failure reasons, and in the audit log. Storm limits: a cooldown per rule (default 15 minutes) and at most 30 runs per rule per hour.

42.2 Approvals

Actions that would change an NF never run on their own. They create an approval: **detect** → **analyse** → **recommend** → **a person approves** → **execute** → **verify** → **record**. The *Approvals* tab shows the target NF, each parameter with its current and proposed value, and who proposed it. **APPROVE AND APPLY** takes a backup, lets the NF validate the change, applies it with confirmed commit or test-then-set, reads it back, and records the result. **REJECT** records the decision; nothing is sent.

A rule can apply changes without approval only if two things are true: the rule's action has *Apply without approval* ticked, and an administrator turned on *Allow automation to apply changes without approval* in *SYSTEM* > *Settings* > *General*, section **Operations**. Both are off by default. Pending approvals expire after 24 hours (configurable).

42.3 Schedules

Schedules run configuration backups, health reports, operations reports and drift checks daily, weekly or every N minutes (at least 15), in this PC's local time. A schedule that was due while MiniSMO was closed runs once at the next start. Schedules are created from the Backups tab, the Reports tab or *Automation* > *Schedules*, where they are listed with their next run and last result. **RUN NOW** asks for confirmation and runs a schedule once; jobs that change the network still create an approval first.

43. Analytics and Troubleshooting

43.1 Analytics

TOOLS > *Analytics* refreshes every 30 seconds. **Window (minutes)** sets the KPI window: 15, 60, 240 or 1440.

- **KPI rankings**: per KPI family (availability, PRB utilisation, throughput, connected users, RRC and handover success, drop rate, latency, packet loss, CPU), the NFs and resources from the worst end, over a chosen window. Families are recognised from the metric names the NFs report (3GPP TS 28.552, O-RAN and simulator names). A family without data shows *no data reported*; no KPI is derived or invented. Counters are ranked by rate.
- **Most alarms** in the last 24 hours.
- **Biggest changes**: the last hour against the 23 hours before it.
- **Alarm correlation**: active alarms raised close together (default 120 s) on topologically connected NFs or on the same managed-object subtree. Each group names a *potential parent alarm* and *likely related alarms*. This is a heuristic, not a root-cause claim.
- **Alarms counted**: simulated, external and NF supervision alarms (Loss of supervision, VES heartbeat missed). Health alarms raised on MiniSMO itself are left out.
- **Flapping**: alarms raised repeatedly (default 5 or more per hour). With *Do not notify or route flapping alarms* on, those raises produce no events for webhooks and automation; the alarms themselves are still shown.

43.2 Investigate

INVESTIGATE is available for an alarm (Alarm Console), an NF (**Investigate** in the *Network Functions* toolbar) and an NF or cell in the network tree (**INVESTIGATE**, **INVESTIGATE CELL**). It collects what MiniSMO has stored about the target and lays it out as:

- **Observed data**: connection state and last error, active alarms and the last 24 hours of alarm history, configuration changes, monitoring gaps, actions taken from MiniSMO, KPI trends, alarms on connected NFs
- **Correlation**: facts that happened close together, for example a configuration change in the 30 minutes before an alarm
- **Possible explanation**: hedged readings ("may", "is possible"), never a diagnosis
- **Recommended check**: what to look at next

Data MiniSMO does not have (for example NF-internal logs) is not shown or guessed.

43.3 Diagnostic Collection

Diagnostics in the *Network Functions* toolbar asks for confirmation, then writes one ZIP per NF to the exports folder and shows **Open file** / **Show in folder**. The ZIP contains: inventory entry (without credentials), capabilities and capability model, monitoring

status, alarms and FM events, the latest configuration snapshot, configuration changes, monitoring gaps and the last 200 NETCONF messages. Configuration and NETCONF payloads are the redacted copies MiniSMO stores. It is read-only.

44. Reports and System Health

SYSTEM > *Health & Capacity* > *Reports*:

- **HEALTH REPORT**: asks for confirmation, then creates the report and opens it. Contents: NFs connected, sites, cells, active alarms by severity, alarms raised and cleared in 24 hours, configuration changes, and a list of **Attention** and **Investigate** items (disconnected NFs, critical and major alarms, locked cells, non-compliant NFs, failed jobs, storage state), plus a table of all NFs. HTML and JSON.
- **OPERATIONS REPORT...**: for a period, per NF alarms raised and cleared, critical alarms, average time to clear, configuration changes and backups, a KPI summary and automation activity. HTML, JSON and CSV.
- **SCHEDULE...**: daily or weekly reports.

Reports are saved to `%LOCALAPPDATA%\Kaldrivon\MiniSMO\exports\reports` and never overwrite each other. Simulated NFs are marked.

The **System health** card shows **HEALTHY**, **DEGRADED** or **CRITICAL** from storage, CPU, memory, the notification queue, NETCONF sessions, the event bus, failed jobs and failed webhook deliveries in the last 24 hours, and the alarm rate. The Dashboard shows the same state next to *What needs attention*, and lists each check that is not healthy there (for example *Memory: 91 % used*). The matching MiniSMO alarm (for example *High memory usage*) is raised only after two readings in a row, one minute apart, so a short peak does not create an alarm; the state on the Dashboard follows the current reading.

45. Integrations

45.1 Northbound REST API

TOOLS > *API & Webhooks* > *API access*. A separate, token-authenticated listener for OSS, ticketing systems and scripts. MiniSMO's own local API is not exposed.

1. Turn on the listener in *SYSTEM* > *Settings* > *General*, section **Operations** (address, port, HTTPS with the collector's self-signed certificate). The default is `127.0.0.1:8770`, this PC only.
2. **NEW TOKEN...**: a name and a scope. The token is shown once; only its hash is stored.
3. Call the API with `Authorization: Bearer <token>`. The OpenAPI description is at `/api/v1/openapi.json`, interactive docs at `/api/v1/docs`.

Endpoint	Scope
`GET /api/v1/nfs`, `/nfs/{id}`, `/inventory`, `/sites`, `/cells`, `/topology`	READ
`GET /api/v1/alarms`, `/kpis`, `/config/{nf}`, `/config/{nf}/backups`, `/health`	READ
`POST /api/v1/alarms/{id}/acknowledge` (local acknowledgement, never sent to the NF)	OPERATE
`POST /api/v1/nfs/{id}/backup`, `/nfs/{id}/health-check`	OPERATE
`POST /api/v1/automation/rules/{id}/run` (network changes still become approvals)	OPERATE

Errors are JSON `{"error": {"code", "message"}}` with a matching HTTP status: 401 for a missing or revoked token, 403 for scope, 429 `RATE_LIMITED` (with `Retry-After`) for more than 20 requests a second per token (bursts up to 40) or after 10 failed authentications from one address within a minute (that address is refused for 60 s). These are security limits, the same in every edition; there is no daily request quota. API actions are recorded in the audit log as `api:<token name>`.

45.2 O-RAN R1 Interface

The northbound listener also serves the O-RAN R1 interface, so rApps can use MiniSMO as their SMO / Non-RT RIC framework for discovering services and for data. It uses the same address, port, HTTPS setting and API tokens; a READ token is enough, because registering an rApp, publishing an API or creating a data job never changes the network. *TOOLS* > *API & Webhooks* > *API access* shows how many rApps are onboarded, how many data jobs and published APIs exist. The endpoints are listed in the OpenAPI description (`/api/v1/openapi.json`, sections *R1 SME (CAPIF)* and *R1 DME*). Errors are RFC 7807 problem details (`application/problem+json`).

Service Management and Exposure (SME)

SME follows the CAPIF core functions of 3GPP TS 29.222, with the standard request and response bodies:

Function	Endpoint
Onboard an rApp (API invoker)	`POST /api-invoker-management/v1/onboardedInvokers` (needs `onboardingInformation.apiInvokerPublicKey` and `notificationDestination`); `PUT` / `DELETE …/{onboardingId}`
Register an API provider	`POST /api-provider-management/v1/registrations` (needs `regSec`; each function in `apiProvFuncs` has a role AEF, APF or AMF and `regInfo.apiProvPubKey`); `PUT` / `DELETE …/{registrationId}`
Publish a service API	`POST /published-apis/v1/{apfId}/service-apis` (a ServiceAPIDescription with `apiName` and `aefProfiles`); `GET` / `PUT` / `DELETE …/{serviceApiId}`
Discover service APIs	`GET /service-apis/v1/allServiceAPIs?api-invoker-id=…` with the optional filters `api-name`, `api-version`, `comm-type`, `protocol`, `aef-id`, `data-format`, `api-cat`

MiniSMO assigns the identifiers (`apiInvokerId`, `apiProvDomId`, `apiProvFuncId`, `apiId`) and returns a `Location` header. A public key or `regSec` that is already registered, or an API name an APF already published, is refused with 403. Publishing needs a registered APF, and every `aefId` must be a registered AEF. Removing a provider also removes the APIs its APFs published.

MiniSMO publishes its own APIs, so an rApp discovers them right after onboarding (they are also in the `apiList` of the onboarding answer): **data-consumer** (the DME below) and **minismo-northbound** (the REST API), both on AEF `AEF_MiniSMO` at the address the rApp used.

Not provided: CAPIF security (TS 29.222 CAPIF_Security_API, OAuth tokens per API) and CAPIF events. Access control is the API token.

Data Management and Exposure (DME)

DME follows the O-RAN-SC Information Coordination Service data consumer API (`/data-consumer/v1`, snake_case bodies). MiniSMO is the producer of four information types:

Information type	Data	Job definition (all optional)
`fm-alarm-events`	Alarms raised and cleared, as MiniSMO stores them (simulated and external NFs)	`nf_ids`, `severities` (CRITICAL, MAJOR, MINOR, WARNING, INDETERMINATE), `include_cleared` (default true)
`cm-change-events`	Configuration changes: NETCONF change notifications and changes made from MiniSMO (changed by, datastore, paths)	`nf_ids`

Information type	Data	Job definition (all optional)
`pm-measurements`	PM samples as collected: NF, metric, resource, time, value (counters as a rate per second over their interval)	`nf_ids`, `metrics` (names or wildcard patterns), `period_s` (60–86400, default 900)
`nf-topology`	A snapshot of the NFs and links when the job is created, then each NF connection change	`nf_ids`

`GET /data-consumer/v1/info-types` lists them and `GET …/info-types/{infoTypeId}` returns the JSON Schema of the job definition (`job_data_schema`). Create or change a job with `PUT /data-consumer/v1/info-jobs/{infoJobId}`:

```
{"info_type_id": "fm-alarm-events", "job_owner": "my-rapp",
 "job_result_uri": "https://rapp.example:8443/alarms",
 "job_definition": {"severities": ["CRITICAL", "MAJOR"]}}
```

The answer is 201 for a new job and 200 for a change; a job definition that does not match the schema gives 400, an unknown information type 404, and changing the type of an existing job 409. MiniSMO sends the data as an HTTP `POST` of a JSON array to `job_result_uri`: events as they happen, PM once per `period_s` with the samples collected since the previous delivery (at most 10,000 per delivery; no empty deliveries). `GET …/info-jobs` lists jobs (filters `infoTypeId`, `owner`), `GET …/info-jobs/{infoJobId}/status` returns `ENABLED` with producer `MiniSMO`, and `DELETE` removes a job (or all jobs of an owner).

`PUT /data-consumer/v1/info-type-subscription/{subscriptionId}` with `owner` and `status_result_uri` subscribes to information type availability: MiniSMO immediately posts each type with its schema and status `REGISTERED`.

With the listener turned off nothing is delivered and jobs report `DISABLED`; they resume when the listener is back. Failed deliveries are counted per job and not retried. Simulated NFs are marked as such in the data (`simulated` in the topology), and only data MiniSMO really has is sent.

45.3 Webhooks

TOOLS > *API & Webhooks* > *Webhooks* sends events as JSON `POST`s:

Setting	Meaning
URL	`https://` recommended
Events	Any of ALARM_RAISED, ALARM_CLEARED, NF_CONNECTED, NF_DISCONNECTED, CONFIG_CHANGED, DRIFT_DETECTED, DRIFT_RESOLVED, JOB_FINISHED, AUTOMATION_ALERT, KPI_THRESHOLD (none = all)

Setting	Meaning
Minimum severity	Alarm events at or above this severity
Secret	Optional. The body is signed with HMAC-SHA256: header `X-Kaldrivon-Signature: sha256=<hex>`. Stored protected, never shown again

SEND TEST asks for confirmation, posts a test event and shows the result. Failed deliveries are retried after 30 s, 2 min, 10 min, 30 min and 1 h, then marked FAILED. To prevent storms, a webhook keeps at most 500 pending deliveries, at most 20 are sent per 5-second cycle, and a webhook pauses after 20 failures in a row (with a notification); enable it again after fixing the endpoint. Delivery status and the last error are listed below the webhooks.

46. Software Campaigns and Audit

46.1 Software Upgrade Campaigns

TOOLS > *Software Campaigns* lists, under **SOFTWARE VERSIONS**, the versions the NFs report per NF type and vendor, with the count and the NFs on each version. **FINDINGS** highlights:

- **Outliers**: a version used by a small share of NFs of the same type and vendor
- **Unknown versions**: NFs that report no software version
- **Mixed versions within a site**

Versions are what the NFs report. MiniSMO does not judge vendor lifecycle status.

PLAN CAMPAIGN... plans a campaign: a name, a target version and the target NFs. For each NF MiniSMO records the current version and a readiness state: *READY* (the NF exposes `o-ran-software-management`), *UNSUPPORTED_BY_NF*, *ALREADY_AT_VERSION* or *UNKNOWN_VERSION*. MiniSMO does not download, install or activate software in this release; use the plan with the vendor's procedure.

46.2 Audit Log

LOG > *Audit Log* records who did what, when, to which target and with which result: NFs added, changed and removed, connections, monitoring, configuration changes and permissions, backups and rollbacks, alarm acknowledgements and notes, template and bulk operations, approvals, automation runs, schedules, webhooks and API tokens, settings and license changes. Actions from automation appear as `automation:<rule>`, from the API as `api:<token>`, and from the application as the Windows user.

Request bodies are never stored, so passwords, keys and tokens cannot reach the log. It keeps a year of audit history by default (configurable), searches by text, action, source, result and time, and exports CSV (**EXPORT CSV** asks for confirmation and shows **Open file** / **Show in folder**).

Tamper evidence. Every entry carries a keyed hash over its own content and the previous entry's hash (HMAC-SHA256 with a random key stored protected for your Windows user). **VERIFY LOG** walks the whole log and says either *intact* with the number of entries checked, or the first entry where the chain breaks: an entry changed, inserted or removed outside MiniSMO, for example by editing the database file. Retention and **PURGE DATA... > Operations** remove entries in a way the check expects. The CSV export has each entry's hash in the `entry_hash` column. This shows tampering; it does not prevent it, and someone who can read your protected keys could rebuild the chain. `GET /api/audit/verify` gives the same result.

MiniSMO is a single-user desktop application: there are no separate logins or roles in this release. Protect access to the PC and the Windows account.

47. Community Edition and Licensing

All features included. Free for up to 5 external NFs. A paid license is required for additional external NFs.

The editions differ only in how many external NFs MiniSMO manages; every feature is in every edition:

Edition	External NFs	How you get it
Kaldrivon Mini SMO Community	up to 5	free, no license file
Kaldrivon Mini SMO Small Deployment	up to the number in the license (25)	commercial license file (`.klicense`)
Kaldrivon Mini SMO Unlimited	no numerical limit	commercial license file

The edition shown follows the capacity of the installed, valid license. *Supported* is a support entitlement, not a license: it does not change the capacity (it stays at 5 without a license) and is not installed in MiniSMO. MiniSMO is an early release for lab, test and demo work.

What counts (one rule, applied in the same place for every way an NF is added: the Windows app, the local API and scripts):

- **One external NF = one stored NF record = one slot.** Connected or not, monitored or not: an external NF uses a slot from the moment it is saved. Disconnecting it, stopping its monitoring or a failed connection never frees the slot; **deleting** it does, immediately.
- **The same NF cannot be stored twice.** For an NF that MiniSMO connects to, the NF is its management endpoint: address plus port. Names are compared case-insensitively, IP addresses in their canonical form, and a DNS name is resolved when the NF is added or edited (and again in the background while MiniSMO runs), so a name and an IP address of the same device are one endpoint. For a Call Home NF it is the caller identity: SSH host key fingerprint, serial number or hardware ID. A second record with the same endpoint is refused ('*O-DU-1*' *already manages this NF*) and takes no slot; edit or delete the existing NF instead. A name that cannot be resolved is compared by name only.
- **Integration numbers.** Every external NF gets a number when it is integrated (1, 2, 3 …, never reused). When more NFs are stored than the capacity allows, the oldest integrations keep their slots; see Unlicensed (hidden) NFs.
- **Simulated NFs never count**: the Demo Network and the NFs of a capacity test, however many.

License files are cryptographically signed by Kaldrivon and validated locally on your PC every time MiniSMO starts. No Internet connection or activation server is involved. If a license file is edited in any way, it stops being valid and MiniSMO runs with Community capacity.

To see your status, open *SYSTEM* > *License* or look at the license indicator in the title bar.

Related: Installing a License, External NF Capacity.

48. Installing a License

1. Save the `.klicense` file you received from Kaldrivon somewhere on your PC, for example `Acme-Telecom-Kaldrivon-MiniSMO.klicense`.
2. Open *SYSTEM* > *License*.
3. Select **INSTALL LICENSE** and choose the file.
4. MiniSMO checks the signature, the product, the validity dates and (if the license is tied to one computer) the machine. If everything checks out, the page shows **License status: Active** and the new external NF capacity. Your NFs, alarms, PM history and configuration stay as they are; nothing needs to be reinstalled or re-onboarded.

The license is copied to `%LOCALAPPDATA%\Kaldrivon\MiniSMO\license\license.klicense`. You can delete the original file afterwards, but keep a copy somewhere safe.

Installing a new license replaces the previous one. A license that has already expired is refused at install time.

To go back to Community, select **REMOVE LICENSE** and confirm. The button is available only while a license file is installed (also an expired or invalid one); in Community it is disabled. Fields the license does not carry (Customer, License ID, License Type and so on) show **–**, and a license without an expiry date shows *No expiration*. Nothing is deleted: the 5 oldest external NFs stay managed and the others become hidden until a license is installed again (see Unlicensed (hidden) NFs).

VIEW DETAILS shows every field of the installed license: license ID, customer, edition, license type, valid-from and expiry dates, whether it is machine-bound, and the signing key ID. A feature list in older license files is shown for information only; it changes nothing.

> **Machine-bound licenses**
>
> Some licenses are tied to a single computer. If yours is, Kaldrivon will ask for your machine fingerprint before issuing it; **VIEW DETAILS** on the License page shows it, with a **COPY** button. A machine-bound license installed on a different PC shows *MACHINE_MISMATCH* and does not raise capacity.

49. Understanding External NF Limits

49.1 External NF Capacity and License Limits

The license page shows:

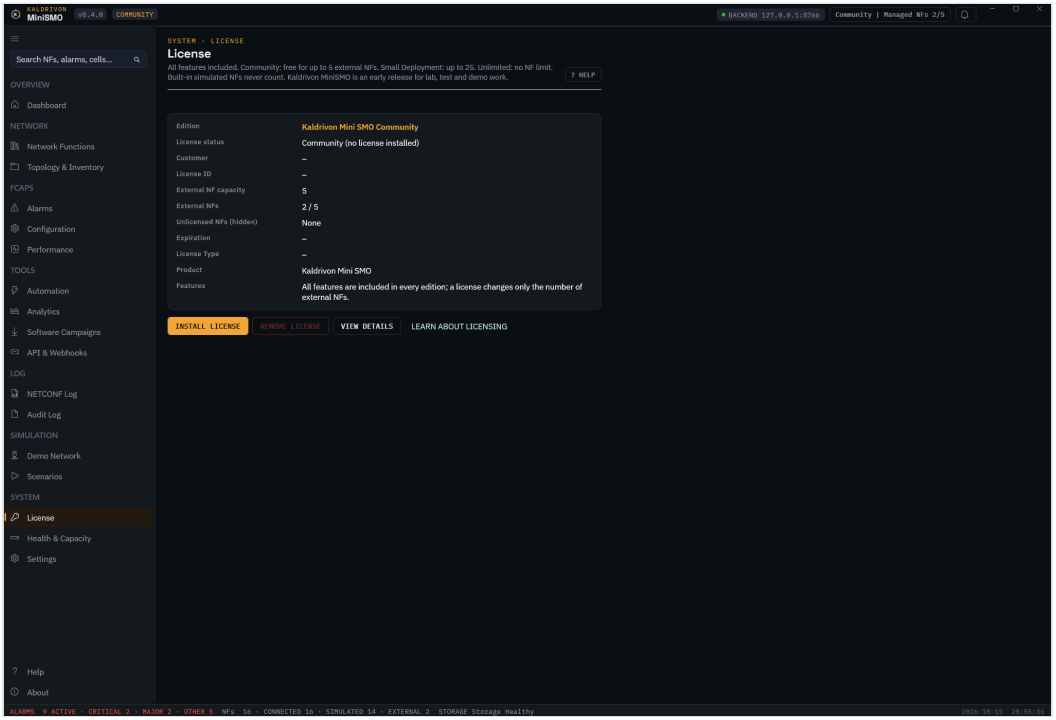
Field	Meaning
External NF capacity	How many external NFs your effective license allows (5 for Community, the licensed number for Small Deployment, or Unlimited)
External NFs	How many external NFs are managed right now, connected or not, for example *3 / 5*
Unlicensed NFs (hidden)	How many NFs beyond the capacity are hidden, or *None*

The title bar shows the same, for example *Community | Managed NFs 5/5 · 3 unlicensed*.

When usage reaches capacity (hidden NFs do not count), **ADD EXTERNAL NF** reports *Kaldrivon Mini SMO is free for up to 5 external NFs (5 in use); all features are included. A paid license is required for additional external NFs …*, with a license *Your license allows 25 external NFs (25 in use) …*, and nothing is created. Existing NFs keep working and can be viewed, edited and deleted. The check is done by the MiniSMO backend itself, atomically with saving the NF, so it also applies to scripts that call the local API and to two requests at the same moment. The northbound API and R1 cannot create NFs. A Call Home NF that dials in without being registered is never added automatically.

To add more external NFs, either delete ones you no longer need, or install a license with higher capacity.

Simulated NFs do not use capacity. You can create the Demo Network at any time.



License page: edition, capacity, external NFs and unlicensed NFs

49.2 Expired License, Downgrades and Unlicensed NFs

When more external NFs are stored than the capacity allows (a license expired, was removed or replaced by one with a lower capacity), MiniSMO keeps the **oldest integrations** (lowest integration numbers) up to the capacity. The newer ones become **unlicensed (hidden)**:

- they are not connected, not monitored and not reported: not in the NF lists, topology, dashboards, alarms, PM, reports, the northbound API or R1, and VES events from them are not applied;
- nothing about them is deleted: their settings, credentials, configuration backups and collected history are kept;
- the License page, the title bar and Settings show how many there are.

A license can expire while MiniSMO runs; the change is applied within a minute, with a notification.

When a license is installed (again), hidden NFs come back automatically, oldest integration first, up to the new capacity (all of them with Unlimited), and monitoring resumes. A notification lists them.

When a slot becomes free (you delete one of the managed NFs), **ADD EXTERNAL NF** first offers the hidden NFs: select one and **BRING BACK** to use the free slot for it, or **ADD A NEW NF** to add a different one.

To remove the hidden NFs for good, open *SYSTEM* > *Settings* > *Unlicensed NFs* and select **DE-INTEGRATE ALL HIDDEN NFS...**. After you confirm, each one is de-integrated with progress shown: MiniSMO first removes its own VES subscription from every NF it can reach, then deletes the NF with its stored data. Nothing else on the NFs is changed.

When a subscription or trial license expires

- The status changes to **LICENSE EXPIRED** and MiniSMO runs with Community capacity: 5 external NFs, every feature.
- The 5 oldest integrations stay managed; the others are hidden as described above.

A license that is not valid yet (its start date is in the future) shows **NOT_YET_VALID** and gives Community capacity until the start date.

50. System Capacity Assessment

50.1 Capacity & Storage Overview

SYSTEM > *Health & Capacity* > *Capacity & Storage* answers three separate questions. MiniSMO keeps them apart on purpose, because they limit different things:

Concept	Question it answers	Where it comes from
License capacity	How many external NFs am I entitled to configure?	Your license (Community: 5)
System capacity	How many NFs can this computer handle at acceptable performance?	The capacity test you run on this PC
Storage capacity	How much history can I keep before cleanup or disk limits are reached?	Retention settings and free disk space

Example: a 25-NF license on a PC whose capacity test suggests about 180 external NFs, with enough disk for 30 days of 5-minute PM history. None of these numbers changes the others. In particular, the capacity test never reduces your license.

The page has three tabs:

- **System Capacity**: computer information, current load, the latest capacity test and **RUN CAPACITY TEST**.
- **Storage**: space used by category, free disk, storage health, growth forecast, **CLEAN UP NOW**, **PURGE DATA...**, exports and backups.
- **History**: previous capacity tests, with view, delete and export.

Retention (how long each kind of history is kept, presets, and the retention impact estimator) is on *SYSTEM* > *Settings* > *Storage & Retention*.

The Dashboard's **LICENSE AND CAPACITY** card shows the Managed NFs, Technical capacity and Storage state. **HEALTH & CAPACITY** on that card opens this page.

All measurements stay on your computer. MiniSMO does not upload machine details, capacity results or storage data.

50.2 Running a Capacity Test

1. Open *SYSTEM* > *Health & Capacity* > *Capacity & Storage*, **System Capacity** tab.
2. Choose the profiles to test (all are selected by default) and the test length: **Quick** (5 s per stage), **Standard** (15 s) or **Thorough** (60 s).
3. Select **RUN CAPACITY TEST**. Progress shows the current profile, stage and live measurements.
4. Select **CANCEL** at any time. Partial results are kept and temporary data is removed.

Measured Operations Capacity (first profile) answers *how many external NFs can this computer manage*. In a temporary instance it drives MiniSMO's real processing, one pipeline after the other, as fast as this computer allows, and measures the CPU time per operation and the database bytes each one adds:

Pipeline	What runs
VES alarm events	collector intake, attribution, alarm raise/clear, alarm history, VES event log
VES heartbeats	collector intake, heartbeat state, VES event log
PM values (VES perf3gpp)	collector intake, 3GPP measurement parsing, PM storage
NETCONF alarm notifications	NETCONF log, o-ran-fm notification parsing, alarm raise/clear, alarm history
Configuration change notifications	NETCONF log, ietf-netconf-notifications parsing, configuration change history
NETCONF reads	`<get>` over real NETCONF/SSH sessions to a local test device; also the memory one session needs
API requests	the local REST API (dashboard, alarms, NFs, PM tiles)

The result is a number of external NFs for this **standard operation of one NF** (shown with every result):

Item	Per NF
Alarm events (VES fault raise/clear)	6 per hour
NETCONF alarm notifications	6 per hour
VES heartbeats	1 per minute
PM values (VES perf3gpp)	500 per 15-minute period
Configuration change notifications	6 per hour
NETCONF reads (`<get>`)	12 per hour
API requests (northbound / rApps)	0.6 per minute
NETCONF sessions	1 persistent session
Windows app on a live page	2 requests per second in total

The NF count is the smallest of three limits:

- **Processing**: 60% of one CPU core (the backend is one process, so one core is the honest ceiling for its work, whatever the number of cores), minus the Windows app's own load, divided by what one NF costs.
- **Memory**: available memory minus 1 GB kept free, divided by the memory one NF needs (its NETCONF session, measured, plus its monitor).
- **Disk**: free disk above the warning threshold (and the maximum data size, if set) divided by what one NF stores per day times your retention settings (alarm and VES history for the FM retention, PM for the 5-minute PM retention, heartbeats for the NETCONF log retention).

The card *Measured operations capacity* shows the NF count, which limit applies, the measured rates (for example *VES alarm events: 250 per second*) and what one NF needs.

The NETCONF session memory includes the local test device's side of each session, so it is an upper bound.

How the simulated profiles work:

- They ramp up step by step: 25, 50, 100, 250, 500, 750 and 1,000 NFs. It never starts at 1,000.
- Each profile runs against a **separate temporary MiniSMO instance** with its own database, created in the `temp` folder and deleted afterwards. Your NFs, data, alarms and license usage are not touched.
- Test NFs are simulated, so the test works in Community and never uses external NF license slots.
- Each stage is watched every second. The test **stops ramping** when conservative safety limits are reached: sustained CPU above 90%, low available RAM, API P95 latency above 1 second, slow database writes, PM scheduling falling behind, errors, or low disk space. It does not try to push your PC to its limit.

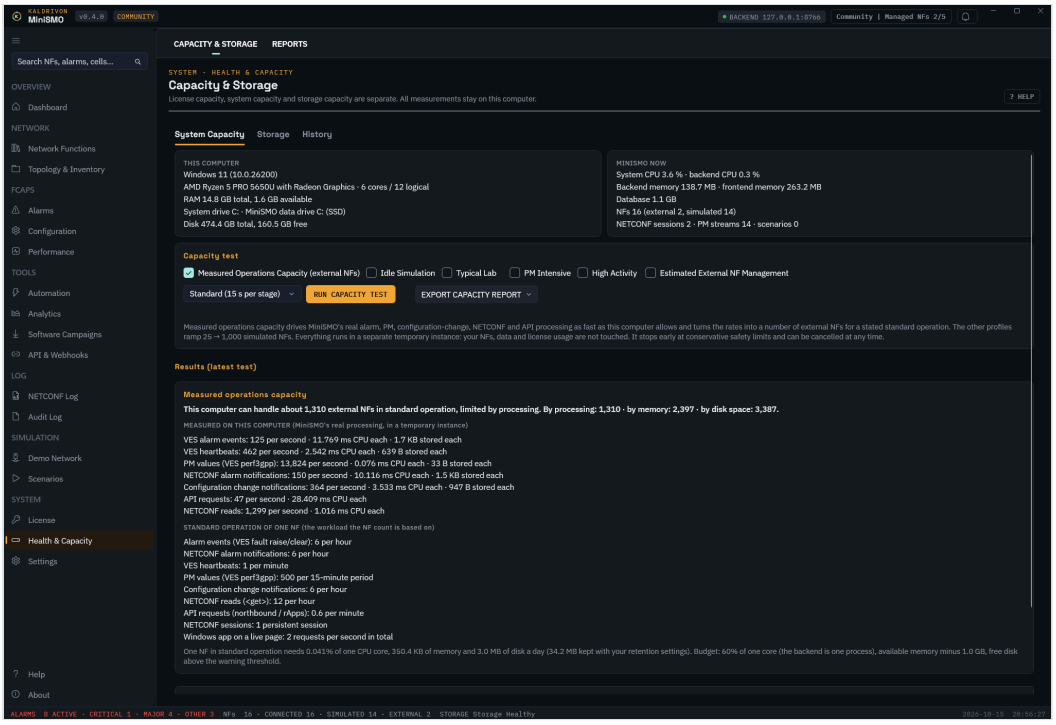
Workload profiles:

Profile	What it simulates
Idle Simulation	NFs exist and keep state; minimal background activity
Typical Lab	PM every 5 s, occasional alarms and configuration changes, dashboard and topology reads
PM Intensive	PM every second, frequent chart and history reads
High Activity	PM every second, many alarms, scenario activity, configuration changes and NETCONF logging
Estimated External NF Management	Real NETCONF/SSH sessions to a local test server, each polled regularly

The external profile is an **estimate**. The test server runs on the same PC, which makes the result conservative, and it does not represent any particular vendor's equipment.

What is measured at each stage: total, backend and frontend CPU; total, backend and frontend RAM; available RAM; API P50 and P95 latency; database write latency; PM and scenario scheduling delay; dropped PM updates; delayed events; event backlog; error rate; NETCONF operation latency (external profile); database growth rate; disk write rate; average PM row size; and NETCONF, FM and scenario log growth.

Only stage summaries are stored. Raw per-second telemetry is discarded after the test.



System Capacity: measured operations capacity after a test

50.3 Understanding Technical Capacity

For each profile the results show two numbers:

- **Maximum Tested**: the largest stage that completed without reaching a safety limit.
- **Recommended**: the number MiniSMO suggests for everyday use. It is never the maximum tested value.

The recommendation is the smallest of three dimensions, so you can see what limits you:

Dimension	How it is derived
CPU/RAM	Scaled from measured CPU at the maximum stage to a 60% CPU target, and limited by RAM headroom
API/Database	Scaled from measured API P95 and database write latency to 250 ms and 100 ms targets
Storage policy	How many NFs your free disk (above the warning threshold) can hold at steady state under the current retention settings

In addition, no recommendation exceeds 80% of the maximum tested value.

The results also show CPU, RAM, P95 latency and storage write rate at the recommended point, and the stage table shows every measurement.

Results depend on the MiniSMO version, this computer, what else is running, and your retention settings. Run the test again after hardware or settings changes. See Capacity vs Storage Constraints.

51. Storage Management

The **Storage** tab shows where MiniSMO's disk space goes:

Category	Contents
Database	The whole SQLite database, including its `-wal` and `-shm` files
PM Data	PM samples and aggregates of the Demo Network, PM read from external NFs (NETCONF, YANG-Push, VES measurements) and file-based PM (O-RAN/3GPP XML files)
FM History	Active and cleared alarms of all NFs and the alarm event history
NETCONF & VES Logs	Stored NETCONF messages, received VES events and recorded monitoring gaps
Configuration History	Configuration snapshots, change notifications, configuration backups of all NFs and the simulated configuration history
Operations & Audit	Audit log, notifications, automation runs, background jobs, webhook deliveries and approvals
Scenario & Capacity History	Scenario runs and capacity test summaries
Inventory & Other	What is left of the database: NFs, sites, capabilities, monitoring settings, templates, rules, schedules, settings, the YANG registry and the database's own overhead (indexes and free pages). Usually small; free pages are reused for new data
YANG Models	Imported `.yang` files
Exports	Files you exported
Application Logs	Backend and application log files
Backups	Database backups

Below the table: **Total data usage**, **Available disk space**, and the storage health state. PM, FM, logs, configuration, operations and scenario sizes are parts of the database total.

The tab updates by itself when files are added or deleted in the data folder, for example when you delete backups or exports in File Explorer: every few seconds MiniSMO compares a light fingerprint of the folders (names, sizes and times only) and measures storage again only when it changed. Nothing is measured on a timer otherwise.

Other actions: **OPEN EXPORT FOLDER**, **DELETE SELECTED EXPORT**, **CLEAN EXPORTS**, **CREATE BACKUP**, and **EXPORT STORAGE REPORT**.

52. Understanding MiniSMO Data Storage

All MiniSMO data lives under `%LOCALAPPDATA%\Kaldrivon\MiniSMO` (see Data Storage Locations). The largest part is usually PM history in the database.

PM history is kept in three tiers:

Tier	Resolution	Default retention
Raw	Every sample (1 to 30 s)	6 hours
5-minute aggregates	Min, max, average and sample count per counter per 5 minutes	7 days
Hourly aggregates	Same, per hour	30 days

Charts pick the tier automatically: raw data for recent periods, then 5-minute and hourly aggregates for longer ranges.

The database uses SQLite in WAL (write-ahead log) mode, which lets the backend write while pages read. The WAL file (`minismo.db-wal`) is checkpointed back into the database after cleanups and on clean shutdown, and it is included in the storage figures.

Freed database space is reused for new data. The database file shrinks gradually through incremental vacuum after cleanups; MiniSMO does not run a full VACUUM during normal use because that would block the database.

53. PM Data Retention

53.1 PM Retention

PM retention controls how long each PM tier is kept (defaults, from the MINIMAL preset: raw 6 hours, 5-minute aggregates 7 days, hourly aggregates 30 days). PM history older than the hourly retention is deleted unless you exported it or changed the retention.

Raw samples are always aggregated into the 5-minute tier **before** they are deleted, and 5-minute aggregates are rolled up into hourly aggregates before they expire. Longer-range charts keep working after raw data is gone; they just show 5-minute or hourly resolution with min/max bands.

Retention is applied automatically (see Automatic Cleanup) and when you select **CLEAN UP NOW**. PM inside its retention window is never deleted, even when disk space is low.

The three tiers apply to simulated PM. PM samples from external NFs (all sources, including measurement files) are stored as received and kept as long as the 5-minute aggregates (7 days by default). They are not aggregated.

53.2 PM Aggregation

For every NF and counter, each aggregate record stores:

Field	Meaning
bucket_start / bucket_end	The 5-minute or hourly period
min / max	Lowest and highest sample in the period
avg	Average of all raw samples in the period
sample_count	Number of raw samples
sum	Only for counters where a sum is meaningful. The simulated counters are rates, levels or percentages, so no sums are stored

Hourly averages are weighted by sample count, so they equal the average of all raw samples in the hour.

Aggregation runs every 5 minutes in the background. It is idempotent: re-running it for the same period updates the existing records instead of creating duplicates, so restarts and overlapping runs are safe.

53.3 Why PM Data Can Grow Quickly

PM volume multiplies quickly:

```
NFs × samples per hour × hours kept
```

For example, 250 NFs sampled every 5 seconds produce 180,000 raw rows per hour, or 4.3 million per day. At about 330 bytes per row that is roughly 1.4 GB per day of raw PM before aggregation.

Aggregates are much smaller: one row per counter per 5 minutes (288 per day) or per hour (24 per day), regardless of the sample rate.

That is why MiniSMO keeps raw data for a short period only and relies on aggregates for long-term history. The biggest levers are the **PM Update Interval**, the **raw retention**, and the number of NFs. Use the retention impact estimator to see the effect before you change anything.

54. FM History Retention

Alarm history is kept for 30 days by default.

- **Active alarms are never removed** because of age. They stay until they are cleared.
- **Cleared alarms** are deleted once their clear time is older than the FM retention.
- External monitoring history (alarm events, configuration change notifications, VES events and monitoring gaps) is also kept for the FM retention. Current external alarms and the latest configuration snapshots are kept.
- An alarm event that came from a VES event does not store a second copy of the event: the alarm history shows the raw event from the VES event log.
- VES heartbeats in the VES event log are protocol traffic: they are kept for the NETCONF log retention (7 days by default), not the FM retention.
- Raw VES events, alarm notifications and configuration snapshots are stored compressed and read back unchanged.

Change FM retention in *SYSTEM* > *Settings* > *Storage & Retention*.

55. NETCONF History Retention

NETCONF messages are kept for 7 days by default. Choose 7, 14, 30, 90 days or a custom value. You can also set a **maximum size** for NETCONF history (for example 500 MB, 1 GB, 5 GB, or no limit). When either the age or the size limit is exceeded, the oldest messages are removed first.

Messages are written to history in small batches (about every half second) instead of one database transaction each; the log is always complete when you open it.

Large replies are capped in history by the **maximum stored payload size** (default 1 MB per message). A larger message is stored with its metadata and the first part of the XML, and marked **PAYLOAD TRUNCATED FOR HISTORY**; the log shows its original size. This only affects the stored history. The reply shown on the Configuration page's NETCONF read tab after a GET or GET-CONFIG is always complete.

56. Configuration History

Every time a simulated NF's configuration is applied or reverted, MiniSMO records one history entry per changed field:

Field	Example
Timestamp	2026-09-26 10:42
NF	DEMO-O-RU-003
Revision	7
Field	pci
Old value / New value	110 → 222
Source	USER, SCENARIO or SYSTEM

Select **HISTORY** on the Configuration page to view it. The last 50 revisions per NF are kept by default; older revisions are removed by retention. The current configuration is never deleted by retention.

57. Automatic Cleanup

When **Automatic cleanup** is on (the default), MiniSMO applies retention in the background without blocking the application:

Job	When	What
Storage check	Every minute	Disk space and storage health, from file and folder sizes (cheap). The per-table breakdown on the Storage page reads the whole database, so it is measured when you open the page and kept for 10 minutes, and again after a cleanup
PM aggregation	Every 5 minutes	Builds 5-minute and hourly aggregates
Light cleanup	Hourly	Expired raw PM and NETCONF messages
Deep cleanup	Daily	All categories, then database maintenance: free pages are returned to Windows in short steps (up to 5 s per run), so the database file shrinks after a cleanup
Protective cleanup	When storage becomes CRITICAL	Removes data already eligible under your retention policy

Cleanup schedule options: *Hourly and daily* (default), *Daily only*, or *Manual* (only **CLEAN UP NOW**). Protective cleanup still runs when storage is critical, but it only removes data that is already past its retention.

Retention never deletes: the license, the NF inventory and stored credentials, current configuration, YANG models, exports (unless export cleanup is enabled), the running scenario, active alarms, or PM inside its retention window.

58. Manual Cleanup

CLEAN UP NOW on the Storage tab first shows what would be removed, for example:

Item	Estimate
Raw PM beyond retention	1.2 GB
Expired NETCONF messages	180 MB
Expired application log files	42 MB
Expired scenario run history	8 MB
Estimated reclaimed	**1.43 GB**

Select **CLEAN UP** to run it or **CANCEL** to close. Only data past its retention is removed. The estimate is approximate; SQLite reuses freed space inside the database first, and the file shrinks gradually.

PURGE DATA... on the Storage tab deletes *all* data of the groups you tick. Each group shows how many rows (or files) it would delete:

Group	Deletes
FM	Alarms and alarm event history of all NFs
PM	PM samples and aggregates, PM from external NFs, file-based PM records and the kept measurement file copies (`pm-files` folder)
CM	Configuration snapshots, change notifications and simulated configuration history
LOGS	NETCONF message log, VES event log and monitoring gaps
BACKUPS	Configuration backups of all NFs, including ones marked KEEP
OPS	Audit log, notifications, automation runs, background jobs, webhook deliveries and approvals
FILES	Files in the data folder: exports, temporary files and database backup copies

Type **CONFIRMED** to enable the button. NFs and their settings, monitoring and file-PM settings, topology, MiniSMO settings, the license, YANG models, templates, rules, schedules, webhooks and API tokens are kept. The purge is recorded in the audit log (after an **OPS** purge, that record is the first new entry). A file still open in another program is left in place. To remove everything, use Reset MiniSMO.

Exports are never removed automatically unless you turn on *Export cleanup*. Use **DELETE SELECTED EXPORT** or **CLEAN EXPORTS** to remove them yourself.

Backups: **CREATE BACKUP** makes a consistent copy of the database using SQLite's online backup method, safe while MiniSMO is running. A *Configuration* backup contains inventory, settings, the YANG registry, scenarios and configuration history, but leaves out PM, NETCONF messages and alarm history to keep it small. A *Full* backup contains everything. Automatic copies made before a database update (`minismo-pre-v<N>-`

`<time>.db`) are full copies of the database, so only the newest two are kept; older ones are removed when MiniSMO starts. Backups you create yourself are never removed automatically.

59. Disk Space Warnings

MiniSMO checks free space on the drive holding its data every minute.

State	Default condition	What happens
HEALTHY	Enough free space	Nothing
WARNING	Less than 10 GB or less than 10% free, whichever is more conservative	Notification and Dashboard warning; cleanup is recommended
CRITICAL	Less than 5 GB or less than 5% free	Protective cleanup of data eligible under retention runs automatically
CLEANUP RUNNING	A cleanup is in progress	Shown until it finishes

The Dashboard's **LICENSE AND CAPACITY** card shows the storage state: *Healthy*, *Warning*, *Critical* or *Cleanup running*. **HEALTH & CAPACITY** opens Capacity & Storage.

Measurement file copies kept (hours, default 24) is how long retrieved measurement files stay on this PC as files. The values from them are PM Data. The longest allowed is worked out from the free disk space and the size of the files that actually arrive (at most 30 days); see Measurement Files. They are shown as *Measurement Files (copies)* in the storage breakdown.

Optionally set a **Maximum MiniSMO managed data size** (10, 25, 50, 100 GB or unlimited). Approaching it gives a warning; reaching it triggers cleanup of eligible data.

If storage stays critical after cleanup, no more data is eligible for deletion: shorten retention, free disk space elsewhere, or remove exports and backups you no longer need. MiniSMO does not delete protected data to make room.

60. Capacity vs Storage Constraints

60.1 License Capacity vs Technical Capacity

The **License vs System** panel puts the numbers side by side:

Field	Example
Licensed External NF Capacity	25
Current External NFs	8
Estimated Technical External NF Capacity	~180
Typical Simulated NF Capacity	~350

A value that has not been measured yet shows **not tested**. Run the capacity test to fill it in. *Licensed External NF Capacity* shows **Unlimited** for an Unlimited license.

If your license allows more external NFs than the capacity test suggests for this PC, MiniSMO shows an information message: *Your license permits more external NFs than this computer's current benchmark suggests for high-activity workloads.* This does **not** invalidate or reduce your license. It means that at full license usage this PC may become slow, and a more powerful PC or lighter workload may be needed.

A license with lower capacity than the computer can handle is equally normal. The license is what you are entitled to; the capacity test is what this PC can do.

60.2 Capacity vs Storage Constraints

A computer can have plenty of CPU and memory but too little disk for the history you ask it to keep. The capacity test therefore reports the storage dimension separately and explains when it is the bottleneck. For example:

Dimension	NFs
CPU/RAM	500
API/Database	400
Storage policy	250
Overall recommended	**250**

With a note such as: *Storage-constrained: CPU/RAM supports ~500 NFs but current PM retention settings limit this to ~250 NFs with the available disk.*

To raise the storage-constrained number, shorten PM retention (see Changing Retention Settings), use a longer PM interval, free disk space, or run MiniSMO on a PC with more disk.

61. Estimating Storage Requirements

61.1 Storage Forecast

The **Forecast** panel on the Storage tab estimates:

- current growth per hour and per day (PM, NETCONF and FM),
- the steady-state size under your retention settings,
- how many days it would take to reach the low-disk warning threshold **if cleanup were disabled**.

PM estimates use the row sizes measured in your own database once enough data exists; before that, calibrated defaults are used. NETCONF and FM growth come from the last 24 hours and 7 days of activity.

The **Retention impact estimator** lets you try settings before applying them. Enter NF count, counters per NF, PM interval, raw retention and aggregate retention to see the approximate disk usage. For example: 250 NFs, 12 counters, 5 s, 24 h raw, 30 d 5-minute and 90 d hourly.

Forecasts are estimates. Actual usage depends on the number of NFs, counters, PM frequency, NETCONF payload sizes, alarm frequency, scenario activity, retention settings and SQLite overhead.

61.2 Changing Retention Settings

Open *SYSTEM* > *Settings* > *Storage & Retention* and pick a preset, or set values individually:

Setting	MINIMAL (default)	STANDARD	EXTENDED
Raw PM	6 h	24 h	72 h
5-minute PM aggregates	7 d	30 d	90 d
Hourly PM aggregates	30 d	90 d	365 d
FM (cleared alarms)	30 d	90 d	365 d
NETCONF messages	7 d	14 d	30 d

Changing any of these values yourself switches the preset to **CUSTOM**, unless the values match a named preset exactly. MiniSMO never changes your preset on its own. **APPLY PRESET** loads a preset; **SAVE RETENTION SETTINGS** saves the values.

Other settings: application log retention (14 days), configuration revisions per NF (50), scenario history (90 days or 500 runs), capacity test history (50), NETCONF maximum size and maximum stored payload, automatic cleanup and schedule, disk warning and critical thresholds, maximum managed data size, and export cleanup (off).

Shortening a retention period takes effect at the next cleanup, or immediately with **CLEAN UP NOW**. Lengthening it only affects data that has not been deleted yet.

62. Exporting Capacity Reports

EXPORT CAPACITY REPORT on the System Capacity or History tab saves the latest (or selected) capacity test as:

- **PDF**: generated by MiniSMO from the HTML report,
- **HTML**,
- **JSON** with every result,
- **CSV** with the detailed stage measurements.

Files are saved to the exports folder as `Kaldrivon-MiniSMO-Capacity-Report-YYYYMMDD-HHMM.<ext>`. The report contains system information, license information, profiles, recommended and maximum tested NF counts, CPU/RAM and storage measurements, and this disclaimer: *Results are estimates based on the tested MiniSMO version, current computer, workload profile, retention settings, and system conditions. Results are not a guarantee of performance with all vendor equipment or production traffic patterns.*

Reports never contain device credentials or NF configuration.

63. Exporting Storage Reports

EXPORT STORAGE REPORT on the Storage tab saves a PDF, HTML, JSON or CSV report with: current total usage, size per category, retention settings, free disk space, estimated growth, estimated steady state, current warnings, and what is currently eligible for cleanup.

Files are saved to the exports folder as `Kaldrivon-MiniSMO-Storage-Report-YYYYMMDD-HHMM.<ext>`. Like all exports, they are created only when you ask and are never sent anywhere by MiniSMO.

64. Settings

SYSTEM > *Settings* has four tabs: **General**, **Storage & Retention**, **VES Collector** and **YANG Models**. The **General** tab holds application preferences. Changes are saved when you select **SAVE**.

Setting	Default	Notes
Theme	Dark	Dark, Light or System
Backend Port (restart required)	8765	Local port of the MiniSMO backend, 1024 to 65535. Takes effect after restarting MiniSMO. Change only if 8765 is used by another program
Auto-start Backend	On	Start the bundled backend automatically when MiniSMO opens
Default NETCONF Port	830	Pre-filled in the Add External NF dialog
NETCONF Connect Timeout (seconds)	10	1 to 120. Applies to TCP, SSH, authentication and the hello exchange
NETCONF Operation Timeout (seconds)	30	1 to 600. Maximum wait for an RPC reply
PM Update Interval (seconds)	5	1, 5, 15 or 30 seconds between stored simulated PM samples
PM read period when none is given (seconds)	300	60 to 86400. How often MiniSMO reads an external NF's PM model with `<get>` when neither you (per NF, *PM poll*) nor the NF's own configuration gives an interval. It is MiniSMO's schedule and is never shown as what the NF is expected to deliver
Simulated alarm activity	normal	off, low, normal or high: how often Demo Network NFs raise, escalate and clear alarms on their own (about 1, 3 or 12 per NF per hour). Also on *SIMULATION* > *Demo Network*
Logging Level	INFO	DEBUG, INFO, WARNING or ERROR for the backend log
Show recent Help topics	On	Show recently viewed topics in the Help Center

The **Integration** section decides what happens when you add an external NF:

Setting	Default	Notes
Connect and start monitoring new NFs	On	Connect, discover and start FM / CM / PM monitoring as soon as the NF is saved

Setting	Default	Notes
Point new NFs at the VES collector (and keep them pointed here)	On	Write the collector address into the NF's VES settings and keep it pointed there. See Pointing an NF at the Collector
Follow address changes of this PC	On	When this PC's address changes, update MiniSMO's own VES subscription and PM file upload entry on the NFs

The **Unlicensed NFs** section lists the external NFs beyond the license capacity (hidden: not connected, monitored or reported; their data is kept) with their integration numbers. **DE-INTEGRATE ALL HIDDEN NFS...** removes them after a confirmation, with progress: MiniSMO first removes its own VES subscription from each NF it can reach, then deletes the NF and its stored data. See Unlicensed (hidden) NFs.

The **Operations** section holds the settings of the northbound API, alarm flapping and correlation, approvals and retention of backups, audit and automation history.

Setting	Default	Notes
Northbound REST API listener (token-authenticated)	Off	See Northbound REST API
Listen on address	127.0.0.1	This PC only. 0.0.0.0 or another address exposes the API to the network
Port	8770	1024 to 65535
HTTPS (self-signed)	On	Uses the collector's certificate
Flapping: raises per hour	5	2 to 100
Do not notify or route flapping alarms	Off	
Correlation window (s)	120	10 to 3600
Approvals expire after (h)	24	1 to 720
Allow automation to apply changes without approval (administrator)	Off	
Backups kept per NF	50	Protected backups are not counted
Audit log retention (days)	365	30 to 3650
Automation history (days)	90	7 to 3650

The **VES collector** section holds the listener settings: **Collector enabled** (on), **Listen on address** (`0.0.0.0` , all network interfaces), **HTTPS port (self-signed)** (8443), **Collector address NFs should use** (empty: the local address that routes to each NF), **Accept events from** (Known NF addresses only) with **Other addresses allowed to send**, **Authentication NFs must use** (none, or basic with **Username** `minismo` and a password). See VES Collector.

The **NETCONF Call Home** section (off by default) has **Listen for Call Home connections**, the listen address (`0.0.0.0`), the port (4334) and **Wait for a call**

(seconds), how long a connect waits for an NF to call (60). It shows the listener state, the calls accepted and refused, a firewall hint, and the **REFUSED CALLERS** with **ADD AS NF...** and **CLEAR REFUSED LIST**. See NETCONF Call Home.

The **Storage & Retention** tab holds the retention policy (presets MINIMAL (default), STANDARD and EXTENDED, or CUSTOM), automatic cleanup and schedule, export cleanup, disk thresholds, the maximum managed data size and the retention impact estimator. See Changing Retention Settings.

Invalid values are refused with a message explaining the allowed range.

65. Logs

MiniSMO writes these logs:

Log	Location	Content
Backend log	`%LOCALAPPDATA%\Kaldrivon\MiniSMO\logs\backend.log`	Startup, migrations, errors, NETCONF session events. JSON lines, rotated at 10 MB, 10 files kept
Application log	`%LOCALAPPDATA%\Kaldrivon\MiniSMO\logs\app.log`	Windows application events, backend start attempts, UI errors. Rotated at 10 MB, 10 files kept
NETCONF Log	In the database, see *LOG* > *NETCONF Log*	Every NETCONF message with full XML

About > **OPEN LOG FOLDER** opens the logs folder in File Explorer.

Rotated log files older than the application log retention (14 days by default) are deleted by cleanup; the current log file is never deleted. Passwords are never written to any log. If you send logs to support, review them first: they can contain host names, IP addresses and device configuration.

66. Data Storage Locations

MiniSMO keeps program files and your data separate.

What	Where
Program files	`C:\Program Files\Kaldrivon\MiniSMO` (or the folder chosen at install)
All user data	`%LOCALAPPDATA%\Kaldrivon\MiniSMO`
Database	`...\MiniSMO\data\minismo.db` (SQLite)
Logs	`...\MiniSMO\logs`
Imported YANG models	`...\MiniSMO\yang`
Exports (alarms, inventory, PM, NETCONF Log, audit, snapshots, diagnostics)	`...\MiniSMO\exports`
Reports	`...\MiniSMO\exports\reports`
Automatic database backups	`...\MiniSMO\backups`
Temporary files	`...\MiniSMO\temp`
Installed license	`...\MiniSMO\license\license.klicense`

`%LOCALAPPDATA%` is usually `C:\Users\<you>\AppData\Local`. Type it into the File Explorer address bar to open it.

Data is per Windows user. Another user on the same PC has a separate inventory and license.

67. Backup Considerations

What to back up:

- `%LOCALAPPDATA%\Kaldrivon\MiniSMO\data\minismo.db` : inventory, configuration, alarms, PM history, NETCONF Log, settings.
- `%LOCALAPPDATA%\Kaldrivon\MiniSMO\yang` : imported YANG files.
- Your original `.klicense` file.

Close MiniSMO before copying the database, so the backend is not writing to it.

Stored device passwords are encrypted with Windows DPAPI for your Windows account on this PC. If you restore the database on another PC or under another Windows user, the passwords cannot be decrypted; edit each external NF and enter its password again.

MiniSMO makes an automatic copy of the database in `backups` before applying any database migration during an upgrade. To make a backup while MiniSMO is running, use **CREATE BACKUP** on *SYSTEM* > *Health & Capacity* > *Capacity & Storage* > *Storage*: it uses SQLite's online backup method, so the copy is consistent. A *Configuration* backup leaves out large PM, NETCONF and alarm history; a *Full* backup includes everything. See Storage Cleanup.

68. Team Mode and Web Access

68.1 Local and Team Mode

MiniSMO runs in one of two modes, chosen in *SYSTEM* > *Settings* > *Access*.

Local mode (the default) is MiniSMO as it has always been: the desktop app on this PC is the only way in, and there is no sign-in. The backend's application API listens only on this PC (127.0.0.1) and accepts only the desktop app and local tools of the same user (see Local API Protection).

Team mode adds web access for other people on your network. MiniSMO then also serves its web UI over HTTPS, and everyone who uses it from a browser signs in with an account and a role (see Users and Roles). The desktop app keeps working as before, as the PC's owner with full rights.

Turning Team mode on. Select **TURN ON TEAM MODE...**. The first time, you create the first admin account (user name and a password of at least 12 characters). MiniSMO then starts the HTTPS listener on port 9443 (setting *team_port*, all network interfaces by default, *team_bind*) and shows:

- the addresses to open in a browser,
- the SHA-256 fingerprint of the certificate (made on this PC; the browser warns about it, so compare the fingerprint before you continue),
- the command that opens the port in this PC's firewall. MiniSMO never changes the firewall by itself; run the command as administrator if other machines must reach MiniSMO.

Going back to Local mode. BACK TO LOCAL MODE... stops the HTTPS listener and ends every browser session. Accounts are kept for the next time.

Only the desktop app on the MiniSMO PC can switch the mode. Nobody can turn Team mode off or on from a browser, not even an admin.

Settings. *team_bind* and *team_port* (address and port of the HTTPS listener), *team_session_idle_min* (a session ends after this many minutes without use, default 480), *team_session_max_h* (and at the latest this many hours after sign-in, default 168), *team_public_url* (the address shown to users when MiniSMO is reached through a port mapping, NAT or a reverse proxy).

68.2 Users and Roles

Accounts exist for Team mode. Manage them in the desktop app (*SYSTEM* > *Users*) or, as an admin, in the web UI (**Users**).

Role	Can
Viewer	see everything: dashboard, NFs, alarms and alarm history, VES collector, PM, audit log; export CSV files

Role	Can
Operator	everything a viewer can, plus acknowledge, annotate and clear alarms, start and stop monitoring, run discovery, test connections, read configuration from NFs, take configuration backups, run reports and previews, check for updates
Admin	everything: add, change and remove NFs, apply configuration changes, settings, licenses, storage, users

A new write action in MiniSMO is admin-only until it is given to a lower role on purpose.

Passwords have at least 12 characters and are stored only as salted scrypt hashes. A new account can be made to change its password at the first sign-in. **RESET PASSWORD…** gives a one-time password, shown once; the user must choose a new password when signing in with it.

Protection. Five wrong passwords lock an account for 15 minutes (an admin can unlock it). One address can try at most ten sign-ins a minute. A wrong user name and a wrong password get the same answer. Changing the role, disabling or removing an account, a password change and a reset end that user's sessions. At least one enabled admin always remains: the last one cannot be removed, disabled or demoted.

Lost the only admin password? On the MiniSMO PC, run the backend with `--reset-password <user>`; it prints a one-time password. `--list-users` lists the accounts. Both work while MiniSMO runs.

Every action is in the Audit Log with the account's name and *WEB* when it was done in a browser, or the PC's user name and *UI* when it was done in the desktop app. Sign-ins, failed sign-ins, sign-outs, account changes and mode switches are recorded too.

68.3 Using MiniSMO in a Browser

In Team mode, open one of the addresses from *SYSTEM* > *Settings* > *Access* (for example `https://192.0.2.20:9443/`) in a current browser and sign in.

The web UI has the same pages as the desktop app, in the same groups (Overview, Network, FCAPS, Tools, Log, Simulation, System). What you can do depends on your role: buttons for actions your role may not take are not shown, and the server refuses them anyway. Files the desktop app opens on the PC (diagnostics, reports, exports, measurement files) are downloads in the browser. Only the desktop app on the MiniSMO PC switches Local / Team mode, runs the factory reset, and downloads and installs updates; the browser shows their state.

Times are shown in your browser's time zone, always with the date. Pages refresh by themselves and pause while the browser tab is hidden. Acknowledgements and notes stay in MiniSMO; they are never sent to the NF.

Security. The web UI uses HTTPS only, a session cookie that scripts cannot read and that is sent only to MiniSMO, a separate token against cross-site requests, and a strict content security policy (no inline or outside scripts). Sign out with **Sign out**; a session also ends after the idle time.

68.4 Running as a Windows Service

Normally MiniSMO runs while its window is open: minimizing the window only pauses the screen's refresh, and alarms, PM, configuration changes and VES keep being collected; closing the window stops MiniSMO. As a **Windows service** it keeps running when the window is closed, when you sign out of Windows and after the PC restarts, so monitoring and Team mode are always available.

Turning it on. In *SYSTEM* > *Settings* > *Access*, select **RUN AS A WINDOWS SERVICE...** and confirm. Windows asks for administrator permission once. MiniSMO then:

- stops, and copies its data folder to `%ProgramData%\Kaldrivon\MiniSMO` (your own folder stays unchanged),
- moves the stored device passwords to the service: they are protected for your Windows account, which a service cannot use, so they are re-encrypted with a key that only the service can read,
- registers the service *Kaldrivon MiniSMO* (`KaldrivonMiniSMO`): automatic start, its own Windows account `NT SERVICE\KaldrivonMiniSMO` with no administrator rights, restart after a failure,
- restricts the folder to that account and administrators; your Windows account may read the logs, exports and the key the app uses, and may start and stop the service,
- starts the service and restarts the app, which then connects to it.

Nothing else changes: the same backend program runs, with the same ports and firewall rules.

In service mode the window shows and controls the service as before. **START SERVICE / STOP SERVICE...** in *Access* start and stop it without administrator rights. **OPEN LOG FOLDER** opens the service's logs. Software updates work as before: the installer stops the service, replaces the program files and starts it again.

Turning it off. STOP RUNNING AS A SERVICE... (administrator permission once) removes the service and makes its data your data folder again, with the device passwords protected for your Windows account. Your data from before the service is kept beside it (`MiniSMO.before-service-<date>`), and the service's logs are in `logs\service-<date>` . This works while the app is open: what the app itself keeps in the folder (its browser data, its own settings and log) stays where it is.

Uninstalling MiniSMO removes the service and keeps its data in `%ProgramData%\Kaldrivon\MiniSMO` .

68.5 Running Without the Desktop App

The backend can run on its own, with Team mode for access. On Windows, use Running as a Windows Service; elsewhere, start it with `--headless` from a service manager. Configuration then comes from the environment as well as from the stored settings:

Variable	Purpose
`MINISMO_HOME`	data folder

Variable	Purpose
`MINISMO_LOG_DIR`, `MINISMO_LOG_STDOUT=1`	log folder; also log to standard output
`MINISMO_HOST`, `MINISMO_PORT`	the local application API (keep it on 127.0.0.1)
`MINISMO_SETTING_<KEY>`	any setting, for example `MINISMO_SETTING_ACCESS_MODE=team`; shown as read-only in the app while set
`MINISMO_ADMIN_USER`, `MINISMO_ADMIN_PASSWORD`	the first admin, created once at start when Team mode is on and no admin exists

Secrets can also come from a file: add `_FILE` to the variable name and give the file's path (for example `MINISMO_ADMIN_PASSWORD_FILE`). They are never written to the logs.

`/health/live` and `/health/ready` answer health checks without revealing anything about the installation. On a stop request the backend closes its NETCONF sessions, monitors and listeners before it exits. See `docs/DOCKER_READINESS.md` in the source for container use (not supported yet).

69. Software Updates

SYSTEM > *About* > *Software updates* gets new versions from Kaldrivon's releases on GitHub (`github.com/yoroyarell/KaldrivonMiniSMO/releases`):

1. **CHECK NOW** asks GitHub for the list of releases (`api.github.com` , no account needed) and shows the newest version above yours on your **Update Channel**: Stable offers releases only, Preview and Internal also pre-releases. Drafts and older versions are never offered, nor a release without its installer and `.sha256` file. With **Check for updates: Once a day** (the default), MiniSMO does this by itself once a day and adds a notification when a new version appears; choose *Only when I select CHECK NOW* to check only when you ask. **Update channel**: Stable offers releases only; Preview and Internal also offer pre-releases. Nothing else is sent, and the PC's proxy settings are used. When GitHub cannot be reached, the card says so.
2. **DOWNLOAD…** (confirmed) downloads `Kaldrivon-MiniSMO-<version>-Setup.exe` and its `.sha256` file into the data folder (`temp\updates`). The download is accepted only over HTTPS from GitHub's own servers, and only if its SHA-256 matches the published value and Windows reports a **valid Authenticode signature by Kaldrivon** (trusted, not revoked, timestamped). Anything else is deleted and the card says why. The `.sha256` file comes from the same place as the installer, so the signature is what shows the file is Kaldrivon's.
3. **INSTALL…** (confirmed) checks the file once more, then starts the installer. Windows asks for administrator permission, MiniSMO closes, and the usual setup wizard replaces the program files. It ends with *Launch Kaldrivon MiniSMO*.

Whether you update from here or run a downloaded `Setup.exe` yourself:

- Your data folder is kept.
- The backend stops, program files are replaced, and on next start the database is backed up and migrated.
- Help content and the User Guide are replaced with the versions matching the new application.

The Help Center shows the help content version and the application version. If they ever differ, a warning is displayed. The API is `GET /api/updates` , `POST /api/updates/check` , `POST /api/updates/download?confirm=true` and `POST /api/updates/install?confirm=true` (checks the file and returns its path; the desktop app starts it).

70. Resetting and Uninstalling

70.1 Reset MiniSMO

RESET MINISMO... at the bottom of *SYSTEM* > *Settings* makes MiniSMO like a fresh installation. It deletes everything in the data folder: all NFs and their integrations, monitoring, the database with all PM, FM and CM history, VES and NETCONF logs, configuration backups, templates, rules, schedules, webhooks, API tokens, settings, stored credentials, YANG models, exports, kept measurement file copies, application logs, certificates and caches.

You confirm three times, each time by typing a word:

1. **RESET**: the dialog lists what is deleted. Choose whether to **Keep the installed license** (on by default; when off, MiniSMO starts in Community edition) and whether to **de-integrate the external NFs first** (on by default).
2. **DELETE ALL DATA**
3. **MINISMO**: a progress window de-integrates each external NF (removes MiniSMO's own VES subscription, stops monitoring and closes the session), then schedules the reset. When you close it, MiniSMO restarts and the backend deletes everything *before* it opens the database or any other file.

Notes:

- De-integration removes only what MiniSMO itself wrote: its 3GPP `NtfSubscriptionControl` entry or, on an O-RU, its O-RAN WG4 configured subscription and heartbeat recipient, and the PM upload entry that points at MiniSMO's receiver. Other managers' entries are never touched. An NF that cannot be reached (or has no standard subscription model) is listed in the progress window; it keeps sending VES to this PC until you change it on the NF, and its events are stored as unattributed until you add the NF again. Without de-integration nothing is sent to any NF.
- The reset is recorded only in the reset result file; the audit log is deleted with everything else. Settings shows the time of the last reset, and how many items could not be deleted because another program had them open.
- Until MiniSMO restarts nothing is deleted, and **CANCEL PENDING RESET** keeps all data.
- In the dev build (the backend is started by Run-MiniSMO-Dev, not by the app), close MiniSMO and start Run-MiniSMO-Dev again after the third step; the reset runs when the backend starts.
- To keep only part of the data, use **PURGE DATA...** on *Capacity & Storage* > *Storage* instead (see Storage Cleanup).

70.2 Uninstalling

1. Close MiniSMO.
2. Open *Settings* > *Apps* > *Installed apps*, find **Kaldrivon MiniSMO** and select **Uninstall**. You can also use the Start Menu entry *Uninstall Kaldrivon MiniSMO*.

3. The uninstaller stops the backend and removes the program files, shortcuts, the Windows Firewall rules it added and the fonts its setup installed (fonts that were on the PC before are left alone). Silent: `unins000.exe /VERYSILENT /SUPPRESSMSGBOXES /NORESTART` in the installation folder.

Your data in `%LOCALAPPDATA%\Kaldrivon\MiniSMO` is **kept**, so reinstalling brings everything back. To remove it as well, delete that folder manually after uninstalling. This also removes your installed license, so keep a copy of the `.klicense` file.

71. Troubleshooting

71.1 Troubleshooting

Start with the symptom:

Symptom	Go to
MiniSMO opens but says the backend could not start	Backend Startup Problems
TEST CONNECTION or CONNECT fails	NETCONF Connection Troubleshooting
License not accepted, or capacity not what you expect	Licensing Troubleshooting
An error message you don't recognise	Common Error Messages
Scenario won't start	Create the Demo Network first
Configuration page has no edit buttons	The selected NF is external and changes are not enabled for it. Choose **ENABLE CHANGES...** (see Changing External Configuration)
No PM data	Simulated NFs: check the Demo Network exists. External NFs: check the NF's **Monitoring** tab (PM state, **METHODS & FALLBACKS**); see External Performance Data
Storage Warning or Critical on the Dashboard	Storage Warnings
PC slow with many NFs	Run a capacity test: Running a Capacity Test

When you contact support, include the MiniSMO version (**About**, at the bottom of the navigation pane), what you did, the exact error text, and the backend log.

71.2 Backend Startup Problems

The MiniSMO window talks to a local backend process, `KaldrivonMiniSMO.Backend.exe`, on `http://127.0.0.1:8765`. If it cannot reach the backend within about 30 seconds, it shows **Backend failed to start** with the reason.

Cause	What you see	What to do
Port 8765 is used by another program	*port 8765 is in use*, a health check from a different service, or *Another program is answering on the backend port and could not prove it is your MiniSMO backend*	Change **Backend Port** in Settings (the diagnostic panel lets you pick a new port), then restart MiniSMO. Or run `netstat -ano` and look for port 8765 to find the program. MiniSMO sends nothing to a program that cannot prove it is your backend
Backend blocked by antivirus or application control	Backend exits immediately	Allow `C:\Program Files\Kaldrivon\MiniSMO\backend\KaldrivonMiniSMO.Backend.exe`
Damaged installation	*backend executable not found*	Reinstall MiniSMO (your data is kept)
Data folder not writable or disk full	Error in backend log about `minismo.db`	Free disk space; check permissions on `%LOCALAPPDATA%\Kaldrivon\MiniSMO`
Database damaged	Migration or database error	Close MiniSMO, move `data\minismo.db` elsewhere, start again (a new database is created), and restore a backup from `backups` if needed
Auto-start Backend is off	"Backend not running"	Select **Start backend** in the diagnostic panel, or turn the setting back on

The diagnostic panel has **Retry**, **Open log folder** and **Open startup troubleshooting** (this topic). The backend log is at `%LOCALAPPDATA%\Kaldrivon\MiniSMO\logs\backend.log`.

71.3 Local API Protection

The MiniSMO window and its backend talk over `http://127.0.0.1:8765` (the **Backend Port**). Loopback is not a password: every program and every Windows user on

the PC, and every web page open in a browser, can reach it. So the backend accepts a request only when:

- it is addressed to the loopback interface (`127.0.0.1` , `localhost` or `::1`); a web page that rebinds its own domain name to 127.0.0.1 is refused (*FORBIDDEN_HOST*);
- it does not come from a web page (browsers add an `Origin` header; MiniSMO never does) (*FORBIDDEN_ORIGIN*);
- it carries the backend's key in the `X-MiniSMO-Key` header (*UNAUTHORIZED* otherwise).

The key is new every time MiniSMO starts. The window hands it to the backend it starts and sends it with every request. The backend also writes it to `data\api.key` in your data folder (`%LOCALAPPDATA%\Kaldrivon\MiniSMO`), which only your Windows account can read, so local tools you run yourself can use the API. Before using a backend, the window asks it to prove that it holds the key (an HMAC over a random challenge on `/health`); a program that took the port without the key is reported and never sent anything.

Other systems, scripts on other PCs and other users use the Northbound REST API with their own token instead.

72. NETCONF Troubleshooting

Run **TEST CONNECTION** and look at the first stage that failed.

Problem	Possible causes	Actions
Host cannot be resolved	Wrong host name; DNS not available; name only resolvable on VPN	Check the spelling; try the IP address; check network and VPN connection
TCP 830 failed (timeout or refused)	Firewall between PC and device; wrong port; NETCONF disabled; device down	`Test-NetConnection <host> -Port 830`; confirm the port on the device; check Windows Firewall and network ACLs; confirm the device is powered and reachable
SSH handshake failed	Port answers but is not SSH; incompatible SSH algorithms	Confirm the port; check the device supports modern SSH key exchange and ciphers
Host key changed	Device replaced or re-keyed; or someone intercepting the connection	Compare the new fingerprint (shown in the error and in the red bar on the Overview tab) with the one the device shows. If legitimate, choose **FORGET HOST KEY AND RECONNECT...**
SSH authentication failed	Wrong username or password; account locked; unsupported auth method (public key only)	Re-enter credentials with EDIT; check the account on the device; enable password or keyboard-interactive auth
NETCONF subsystem unavailable	SSH works but the `netconf` subsystem is not configured on this port or for this user	Enable NETCONF over SSH on the device; check you are using the NETCONF port, not the CLI port
Server Hello timeout	Incompatible NETCONF implementation; device NETCONF service hung or overloaded; timeout too short	Retry; raise **NETCONF Connect Timeout**; restart the device's NETCONF service
Server hello is not well-formed XML	Non-compliant implementation	Capture the hello from the NETCONF Log and report it to the device vendor
Datastore not supported	Device did not advertise `:candidate` or `:startup`	Use *running*
RPC error in reply	Device refused the request (access control, bad filter)	Read the error message in the reply; check NACM permissions; remove or correct the subtree filter

Problem	Possible causes	Actions
Timed out waiting for server response	Large reply; slow device	Use a subtree filter; raise **NETCONF Operation Timeout**
Community supports up to 5 external managed NFs (or the licensed limit)	License capacity is used up	Delete an unused external NF or install a higher-capacity license

Every attempt, successful or not, leaves the exchanged XML in the NETCONF Log, which helps when you talk to the device vendor.

73. Licensing Troubleshooting

Problem	Likely cause	What to do
License signature is invalid	The file was edited, re-saved by an editor that changed it, or damaged in transfer	Install the original file you received. Do not open and save it in a text editor
Not a Kaldrivon license file / license file is not valid JSON	Wrong file, or an email client changed it	Ask Kaldrivon to resend it as an attachment or in a zip
License signing key … is not trusted by this build	The license was issued with a newer signing key than this version knows about	Update MiniSMO, or ask Kaldrivon for a license issued for this version
This license is not for Kaldrivon MiniSMO	License for a different Kaldrivon product	Ask Kaldrivon for the MiniSMO license
Status **LICENSE EXPIRED**	The subscription or trial end date has passed	Renew. Your data is kept
Status **NOT_YET_VALID**	The license start date is in the future	Wait for the start date or check your PC's clock
Status **MACHINE_MISMATCH**	Machine-bound license installed on another PC, or Windows was reinstalled	Ask Kaldrivon for a license for your new machine fingerprint
Kaldrivon Mini SMO is free for up to 5 external NFs …	Usage equals capacity	Delete unused external NFs or install a license
N unlicensed in the title bar, NFs missing from the lists	More NFs are stored than the capacity allows; the newest are hidden	Install a license, bring one back into a free slot, or de-integrate them in Settings
… already manages this NF (DUPLICATE_NF)	An NF with the same address and port (or Call Home identity) is already stored	Edit or delete the existing NF
Status **INVALID** after it used to work	The stored license file was modified or damaged	Reinstall the original license file

If none of these apply, see Support Information.

74. Common Error Messages

Message	Meaning	Where to look
Community supports up to 5 external managed NFs (5/5 in use)	No free external NF capacity	External NF Capacity
A network function named '...' already exists	Names must be unique	Pick another name
Password is required	Password and keyboard-interactive sign-in need a password	Add External NF dialog
A private key is required for public-key authentication	SSH key sign-in chosen without a key	SSH Authentication Methods
NETCONF port must be between 1 and 65535	Invalid port	Use 830 unless the device uses another port
Host Resolution: cannot resolve '...'	DNS failure	NETCONF Connection Troubleshooting
TCP Connection: timed out after 10s	Device unreachable or port filtered	NETCONF Connection Troubleshooting
Authentication: authentication failed	Wrong credentials	NETCONF Connection Troubleshooting
NETCONF subsystem: ... refused	NETCONF not enabled on that port	NETCONF Connection Troubleshooting
The device does not advertise the '...' datastore	Unsupported datastore	Running, Candidate and Startup Datastores
A parameter of an external NF cannot be edited	Changes are not enabled for that NF, or the parameter is a key, `config false` or not in a writable datastore	How MiniSMO Writes to External NFs
alarms can only be generated on simulated network functions	FM generation is simulator-only	About Simulated Alarms
No simulated network functions. Create the Demo Network first.	Scenarios need the Demo Network	Creating the Demo Network
The Demo Network already exists	Only one Demo Network	Use **RESET** on *SIMULATION > Demo Network*
License signature is invalid	License file modified or damaged	Licensing Troubleshooting

Message	Meaning	Where to look
LICENSE EXPIRED	Subscription or trial ended	Expired License, Downgrades and Unlicensed NFs
Backend failed to start	Local backend unavailable	Backend Startup Problems
This action is destructive; repeat the request with confirm=true	API call without confirmation (scripts only)	Add `confirm=true`

"Not well-formed XML" or "invalid XML characters" in the NETCONF Log

XML 1.0 does not allow control characters other than tab, line feed and carriage return. Some devices still send them, most often the `^C` delimiter of a configuration banner. Such a reply is not XML. When the only problem is these characters, MiniSMO replaces each one with the replacement character `�` (U+FFFD), uses the rest of the reply, and adds an *invalid-xml-characters* entry (WARNING) to the NETCONF Log saying how many were replaced. Nothing else in the reply changes. If the reply has other errors, the request fails at once with the parser's reason, and the *malformed* entry shows the reason and where in the reply it occurred, instead of a later "timed out waiting for server response". To remove the cause on the device, recreate the banner with a printable delimiter.

When the whole configuration reply is not XML for another reason, for example one model's data with an element the device never closes (seen on Cisco IOS XE in `ietf-diffserv-classifier`), MiniSMO reads the configuration **part by part**: one `<get-config>` with a subtree filter for each top-level part named in the reply (RFC 6241). Every part the NF returns as XML is kept; the parts it cannot return are named in a *get-config (by parts)* entry of the NETCONF Log, on the Monitoring CM card and in Configuration, and are not in the snapshots. Later reads, by any session to that NF, go straight to the parts; the complete read is tried again every 6 hours in case the device was fixed. The session stays connected. Report the broken part to the device vendor.

75. Security Notes

- **Local only API.** The MiniSMO backend API listens on `127.0.0.1` (this PC only). It is not reachable from the network. It has no user authentication, so do not expose it through port forwarding or proxies.
- **Northbound REST API.** Off by default. When you turn it on it listens on `127.0.0.1:8770` (HTTPS with the collector certificate) unless you choose another address, and every request needs a bearer token. Tokens are shown once and stored only as a SHA-256 hash; revoke a token as soon as it is no longer needed.
- **Webhooks** send event data to the URLs you configure. Use HTTPS and a signing secret; the secret is encrypted with DPAPI and never shown again.
- **Audit log.** Changes made in MiniSMO (settings, licenses, NFs, configuration, automation, tokens) are recorded with the Windows user name. Passwords, secrets and tokens are never written to the audit log.
- **VES collector.** The VES event listener is the one network-facing part: HTTPS only, on 8443, on all interfaces, allowed through Windows Firewall on Domain and Private networks by the installer. It only accepts VES event POSTs, and by default only from the addresses of your NFs. Turn on HTTP Basic authentication in Settings when NFs outside a closed lab can reach this PC, or switch the collector off.
- **Device passwords and SSH keys** are encrypted with Windows DPAPI under your Windows account before they are stored. They are never written in plain text to the database, logs or exports, and are never shown again after you enter them.
- **SSH host keys** are pinned on first connect. A changed key stops the connection until you confirm the change with **Forget host key**.
- **Writes to external devices are limited**: the VES destination (set automatically when the NF is added, unless turned off) and configuration changes you enable and confirm per NF. One more request acts on the NF without changing its configuration: a measurement file upload (`file-upload`) that you confirm. Every other write is refused before it is sent.
- **Licenses** are signed by Kaldrivon and checked locally on every start. Changing any value in a license file invalidates it. Licensing controls only the number of external NFs, never features or access to your data: removing or letting a license expire keeps all data, NFs and monitoring.
- **Help and documentation** are bundled and rendered locally. The Help Center does not run scripts or load anything from the Internet.
- **Capacity and storage data** stay on your computer. Machine details, capacity results and storage figures are never uploaded; reports are only created when you export them.
- **Logs and exports** can contain device names, addresses and configuration. Treat them as you would any network documentation.
- **Updates**: only install MiniSMO from Kaldrivon and check the SHA-256 hash published with each release.

76. Known Limitations

Version 0.6.0 has these limitations:

- Configuration writes to external NFs happen only for NFs where you have enabled editing, after a preview and your confirmation (automation actions that change the network always wait for approval).
- No user accounts or role-based access control; one Windows user. Northbound API tokens have READ or OPERATE scope only.
- Software campaigns are planned and checked, not executed: MiniSMO does not download or activate software on NFs.
- Webhook delivery is at-least-once with retries; receivers should tolerate duplicates.
- Scheduled jobs run only while MiniSMO is running.
- VES provisioning uses the 3GPP `NtfSubscriptionControl` model (TS 28.623) and, for O-RUs, the O-RAN WG4 configured subscription (`o-ran-ves-subscribed-notifications`). For NFs with neither, set the VES destination with the vendor's tools; the collector accepts their events anyway. Neither model has credential fields, so collector credentials are set on the NF. Event-collector discovery by DHCP (O-RAN WG4) is not used.
- No complete YANG-driven user interface. YANG models are stored and displayed, and used for constraints when you edit an external NF's configuration.
- No configured (persistent) NETCONF subscriptions; subscriptions are dynamic and end with the session.
- No guaranteed O-RAN conformance; MiniSMO is not a conformance test tool.
- No A1, O2 or E2 interfaces.
- No Non-RT RIC and no rApp runtime. The R1 interface offers CAPIF service discovery and data jobs; CAPIF security and CAPIF events are not provided, and the DME information types are MiniSMO's own.
- No high availability; single PC, single backend process.
- No multi-user access; one Windows user, local only.
- No Kubernetes or container platform required (or supported).
- Topology relationships for external NFs are not discovered; you place external O-RUs and O-DUs yourself (stored in MiniSMO only).
- One scenario can run at a time.
- Capacity test results are estimates for the tested PC and workload profiles, not guarantees for all vendor equipment or production traffic.
- External device compatibility depends on each device's NETCONF behaviour. Devices that deviate from RFC 6241/6242 may fail the hello exchange.

77. Glossary

Term	Meaning
Alarm	A notification that a fault condition exists on an NF, with a type and severity. Cleared when the condition goes away
Capability	A feature or YANG module a NETCONF server announces in its hello message, identified by a URI
CM	Configuration Management: reading and changing NF configuration
Datastore	A named copy of configuration on a NETCONF server: running, candidate or startup
External NF	A real network function managed by MiniSMO over NETCONF/SSH. Counts against license capacity
FM	Fault Management: alarm handling
NETCONF	Network Configuration Protocol (RFC 6241), XML-based protocol for managing network devices
NF	Network Function: a network element such as an O-RU or O-DU
O1	O-RAN interface between the SMO and O-RAN network functions for FCAPS management, based on NETCONF/YANG among others
O-CU-CP	O-RAN Central Unit, Control Plane: RRC and PDCP-C, controls one or more O-DUs over F1-C
O-CU-UP	O-RAN Central Unit, User Plane: PDCP-U/SDAP, carries user traffic to and from O-DUs over F1-U
O-DU	O-RAN Distributed Unit: real-time L2 and upper PHY, serves one or more O-RUs over fronthaul
O-RAN	Open RAN architecture specified by the O-RAN Alliance, with open interfaces between RAN components
O-RU	O-RAN Radio Unit: lower PHY and RF, connected to an O-DU over the open fronthaul
PM	Performance Management: counters and KPIs such as PRB utilization and throughput
RPC	Remote Procedure Call: a NETCONF request such as `<get>`, answered with an `<rpc-reply>`
Scenario	A scripted sequence of simulated events, such as a cell outage
Simulation	MiniSMO's built-in generation of synthetic NF state, alarms and counters
SMO	Service Management and Orchestration: the O-RAN management layer that hosts FCAPS functions and the Non-RT RIC
SSH	Secure Shell: the encrypted, authenticated transport NETCONF runs over

Term	Meaning
YANG	Data modeling language (RFC 7950) that defines the structure of NETCONF configuration and state data
Hello	The first NETCONF message each side sends, carrying capabilities and (from the server) the session ID
Session ID	Number the NETCONF server assigns to a session in its hello
Message ID	Identifier on each RPC, echoed in the matching reply
Subtree filter	XML fragment that limits what a `<get>` or `<get-config>` returns
Host key	The SSH server's public key; MiniSMO pins its fingerprint on first connect
.klicense	Kaldrivon license file format; signed, validated locally
PRB	Physical Resource Block: the unit of radio resource allocation in NR
PCI	Physical Cell Identity (0 – 1007 in NR)
NR-ARFCN	NR Absolute Radio Frequency Channel Number, identifies the carrier frequency
TAC	Tracking Area Code
MCC / MNC	Mobile Country Code / Mobile Network Code, together the PLMN identity

78. Support Information

78.1 Support Information

Before contacting support, collect:

1. The version from **About** at the bottom of the navigation pane (and the Help content version shown in the Help Center).
2. What you were doing and the exact error text (the **COPY** buttons help).
3. The backend log from `%LOCALAPPDATA%\Kaldrivon\MiniSMO\logs` (**About > OPEN LOG FOLDER**).
4. For NETCONF problems: the relevant NETCONF Log entries exported as JSON.
5. For licensing problems: your License ID (shown on *SYSTEM* > *License*). Do not send the license file unless asked.

Contact: **info@kaldrivon.com** Website: **https://kaldrivon.com**

78.2 About MiniSMO

About (at the bottom of the navigation pane) shows:

- Product name and version: **Kaldrivon MiniSMO 0.6.0**
- Description: RAN operations without the Tier-1 OSS footprint: O1 / NETCONF and VES management of O-RAN network functions, with a built-in simulated network for labs and demos.
- The data folder.
- Company: Kaldrivon, © 2026 Kaldrivon, https://kaldrivon.com

Buttons: **OPEN USER GUIDE** (opens the installed HTML User Guide), **OPEN HELP**, **VIEW LICENSE** and **OPEN LOG FOLDER**.

Kaldrivon MiniSMO is an independent product and is not represented as certified or endorsed by the O-RAN Alliance, 3GPP, ETSI, any network operator, or any equipment vendor.

Third-party open-source components and their licenses are listed in `THIRD-PARTY-NOTICES.txt` in the installation folder.

Keyword Index

1.0 — NETCONF over SSH
1.1 — NETCONF over SSH
1000 nf — Running a Capacity Test
2222 — Communication Matrix
32.300 — Identity Mapping
32.435 — Measurement Files (File-based PM)
3gpp — YANG Library Discovery
4334 — Communication Matrix
5-minute — PM Aggregation, PM Retention, Understanding MiniSMO Data Storage
830 — Communication Matrix
8443 — Communication Matrix, VES Collector
8830 — Lab Simulator Checks
a1 — Known Limitations
about — About MiniSMO
about minismo — What Kaldrivon MiniSMO Is
access — Local and Team Mode
accounts — Users and Roles
acknowledge — Write Protection
action — Automation
add — Adding an External NF
add as nf — NETCONF Call Home
add external network function — Adding an External NF
admin — Users and Roles
admin state — Configuration Management
aggregation — PM Aggregation
alarm — About Simulated Alarms, External Alarms, FM Retention, Fault Management
alarm console — Fault Management
alarm history — Alarm History
alarm search — Alarm History
alarm types — About Simulated Alarms
alarms — Dashboard, Fault Management
alerts — Notifications
all features — Editions and Capacity
allow list — NETCONF Call Home
allowance exceeded — What a License Changes
always on — Running as a Windows Service
analytics — Analytics
antivirus — Backend Startup Problems
api — Northbound REST API
api key — Local API Protection
apply — Configuration Management
approval — Approvals
approve — Approvals
arfcn — Configuration Management
assign — Matching Events to NFs
audit — Audit Log, Configuration History
audit purge — Storage Cleanup
authentication — NETCONF Connection Test, NETCONF Connection Troubleshooting, SSH Authentication Methods
automatic — Automatic Cleanup
automation — Automation
average — PM Aggregation
backend — Backend Startup Problems, First Launch
backend port — Local API Protection, Settings
backend.log — Logs
background — Running as a Windows Service
backup — Backup Considerations, Comparing Configurations, Configuration Backups, Storage Cleanup
backups — Configuration Backups, Storage Capacity, Storage Cleanup
backups purge — Storage Cleanup
behind nat — NETCONF Call Home
bell — Notifications
benchmark — Exporting Capacity Reports, Running a Capacity Test, Understanding Technical Capacity
benchmark test — Running a Capacity Test
blocked — Write Protection
bottleneck — Capacity vs Storage Constraints
bring back — Expired License, Downgrades and Unlicensed NFs
browser — Local and Team Mode, Using MiniSMO in a Browser
bulk — Bulk Operations
bulk pm — Measurement Files (File-based PM)
call home — NETCONF Call Home
campaign — Software Upgrade Campaigns
cancel — Running a Capacity Test
candidate — Changing External Configuration, Running, Candidate and Startup Datastores
capabilities — Capability Discovery, NETCONF Capability Discovery, NF Identity and Cells, The Hello Exchange
capability — NETCONF Capability Discovery
capability matrix — Capability Discovery

capacity — Capacity & Storage Overview, Capacity vs Storage Constraints, Community Edition and Licensing, Editions and Capacity, Exporting Capacity Reports, External NF Capacity and License Limits, License Capacity vs Technical Capacity, Running a Capacity Test, Understanding Technical Capacity, What a License Changes
capacity and storage — Capacity & Storage Overview
capacity test — Running a Capacity Test
capif — O-RAN R1 Interface
cells — NF Identity and Cells
change configuration — Changing External Configuration
chunked — NETCONF over SSH
clean up now — Storage Cleanup
cleanup — Automatic Cleanup, Storage Cleanup
clear alarm — Fault Management
cleared alarms — Alarm History
cm — Configuration Management, Glossary
collector — VES Collector
collector address — Pointing an NF at the Collector
commit — Changing External Configuration, How MiniSMO Writes to External NFs
communication matrix — Communication Matrix
community — Community Edition and Licensing, Editions and Capacity, First Launch
community limits — What a License Changes
community vs pro — Editions and Capacity
compare — Comparing Configurations, External Configuration History
compare with — External Performance Data
compliance — Desired State and Drift
config backup — Configuration Backups
config change — External Configuration History
configuration — Configuration History, Configuration Management, Running GET-CONFIG
configuration change — Changing External Configuration
configuration history external — External Configuration History
confirm — Confirming the SSH Host Key
confirm host key — Confirming the SSH Host Key
confirmation — Application Layout
confirmed — Storage Cleanup
confirmed commit — Rollback
confirmed-commit — Changing External Configuration
congestion — Running Scenarios
connected — NETCONF Connection Test
connection — NETCONF Connection Troubleshooting
connection health — Subscription States
contact — Support Information
container — Running Without the Desktop App
content-id — YANG Library Discovery
conventions — Introduction
copy — Viewing Raw NETCONF XML
correlation — Analytics
counter — External Performance Data
counters — About PM Simulation, Performance Management
cpu — Running Scenarios, Running a Capacity Test, Understanding Technical Capacity
create — Creating the Demo Network
critical — About Simulated Alarms, Storage Warnings
csv — External Performance Data
daily — Schedules
dashboard — Dashboard
data — Data Storage Locations
data job — O-RAN R1 Interface
database — Backup Considerations, Data Storage Locations
database size — Storage Capacity, Understanding MiniSMO Data Storage
datastore — Running GET-CONFIG, Running, Candidate and Startup Datastores
date range — Alarm History
de-integrate — Expired License, Downgrades and Unlicensed NFs
deconfigure — Pointing an NF at the Collector
definitions — Glossary
delete — Network Function Inventory
delete all data — Reset MiniSMO, Storage Cleanup
delete everything — Reset MiniSMO
delete nf — Pointing an NF at the Collector
deleted nf — Pointing an NF at the Collector
demo network — Creating the Demo Network, Understanding the Demo Topology
desired — Configuration Templates
desired state — Desired State and Drift

diagnostics — Diagnostic Collection, Network Function Inventory
diff — Comparing Configurations, External Configuration History
disclaimer — About MiniSMO
discover — Capability Discovery
discovery — YANG Library Discovery
disk — Capacity & Storage Overview, Capacity vs Storage Constraints, Storage Capacity, Storage Forecast, Storage Warnings, System Requirements, Why PM Data Can Grow Quickly
disk space warnings — Storage Warnings
distinguished name — Identity Mapping
dme — O-RAN R1 Interface
dn — Identity Mapping
dn prefix — Identity Mapping
dns rebinding — Local API Protection
docker — Lab Simulator Checks, Running Without the Desktop App
downgrade — Editions and Capacity, Expired License, Downgrades and Unlicensed NFs, What a License Changes
dpapi — Backup Considerations, Security Notes
drift — Desired State and Drift
drift detection — Desired State and Drift
e2 — Known Limitations
edit — Configuration Management
edit external config — Changing External Configuration
edit-config — Changing External Configuration, How MiniSMO Writes to External NFs, Pointing an NF at the Collector, Write Protection
edition — Community Edition and Licensing, Editions and Capacity
editions — Editions and Capacity
email — Support Information
entitlement — Capacity & Storage Overview, License Capacity vs Technical Capacity
environment — Running Without the Desktop App
error — Common Error Messages
errors — Common Error Messages
estimate — Storage Forecast
estimator — Storage Forecast
event listener — VES Collector
eventlistener — VES Collector
example — Example: Adding an External O-RU
expired — Expired License, Downgrades and Unlicensed NFs
expired license — Expired License, Downgrades and Unlicensed NFs
export — Exporting Capacity Reports, Exporting Storage Reports, Inventory and Network Tree, NETCONF Log
export csv — Fault Management
export dialog — Application Layout
exports — Storage Capacity, Storage Cleanup
extended — Changing Retention Settings
external — Network Function Inventory
external alarms — External Alarms
external nf — Adding an External NF, Example: Adding an External O-RU, External NF Capacity and License Limits
external nf limit — External NF Capacity and License Limits
external nf monitoring — Dashboard
external pm — External Performance Data
f1 — Application Layout, Understanding the Demo Topology
factory reset — Reset MiniSMO
failed to start — Backend Startup Problems
fallback — Method Choice and Fallback
fault — VES Collector
fcaps — Monitoring External NFs
features — Editions and Capacity, Version 0.4 Scope
file — Measurement Files (File-based PM)
file ready — Measurement Files (File-based PM)
file-based pm — Measurement Files (File-based PM)
file-upload — How MiniSMO Writes to External NFs, Measurement Files (File-based PM), Write Protection
fileready — Measurement Files (File-based PM)
files — Storage Cleanup
files on nf — Measurement Files (File-based PM)
find — Search
findings — Software Upgrade Campaigns
fingerprint — Confirming the SSH Host Key
firewall — Communication Matrix, NETCONF Connection Requirements, NETCONF Connection Troubleshooting
firewall 4334 — NETCONF Call Home
firewall rules — Communication Matrix

first run — First Launch
flapping — Analytics
fm — FM Retention, Fault Management, Glossary
folder — Data Storage Locations
forbidden host — Local API Protection
forbidden origin — Local API Protection
framing — NETCONF over SSH
free — Community Edition and Licensing
free edition — Community Edition and Licensing
free space — Storage Capacity, Storage Cleanup, Storage Warnings
fresh — Reset MiniSMO
freshness — Subscription States
fronthaul — Understanding the Demo Topology
ftpes — Measurement Files (File-based PM)
gap — Subscription States
generate alarm — Fault Management
get — Running GET
get config — Running GET-CONFIG
get-config — Running GET-CONFIG
get-schema — Changing External Configuration
glossary — Glossary
granularityperiod — External Performance Data
growth — Storage Forecast, Why PM Data Can Grow Quickly
guide — Introduction
hardware id — NETCONF Call Home
headless — Running Without the Desktop App
health — Running Without the Desktop App
health report — Reports and System Health
heartbeat — VES Collector
heartbeat missed — Fault Management
hello — NETCONF Capability Discovery, NETCONF Connection Test, The Hello Exchange
hello timeout — NETCONF Connection Troubleshooting
help with netconf connection — NETCONF Connection Requirements
hidden nf — Expired License, Downgrades and Unlicensed NFs
hidden nfs — Expired License, Downgrades and Unlicensed NFs
history — Alarm History, Audit Log, Changing Retention Settings, Configuration History, FM Retention, NETCONF Log Retention, PM Retention, Why PM Data Can Grow Quickly
hmac — Webhooks
home page — Dashboard
host key — Confirming the SSH Host Key, NETCONF Connection Test, NETCONF over SSH, Security Notes
host key fingerprint — NETCONF Call Home
host key not confirmed — Confirming the SSH Host Key
hostname — Adding an External NF
hourly — PM Aggregation, PM Retention, Understanding MiniSMO Data Storage
how licensing works — Community Edition and Licensing
how many nfs — Capacity & Storage Overview, License Capacity vs Technical Capacity, Running a Capacity Test
https — Local and Team Mode, Using MiniSMO in a Browser
human in the loop — Approvals
iana — Why Port 830?
identity — Identity Mapping, NF Identity and Cells
identity conflict — Identity Mapping
ietf-alarms — External Alarms
ietf-hardware — NF Identity and Cells
ietf-yang-library — YANG Library Discovery
import — Importing a YANG Model
import file — Measurement Files (File-based PM)
importing yang models — Importing a YANG Model
inbound — Communication Matrix
install — Installation
install license — Installing a License
installer — Installation
integration — Settings, Webhooks
integration number — Expired License, Downgrades and Unlicensed NFs
internal — Fault Management
internal alarm — Fault Management
interval — Performance Management
inventory — External Alarms, Inventory and Network Tree, Network Function Inventory
inventory & other — Storage Cleanup
investigate — Investigate
ip — Search

job — Bulk Operations
keep — Configuration Backups
key — SSH Authentication Methods
keyboard — Application Layout
keyboard-interactive — SSH Authentication Methods
klicense — Community Edition and Licensing, Installing a License
kpi ranking — Analytics
kpi rankings — Analytics
kpi threshold — Automation
lab — Example: Adding an External O-RU, Lab Simulator Checks
lab setup — Lab Simulator Checks
label — Identity Mapping
layers — Topology View
layout — Application Layout
license — Community Edition and Licensing, Editions and Capacity, Expired License, Downgrades and Unlicensed NFs, External NF Capacity and License Limits, Installing a License, Licensing Troubleshooting, What a License Changes
license and capacity — Dashboard
license indicator — Dashboard
license limit — External NF Capacity and License Limits
licensing — Community Edition and Licensing
limit — External NF Capacity and License Limits
limit reached — External NF Capacity and License Limits
limitations — Known Limitations
link failure — Running Scenarios
list — Network Function Inventory
local — Local and Team Mode
local api — Local API Protection
local api security — Local API Protection
local note — Fault Management, Write Protection
localappdata — Data Storage Locations
localhost — Security Notes
lockout — Users and Roles
log — Audit Log, NETCONF Log
log folder — Logs
logs — Logs
loss of supervision — Fault Management
low disk — Storage Warnings
machine mismatch — Licensing Troubleshooting
machine-bound — Installing a License
major — About Simulated Alarms
man-in-the-middle — Confirming the SSH Host Key
managed element — Identity Mapping
managed element id — Identity Mapping
managed object id — Identity Mapping
manual cleanup — Storage Cleanup
many nfs — Bulk Operations
max — PM Aggregation
maximum tested — Understanding Technical Capacity
meascollecfile — Measurement Files (File-based PM)
measurement job — External Performance Data
measurement-result-stats — External Performance Data
memory — System Requirements
menu — Application Layout
message id — NETCONF Log
messages — Common Error Messages
method — Method Choice and Fallback
method choice — Method Choice and Fallback
methods & fallbacks — Method Choice and Fallback
min — PM Aggregation
minimal — Changing Retention Settings
minismo — Reset MiniSMO
mo id — Identity Mapping
mode — Network Function Inventory
model — NF Identity and Cells
models — YANG Model Registry
module — NETCONF Capability Discovery, YANG Model Registry
modules — YANG Library Discovery
monitoring — Monitoring External NFs
monitoring column — Monitoring External NFs, Network Function Inventory
most alarms — Analytics
multi-user — Local and Team Mode
nacm — NETCONF Connection Requirements
namespace — Method Choice and Fallback
native id — Identity Mapping
navigation — Application Layout
netconf — Adding an External NF, Glossary, NETCONF Connection Troubleshooting, What is NETCONF?
netconf call-home — NETCONF Call Home

netconf log — NETCONF Log, NETCONF Log Retention
netconf manager — What Kaldrivon MiniSMO Is
netconf troubleshooting — NETCONF Connection Troubleshooting
network — Communication Matrix
network function — Adding an External NF
network functions — Network Function Inventory
network tree — Inventory and Network Tree
new device — Adding an External NF
nf connects to minismo — NETCONF Call Home
nf dials in — NETCONF Call Home
nf list — Network Function Inventory
no site — Inventory and Network Tree
no writes — Write Protection
nodes — Topology View
non-rt ric — O-RAN R1 Interface
northbound — Northbound REST API
northbound api — Northbound REST API
not supported — Known Limitations
not yet valid — Expired License, Downgrades and Unlicensed NFs
notifications — Monitoring External NFs, Notifications
nts — Pointing an NF at the Collector
o-du — Adding an External NF, Creating the Demo Network
o-du-001 — Understanding the Demo Topology
o-du-002 — Understanding the Demo Topology
o-ran — Glossary
o-ran-fm — External Alarms
o-ran-performance-management — External Performance Data
o-ran-sc — Lab Simulator Checks
o-ru — Adding an External NF, Creating the Demo Network, Example: Adding an External O-RU
o-ru measurements — External Performance Data
o1 — Glossary, What is NETCONF?
o2 — Known Limitations
old alarms — Alarm History
open file — Application Layout
openapi — Northbound REST API
operational state — Running GET
operations — Storage Cleanup
operations report — Reports and System Health
operator — Users and Roles
out of order — External Alarms
outage — Running Scenarios
outbound — Communication Matrix
overview — What Kaldrivon MiniSMO Is
password — Adding an External NF, SSH Authentication Methods, Users and Roles
passwords — Security Notes
past alarms — Alarm History
pause — Running Scenarios
payload — NETCONF Log Retention
pci — Configuration Management
pdf — Exporting Capacity Reports, Exporting Storage Reports
perf3gpp — External Performance Data, VES Collector
perfmetricjob — External Performance Data
performance — Running a Capacity Test, Understanding Technical Capacity
pm — About PM Simulation, External Performance Data, Glossary, Performance Management
pm files — Measurement Files (File-based PM)
pm sources — Method Choice and Fallback
pm storage — PM Aggregation, PM Retention, Understanding MiniSMO Data Storage, Why PM Data Can Grow Quickly
pnfregistration — VES Collector
point nf — Pointing an NF at the Collector
polling — Method Choice and Fallback, Subscription States
port — Why Port 830?
port 4334 — NETCONF Call Home
port 830 — Adding an External NF, NETCONF Connection Requirements, NETCONF Connection Troubleshooting, Why Port 830?
port 8765 — Backend Startup Problems, Local API Protection
port list — Communication Matrix
ports — Communication Matrix
prb — Performance Management
preference — Method Choice and Fallback
preferences — Settings
presets — Changing Retention Settings
private key — SSH Authentication Methods

pro — Analytics, Approvals, Automation, Bulk Operations, Configuration Templates, Desired State and Drift, Diagnostic Collection, Editions and Capacity, Investigate, Northbound REST API, Rollback, Schedules, Software Upgrade Campaigns, Webhooks
pro operations — Settings
problem — Troubleshooting
programdata — Running as a Windows Service
provisioning — Pointing an NF at the Collector
public key — SSH Authentication Methods
purge — Storage Cleanup
purge data — Storage Cleanup
purpose — What Kaldrivon MiniSMO Is
pyang — Importing a YANG Model
r1 — O-RAN R1 Interface
ram — Running a Capacity Test, Understanding Technical Capacity
ramp — Running Scenarios
rapp — Known Limitations, O-RAN R1 Interface
rate — External Performance Data
rate limit — What a License Changes
raw — Viewing Raw NETCONF XML
raw pm — PM Retention
reachability — NETCONF Connection Requirements
read only — Write Protection
read-only — How MiniSMO Writes to External NFs, Write Protection
receiving — Subscription States
recommended — Understanding Technical Capacity
reconnecting — Subscription States
redacted — External Configuration History
refused callers — NETCONF Call Home
region — Inventory and Network Tree
registry — YANG Model Registry
remote — Using MiniSMO in a Browser
remove — Uninstalling
remove license — Installing a License
remove ves destination — Pointing an NF at the Collector
replay — Capability Discovery, Subscription States
report — Exporting Capacity Reports, Exporting Storage Reports, Reports and System Health
request upload — Measurement Files (File-based PM)
requirements — NETCONF Connection Requirements, System Requirements
reset — Creating the Demo Network, External Performance Data, Reset MiniSMO, Running Scenarios
reset software — Reset MiniSMO
rest — Northbound REST API
restart — Running as a Windows Service
restore — Backup Considerations, Rollback
resync — Monitoring External NFs
resync alarms — Monitoring External NFs
retention — Alarm History, Automatic Cleanup, Capacity vs Storage Constraints, Changing Retention Settings, FM Retention, NETCONF Log Retention, PM Retention, Performance Management
retention impact estimator — Storage Forecast
retrieve-file-list — Measurement Files (File-based PM), Write Protection
retry failed — Measurement Files (File-based PM)
reverse ssh — NETCONF Call Home
revert — Configuration Management
revision — NETCONF Capability Discovery, YANG Model Registry
revisions — Configuration History
rfc 5277 — Capability Discovery
rfc 6241 — What is NETCONF?
rfc 6242 — Why Port 830?
rfc 6470 — Capability Discovery, External Configuration History
rfc 8071 — NETCONF Call Home
rfc 8525 — YANG Library Discovery
rfc 8639 — Capability Discovery
roles — Users and Roles
rollback — Changing External Configuration, Rollback
root cause — Investigate
rpc — Running GET, What is NETCONF?
rpc-error — Changing External Configuration
rule — Automation
rules — Automation
run — Running Scenarios
run alarm correlation — Analytics
run kpi analysis — Analytics
running — Running GET-CONFIG, Running, Candidate and Startup Datastores

rx_on_time — External Performance Data
safeguards — What a License Changes
scenario — Running Scenarios
schedule — Automatic Cleanup, Schedules
scheduled backup — Schedules
scheduled report — Schedules
scope — Version 0.4 Scope
search — NETCONF Log, Search
search alarms — Alarm History
security — Local API Protection, Security Notes
serial — NF Identity and Cells, Search
serial number — NETCONF Call Home
server — Local and Team Mode
service — Running Without the Desktop App, Running as a Windows Service
service discovery — O-RAN R1 Interface
session id — NETCONF Log
session-id — The Hello Exchange
settings — Settings
setup — Installation
setup.exe — Installation
severity — About Simulated Alarms, Fault Management
sftp — Measurement Files (File-based PM)
sha256 — Installation
shortcuts — Application Layout
show in folder — Application Layout
sign in — Users and Roles, Using MiniSMO in a Browser
sign out — Running as a Windows Service
signature invalid — Licensing Troubleshooting
simulated — Creating the Demo Network, Network Function Inventory
simulated alarm activity — Settings
simulation — About PM Simulation
simulator — Creating the Demo Network, Lab Simulator Checks, What Kaldrivon MiniSMO Is
site — Inventory and Network Tree
site / region — Network Function Inventory
small deployment — Community Edition and Licensing
smartscreen — Installation
sme — O-RAN R1 Interface
smo — Glossary, What Kaldrivon MiniSMO Is
snapshot — Configuration Backups, External Configuration History
software — Software Upgrade Campaigns
software version — Software Upgrade Campaigns
source name — Matching Events to NFs
sourceid — Matching Events to NFs
sourcename — Matching Events to NFs
sources — Method Choice and Fallback
ssh — Confirming the SSH Host Key, NETCONF Connection Test, NETCONF over SSH, SSH Authentication Methods
ssh fingerprint — Confirming the SSH Host Key
ssh key — SSH Authentication Methods
stages — NETCONF Connection Test
stale — Subscription States
standard — Changing Retention Settings
start monitoring — Monitoring External NFs
start over — Reset MiniSMO
startup — Backend Startup Problems, First Launch, Running, Candidate and Startup Datastores
startup troubleshooting — Backend Startup Problems
status — Dashboard
stop — Monitoring External NFs, Running Scenarios
stop monitoring — Monitoring External NFs
stop sending — Pointing an NF at the Collector
storage — Capacity & Storage Overview, Capacity vs Storage Constraints, Changing Retention Settings, Data Storage Locations, Exporting Storage Reports, Storage Capacity, Storage Forecast, Storage Warnings, Understanding MiniSMO Data Storage, Why PM Data Can Grow Quickly
storage and retention — Changing Retention Settings
storage forecast — Storage Forecast
storage management — Storage Capacity
subscribe — Monitoring External NFs
subscription — Monitoring External NFs
subscriptions — Monitoring External NFs
subsystem — NETCONF Connection Troubleshooting, NETCONF over SSH
subtree filter — Running GET
summary — Dashboard
supervision — Monitoring External NFs
support — Diagnostic Collection, Support Information, Troubleshooting
supported — Community Edition and Licensing
sync — External Alarms

synthetic — About PM Simulation
synthetic data — Introduction
system capacity — Capacity & Storage Overview
system health — Reports and System Health
tabs — Application Layout
team — Local and Team Mode
technical capacity — License Capacity vs Technical Capacity
template — Configuration Templates
templates — Configuration Templates
terms — Glossary
test — Lab Simulator Checks
test connection — NETCONF Connection Test
test-netconnection — NETCONF Connection Requirements
theme — Settings
third-party — About MiniSMO
throughput — Performance Management
tiles — Dashboard
timeout — Settings
token — Northbound REST API
toolbar — Network Function Inventory
topology — Topology View, Understanding the Demo Topology
tree — Inventory and Network Tree, Viewing Raw NETCONF XML
trigger — Automation
troubleshoot — Investigate
troubleshooting — Licensing Troubleshooting, NETCONF Connection Troubleshooting, Troubleshooting
truncated — NETCONF Log Retention
trust on first use — Confirming the SSH Host Key
tx power — Configuration Management
unassigned — Topology View
unattributed — Matching Events to NFs
unauthorized — Local API Protection
uninstall — Uninstalling
unlicensed — Expired License, Downgrades and Unlicensed NFs
unlicensed nf — Expired License, Downgrades and Unlicensed NFs
unlimited — Community Edition and Licensing
unsupported — Subscription States
update — Software Updates
update channel — Settings
upgrade — Editions and Capacity, Software Updates, Software Upgrade Campaigns
uri — NETCONF Capability Discovery
users — Users and Roles
uuid — Identity Mapping
vacuum — Understanding MiniSMO Data Storage
validate — Changing External Configuration
vendor — NF Identity and Cells
version — About MiniSMO, Software Updates, The Hello Exchange
version 0.3 — Version 0.4 Scope
versions — Software Upgrade Campaigns
ves — Capability Discovery, Matching Events to NFs, Pointing an NF at the Collector, VES Collector
ves 7.2.1 — VES Collector
ves card — Monitoring External NFs
ves collector — Settings
ves endpoint — Pointing an NF at the Collector
ves-endpoint — Pointing an NF at the Collector
viewer — Users and Roles
vswr — About Simulated Alarms
wal — Understanding MiniSMO Data Storage
walkthrough — Example: Adding an External O-RU
warning — Storage Warnings
web — Local and Team Mode
web ui — Using MiniSMO in a Browser
webhook — Webhooks
webview2 — System Requirements
wg4 — External Performance Data
what is included — Version 0.4 Scope
what needs attention — Dashboard
whitelist — NETCONF Call Home
who — Audit Log
windows 11 — System Requirements
windows service — Running as a Windows Service
wipe — Reset MiniSMO
write — How MiniSMO Writes to External NFs, Write Protection
x-minismo-key — Local API Protection
xml — Viewing Raw NETCONF XML, What is NETCONF?
yang — Glossary, Importing a YANG Model, NETCONF Capability Discovery, YANG Model Registry
.yang — Importing a YANG Model

yang constraints — Changing External Configuration

yang library — YANG Library Discovery

yang-push — Capability Discovery, External Performance Data, Method Choice and Fallback

zip — Diagnostic Collection